目录

动态规划 **4**

动态规划 - SOS DP (高维前缀和) . . . 4

图论 **4**

图论 - Kruskal 重构树 (瓶颈路问题) . . . 4
图论 - 匈牙利算法 (二分图最大匹配) . . . 5
图论 - 最小生成树 (MST) - Kruskal 算法 . . . 6
最小生成树 - Prim 算法 (堆优化版) . . . 7
图论与树 - DFS 序 (DFN) . . . 8
数据结构/图论 - 树上启发式合并 (Dsu on Tree) . . . 8
数据结构/图论 - 重链剖分 (树链剖分 / HLD) . . . 10
图论 - 欧拉路径与欧拉回路 (Eulerian Path/Circuit) . . . 11
网络流 - 最小费用流 (SPFA 增广) . . . 12
图论 - 网络流 Dinic (最大流) . . . 13
连通 - Kosaraju (强连通分量与缩点) . . . 15
图论 - Tarjan 强连通分量 (SCC) 与缩点 . . . 16
图论 - 割点与割边 (桥) (Articulation Points & Bridges) . . . 16
连通 - 圆方树 (关闭路口后的连通性) . . . 17

字符串 **19**

字符串 - AC 自动机 (Aho-Corasick Automaton) . . . 19
字符串-KMP . . . 20
字符串 - Z 函数 (扩展 KMP) . . . 20
字符串 - Manacher 算法 (马拉车) . . . 20
字符串 - 后缀数组 (倍增与计数排序) . . . 21
字符串 - 后缀自动机 (SAM) . . . 22
字符串 - 字典树 (Trie) . . . 24

数学 **25**

博弈论 - SG 函数与 Sprague-Grundy 定理 . . . 25
多项式 - FFT (快速傅里叶变换) . . . 26

多项式 - NTT (快速数论变换) . . . 27
数学 - 卢卡斯定理与扩展卢卡斯定理 (Lucas & exLucas) . . . 28
数论 - 中国剩余定理与扩展 (CRT & EXCRT) . . . 30
数论 - 欧拉函数 & 扩展欧拉定理 . . . 31
数论 - 莫比乌斯反演 . . . 32
数论 - 整除分块 (数论分块) . . . 33
数学 - 矩阵运算全家桶 (无重载 N*M 版) . . . 33
矩阵与递推 - 矩阵快速幂 . . . 35
数据结构 - 异或线性基 (XOR Linear Basis) . . . 35
组合数学 - 卡特兰数 (Catalan Number) . . . 36
组合数学 - 多重集组合数 . . . 37
组合数学 - 广义卡特兰数 . . . 38
组合数学 - k 部分拆数 (k-Part Integer Partition) . . . 38
组合数学 - 互异分拆数 (Distinct Integer Partition) . . . 39
数学 - 斐波那契快速倍增法 (Fast Doubling) . . . 40
组合数学 - 斯特林数 (Stirling Number) . . . 40
数学 - 范德蒙德卷积 (Vandermonde's Convolution) . . . 41
数学 - 皮萨诺周期 (Pisano Period) 与超大数降维 . . . 42
组合数学 - 错位排列 (Derangement) . . . 43
数论 - 线性筛质数 (欧拉筛) . . . 43
数学 - 高斯消元 (高斯-若尔当消元法) . . . 44

数据结构 **45**

数据结构 - CDQ 分治 (三维偏序) . . . 45
数据结构 - ST 表 (RMQ) . . . 47
分块 - 区间加 & 区间秩查询 . . . 47
分块 - 回滚莫队 (只增不删) . . . 48
分块 - 莫队 (离线区间查询) . . . 49
数据结构 - 带删除并查集 . . . 50
数据结构 - 带权并查集 (Weighted DSU) . . . 50
数据结构/图论 - LCA 转 RMQ (欧拉序/DFN + ST 表) . . . 51
树 - 最近公共祖先 (LCA) - 倍增法 . . . 52

数据结构 - 离散化权值树状数组 (支持查排名/第 k 小/前驱/后继) . . . 53
树 - 笛卡尔树 . . . 54
线段树 - 主席树 (可持久化权值线段树) . . . 54
数据结构 - 懒标记线段树 . . . 55

杂项 **56**
杂项 - 多进程模板 . . . 56
数据结构 - 珂朵莉树 (ODT / Old Driver Tree) . . . 56

算法基础 **57**
算法 - 三分查找 (二分斜率/导数逼近) . . . 57
算法基础 - 点差分 . . . 58
算法基础 - 边差分 . . . 59
基础算法 - 高精度运算 (BigInt) . . . 60

计算几何 **61**
计算几何 - 二维点与向量基础 (2D Geometry) . . . 61
计算几何 - 凸包 Graham 扫描法 (Convex Hull - Graham Scan) . . . 62
计算几何 - 圆的交点与切线全家桶 (无重载运算符纯净版) . . . 63
数据结构 - 扫描线求矩形面积并 . . . 65
计算几何 - 线段与射线交点全家桶 (无重载运算符纯净版) . . . 66
计算几何 - 简单多边形面积 (Polygon Area) . . . 67

动态规划

动态规划 - SOS DP (高维前缀和)

算法说明

- **用途**：求每个集合的所有子集权值之和，即 $w[k] = \sum_{s \subseteq k} a[s]$，其中 a 为原始权值。
- **核心思想**：外层枚举位 `j`，内层枚举状态 `k`。若第 `j` 位为 1，执行 `w[k]+=w[k^(1LL<<j)]`，加入删除这一位后的贡献。
- **使用方法**：将原始权值填入 `w[0..(1LL<<m)-1]`，转移后 `w[k]` 即为答案，包含空集和自身。
- **时间复杂度**：$O(m2^m)$。
- **空间复杂度**：$O(2^m)$，固定数组实际按最大位数 `M` 分配 $O(2^M)$ 空间。

File: sosdp.cpp

```cpp
#include<bits/stdc++.h>
using namespace std;
#define int long long
#define endl '\n'
const int M=20;
int w[1LL<<M]; // 开始存原始数值，结束存每个下标的所有子集之和
void solve()
{
    int m;
    cin>>m; // 二进制位数，一共有2^m个下标，m<=M
    for(int k=0;k<(1LL<<m);k++)cin>>w[k]; // 初始时，每个位置只有自己的数值
    for(int j=0;j<m;j++) // 逐位处理：本轮允许删掉第j位的1
    {
        for(int k=(1LL<<m)-1;k>=0;k--) // 遍历所有下标，完成这一轮更新
        {
            if((k&(1LL<<j))!=0) // 第j位是1，才有保留和删除两种情况
            {
                w[k]+=w[k^(1LL<<j)]; // w[k]已有保留这一位的情况，再加入删除这一位的情况
            }
        }
    }
    for(int k=0;k<(1LL<<m);k++)cout<<w[k]<<" ";
    // 所有位处理完，输出每个下标的子集和
    cout<<endl;
}
signed main()
{
    cin.tie(0);
    ios::sync_with_stdio(0);
    solve();
    return 0;
}
```

图论

图论 - Kruskal 重构树 (瓶颈路问题)

算法说明

- **用途**：求无向图中两点间所有路径的**最大边权最小**（或**最小边权最大**）的瓶颈值。
- **核心性质**：
  1. **建树方式**：Kruskal 连边时，新建虚拟节点作为两连通块根的父节点，点权赋为该边权。
  2. **树的形态**：二叉树（原图不连通时为森林）。节点总数 $2N-1$（叶子为原图点，内部节点为原图边）。
  3. **堆性质**：最大（小）生成树对应的重构树是小（大）根堆。
  4. **关键结论**：两点间路径的瓶颈最值，等于它们在重构树上 **LCA 的点权**。
- **时间复杂度**：预处理 $O(M \log M)$，单次查询 $O(\log N)$。
- **注意事项（避坑指南）**：
  1. **空间翻倍**：所有点相关数组（并查集、邻接表、深度、倍增等）必须开到 $2N$。
  2. **不连通处理**：原图可能是森林，建树后需遍历 $1 \sim id$，对所有 `find(i) == i` 的根节点做 DFS。
  3. **查询前提**：查 LCA 前必须判断 `find(u) == find(v)`，不连通则无解。

File: Kruskal 重构树.cpp

```cpp
#include<bits/stdc++.h>
using namespace std;

#define int long long
#define endl '\n'
#define iint __int128_t
using PII = pair<int,int>;
using PSI= pair<string,int>;
using tup= tuple<int,int,int>;

// 注意：空间必须开 2*N
const int N = 2e5+10;
const int INF = 1e9;
int fe[N], va[N], de[N], f[N][20];
vector<int> g[N];
int find(int x) {
    return fe[x] == x ? x : fe[x] = find(fe[x]);
}

void dfs(int u, int fa) {
    de[u] = de[fa] + 1;
    f[u][0] = fa;
    for(int i = 1; i < 20; i++) f[u][i] = f[f[u][i-1]][i-1];
    for(int v : g[u]) {
        if(v != fa) dfs(v, u);
```

```
    }
}

int lca(int a, int b) {
    if(de[a] < de[b]) swap(a, b);
    for(int i = 19; i >= 0; i--)
        if(de[f[a][i]] >= de[b]) a = f[a][i];
    if(a == b) return a;
    for(int i = 19; i >= 0; i--)
        if(f[a][i] != f[b][i]) a = f[a][i], b = f[b][i];
    return f[a][0];
}

void solve() {
    int n, m;
    if(!(cin >> n >> m)) return;

    // 1. 初始化
    for(int i = 0; i <= 2 * n; i++) {
        g[i].clear();
        fe[i] = i;
        va[i] = de[i] = 0;
        for(int j = 0; j < 20; j++) f[i][j] = 0;
    }

    // 2. 读边
    vector<tup> ar(m);
    for(int i = 0; i < m; i++) {
        int u, v, w;
        cin >> u >> v >> w;
        ar[i] = {w, u, v};
    }

    // 3. 排序 (求最小边权最大用 rbegin 降序，最大边权最小用 begin 升序)
    sort(ar.rbegin(), ar.rend());

    // 4. Kruskal 建树
    int id = n;
    for(auto [w, u, v] : ar) {
        int fu = find(u), fv = find(v);
        if(fu != fv) {
            va[++id] = w;
            fe[fu] = fe[fv] = id; // 旧根连向新根
            g[id].push_back(fu);
            g[id].push_back(fv);
        }
    }

    // 5. 森林 DFS 预处理
    for(int i = 1; i <= id; i++) {
        if(find(i) == i) dfs(i, 0);
    }

    // 6. 处理询问
    int q;
    cin >> q;
    while(q--) {
        int x, y;
        cin >> x >> y;
        if(find(x) != find(y)) cout << -1 << endl;
        else cout << va[lca(x, y)] << endl;
    }
}
```

图论 - 匈牙利算法 (二分图最大匹配)

何时使用

解决经典的“分配/配对”问题（如：男女相亲、工人分配任务）。图**必须是二分图**，核心特征是求**最大匹配数**。

复杂度

- **时间复杂度**：$O(V \times E)$。其中 V 是左侧集合的点数，E 是总边数。由于很难跑满最坏情况，实际运行速度往往远快于理论上界。
- **空间复杂度**：$O(V + E)$（使用 vector 邻接表存图）。

核心步骤

1. **单向建图**：只需要从左侧集合连向右侧集合。
2. **find(u) 寻找增广路**：遍历左侧节点 u 看中的右侧节点 v。如果 v 还没被访问，标记访问。如果 v 没对象，或者 v 的现任对象 match[v] 能**腾出位置（递归寻找）**，则牵手成功，match[v]=u。
3. **清空 vis**：每次给新的左侧节点找对象前，**必须清空** vis 数组。

File: 二分图最大权匹配.cpp

```
#include<bits/stdc++.h>
using namespace std;

#define int long long
#define endl '\n'

const int MAXN=510;

vector<int> e[MAXN]; // e[u]: 左侧节点 u 指向的右侧节点集合
int match[MAXN];     // match[v]=u: 右侧节点 v 当前匹配的左侧节点是 u
bool vis[MAXN];      // vis[v]: 右侧节点 v 在本轮寻找中是否被尝试过

// ================== 核心：寻找增广路 ==================
bool find(int u)
{
    for(int v:e[u])
    {
        if(!vis[v])
```

```
        {
            vis[v]=true; // 标记为已访问，防止死循环
            // 如果 v 还没有对象，或者 v 的现任能腾出位置去找别人
            if(!match[v]||find(match[v]))
            {
                match[v]=u; // 牵手成功
                return true;
            }
        }
    }
    return false;
}

void solve()
{
    /* 实战调用示例
    int n1,n2,m;
    cin>>n1>>n2>>m;
    for(int i=1;i<=m;i++)
    {
        int u,v; cin>>u>>v;
        e[u].push_back(v); // 只需要建单向边：左 -> 右
    }

    int ans=0;
    for(int i=1;i<=n1;i++)
    {
        memset(vis,0,sizeof vis); // 【极其重要】：每次给新左侧节点找对象前，必须清空 vis!
        if(find(i))ans++;
    }
    cout<<ans<<endl; // 输出最大匹配数
    */
}
```

```
signed main()
{
    cin.tie(0);
    ios::sync_with_stdio(0);
    solve();
    return 0;
}
```

图论 - 最小生成树 (MST) - Kruskal 算法

算法说明

- **用途**：在无向连通图中寻找一棵包含所有顶点且边权之和最小的树。
- **时间复杂度**：$O(M \log M)$（M 为边数，瓶颈在边排序）。
- **适用场景**：稀疏图（边数较少）。基于贪心思想，配合并查集，代码极简，是比赛中最常用的求 MST 算法。
- **注意事项**：
  1. 必须在读入边之前或之后，初始化并查集 `p[i] = i`。
  2. 如果最终加入的边数不等于 $N-1$，说明图不连通，不存在最小生成树。
  3. 累加的边权总和极易爆 `int`，务必使用 `long long` 存储 `res`。

File: 最小生成数 Kruskal.cpp

```
#include<bits/stdc++.h>
using namespace std;
#define int long long
#define endl '\n'
// 将边权 w 放在第一个位置，利用 tuple 默认字典序实现自带按边权排序
using Edge = tuple<int, int, int>;
void solve()
{
    int n, m;
    cin >> n >> m;
    vector<Edge> edges(m);
    vector<int> p(n + 1); // 并查集父节点数组 (原 fe)
    // lambda 并查集寻根函数（路径压缩）
    function<int(int)> find = [&](int x)->int
        {
            if(p[x] != x)p[x] = find(p[x]);
            return p[x];
        };

    // 1. 初始化并查集
    for(int i = 1;i <= n;i++)p[i] = i;
    // 2. 读入边，存入顺序为 {边权, 端点u, 端点v}
    for(int i = 0;i < m;i++)
    {
        int u, v, w;
        cin >> u >> v >> w;
        edges[i] = { w,u,v };
    }

    // 3. 按边权从小到大排序 (tuple 默认比对第一个元素)
    sort(edges.begin(), edges.end());

    int res = 0; // 记录最小生成树总边权 (原 ans)
    int cnt = 0; // 记录成功加入生成树的边数

    // 4. Kruskal 贪心加边
    for(int i = 0;i < m;i++)
    {
        auto [w, u, v] = edges[i]; // C++17 结构化绑定拆包
        u = find(u), v = find(v);
        if(u != v)
        {
            p[u] = v;
            res += w;
```

```
            cnt++; // 成功连边，计数器加 1
        }
    }

    // 5. 判断连通性
    // 原代码通过再跑一遍 O(N) 循环判所有点祖先是否相同。
    // 更优解法：一棵 n 个点的树必定刚好有 n-1 条边，直接判断 cnt 即可。
    if(cnt < n - 1)
    {
        cout << "orz" << endl; // 无法连通的情况
    }
    else
    {
        cout << res << endl;
    }
}
```

最小生成树 - Prim 算法 (堆优化版)

算法说明

用途：求无向图的最小生成树 (MST) 的权值之和。

时间复杂度：O(ElogE) 或 O(ElogV)。

适用场景：适合稠密图。对于稀疏图，Kruskal 算法通常代码更短，但堆优化的 Prim 性能也十分优秀。

File: 最小生成树 prim.cpp

```
#include<bits/stdc++.h>
using namespace std;

const int N = 5050, M = 2e5 + 10;
const int INF = 0x3f3f3f3f;

// 链式前向星存图
struct E {
    int v, w, x;
} e[M * 2]; // 无向图记得开两倍空间

int n, m, h[N], cnte;
int dis[N];     // 存储点到当前生成树集合的最小距离
bool vis[N];    // 标记点是否已加入生成树
int res = 0;    // 最小生成树权值和
int cnt = 0;    // 已加入生成树的点数量

void adde(int u, int v, int w) {
    e[++cnte] = E{ v, w, h[u] };
    h[u] = cnte;
}

struct S {
    int u, d;
    // 优先队列小根堆，距离小的优先
    bool operator<(const S& x) const {
        return d > x.d;
    }
};

priority_queue<S> q;

// 返回 true 表示成功生成 MST，false 表示图不连通
bool Prim() {
    // 初始化
    memset(dis, 0x3f, sizeof(dis));
    memset(vis, 0, sizeof(vis));
    res = 0;
    cnt = 0;

    // 从 1 号点开始
    dis[1] = 0;
    q.push({ 1, 0 });

    while(!q.empty()) {
        // 优化：如果已经选了 n 个点，说明生成树构建完成
        if(cnt >= n) break;

        int u = q.top().u;
        int d = q.top().d;
        q.pop();

        if(vis[u]) continue;
        vis[u] = 1;
        res += d;
        cnt++;

        for(int i = h[u]; i; i = e[i].x) {
            int v = e[i].v;
            int w = e[i].w;
            // 如果通过 u 到达 v 的距离比 v 当前记录的距离更短
            if(w < dis[v]) {
                dis[v] = w;
                q.push({ v, w });
            }
        }
    }

    // 判断是否所有点都在生成树内
    return cnt == n;
}

/* 使用示例:
 cin >> n >> m;
 for(int i=1; i<=m; i++){
     int u,v,w; cin>>u>>v>>w;
     adde(u,v,w); adde(v,u,w);
 }
 if(Prim()) cout << res << endl;
 else cout << "impossible" << endl;
```

```
*/
```

图论与树 - DFS 序 (DFN)

算法说明

- **用途**：将树结构“拍扁”为一维线性数组，把“子树操作”完全转化为“连续数组区间操作”，无缝衔接线段树或树状数组。
- **时间复杂度**：预处理 $O(N)$，获取子树区间及祖孙判定 $O(1)$。
- **核心思想**：DFS 遍历时分配递增时间戳。节点 u 及其整棵子树分配到的 DFN 必定是连续闭区间：**[dfn[u], dfn[u]+sz[u]-1]**。
- **常见场景**：
  1. 子树权值批量更新/求和。
  2. $O(1)$ 判断祖孙关系（v 在 u 的管辖区间内，则 u 是 v 的祖先）。
  3. 树上启发式合并 / 树上莫队。

File: 树-dfn.cpp

```cpp
#include<bits/stdc++.h>
using namespace std;
#define int long long
using PII = pair<int,int>;

const int MAXN=2e5+5;
vector<int> adj[MAXN];
int dfn[MAXN],sz[MAXN];
int timer;

void dfs(int u,int p)
{
    dfn[u]=++timer;
    sz[u]=1;
    for(int v:adj[u])
    {
        if(v!=p)
        {
            dfs(v,u);
            sz[u]+=sz[v];
        }
    }
}

/* 核心调用：
 * timer=0; dfs(root,0);
 * * 1. 子树区间：
 * int L=dfn[u], R=dfn[u]+sz[u]-1;
 * * 2. 祖先判定 (u是否为v的祖先)：
 * bool ok=(dfn[u]<=dfn[v] && dfn[v]<=dfn[u]+sz[u]-1);
 */
```

数据结构/图论 - 树上启发式合并 (Dsu on Tree)

算法说明

- **用途**：用于离线解决**无修改的子树查询问题**（如：求每个子树内出现次数最多的颜色之和、子树内某种特征的节点数量等）。
- **时间复杂度**：$O(N \log N)$。直接暴力统计是 $O(N^2)$ 的，而 Dsu on Tree 利用了“重链剖分”的思想，把复杂度降到了极限。
- **核心思想：“轻儿子查完就删，重儿子查完保留”**。
  1. 跑一遍 DFS 求出每个节点的重儿子（子树节点数最多的儿子）。
  2. 再跑一遍 DFS 回答询问：
     - 先遍历所有的轻儿子，统计完答案后**立刻清除**它们对全局数组（如计数桶）的贡献。
     - 最后遍历重儿子，统计完答案后**保留**它的贡献。
     - 此时全局数组里已经有了重儿子的信息，我们再把当前节点和所有轻儿子重新暴力加进全局数组中，就能得到整棵子树的完整信息，记录答案。
     - 如果当前节点是它父亲的轻儿子，就把整棵树的贡献统统清除（为下一个兄弟节点腾出干净的全局数组）。

File: 树上启发式合并.cpp

```cpp
#include<bits/stdc++.h>
using namespace std;

#define int long long
#define endl '\n'

const int MAXN=1e5+10;

vector<int> e[MAXN]; // 存树的边
int col[MAXN];       // col[i]: 节点 i 的颜色
int sz[MAXN];        // sz[i]: 以 i 为根的子树大小
int son[MAXN];       // son[i]: 节点 i 的重儿子编号
int ans[MAXN];       // ans[i]: 存储节点 i 的最终查询答案

// --- 全局统计变量 ---
int cnt[MAXN]; // cnt[c]: 颜色 c 出现的次数桶
int maxc;      // maxc: 当前子树中出现次数最多的颜色的【频次】
int sum;       // sum: 当前子树中所有达到最大频次的【颜色编号之和】
int Son;       // Son: 标记当前的重儿子，在暴力统计时遇到它直接跳过，避免重复计算

// --- 1. 第一遍 DFS：寻找重儿子 ---
void dfs1(int u,int fa)
{
    sz[u]=1;
```

```
    for(int v:e[u])
    {
        if(v==fa)continue;
        dfs1(v,u);
        sz[u]+=sz[v];
        // 如果子节点的 sz 大于当前记录的重儿子的 sz，则更新重儿子
        if(sz[v]>sz[son[u]]) son[u]=v;
    }
}

// --- 2. 暴力统计/清除函数 ---
// val = 1 表示增加贡献，val = -1 表示清除贡献
void add(int u,int fa,int val)
{
    cnt[col[u]]+=val; // 更新颜色桶

    // 如果是增加贡献，顺便维护我们要求的答案 maxc 和 sum
    if(val>0)
    {
        if(cnt[col[u]]>maxc)
        {
            maxc=cnt[col[u]]; // 频次打破了记录，更新最高频次
            sum=col[u];       // 最高频次的颜色变了，sum 直接被新颜色覆盖
        }
        else if(cnt[col[u]]==maxc)
        {
            sum+=col[u];      // 频次并列第一，累加颜色编号
        }
    }

    // 递归处理子树
    for(int v:e[u])
    {
        // 【核心剪枝】：如果 v 是重儿子，说明它已经在桶里了，绝对不能再暴力算一遍！
        if(v==fa||v==Son)continue;
        add(v,u,val);
    }
}

// --- 3. 第二遍 DFS：树上启发式合并 ---
// keep: 标记当前节点 u 是否是它父亲的重儿子 (1:保留贡献, 0:清空贡献)
void dfs2(int u,int fa,bool keep)
{
    // 第一步：先处理所有的【轻儿子】 (查完后清除它们的贡献)
    for(int v:e[u])
    {
        if(v==fa||v==son[u])continue;
        dfs2(v,u,0); // keep = 0，查完即毁
    }

    // 第二步：处理唯一的【重儿子】 (查完后保留它的贡献)
    if(son[u])
    {
        dfs2(son[u],u,1); // keep = 1，查完保留
        Son=son[u];       // 标记当前重儿子，防止下一步 add 时把它重复算进去
    }

    // 第三步：此时桶里只有重儿子的信息。
    // 我们暴力把当前节点 u 和所有轻儿子的信息加进桶里，凑齐整棵子树的信息
    add(u,fa,1);

    Son=0; // 【必须重置】用完就扔，防止影响退栈后它父亲的计算

    // 第四步：记录当前子树的答案
    ans[u]=sum;

    // 第五步：如果当前节点 u 是它父亲的【轻儿子】，那它查完后也必须挥刀自宫，清空全家老小
    if(!keep)
    {
        add(u,fa,-1); // val = -1 撤销计数
        maxc=sum=0;   // 【必须重置】全局极值变量
    }
}

void solve()
{
    /* 实战调用示例
    int n; cin>>n;
    for(int i=1;i<=n;i++)cin>>col[i];
    for(int i=1;i<n;i++)
    {
        int u,v; cin>>u>>v;
        e[u].push_back(v);
        e[v].push_back(u);
    }

    dfs1(1,0);   // 第一遍找出重儿子
    dfs2(1,0,1); // 第二遍启发式合并得出答案

    for(int i=1;i<=n;i++) cout<<ans[i]<<" ";
    cout<<endl;
    */
}
```

数据结构/图论 - 重链剖分 (树链剖分 / HLD)

何时使用

树上**带修改**的路径/子树操作（如：u 到 v 路径集体加 k、求子树权值和）。

核心步骤

1. **dfs1**：求深度 dep、父亲 fa、子树大小 sz，并选出重儿子 son。
2. **dfs2**：连重链，求顶点 topf，分配连续的 DFS 序 dfn（**必须优先走重儿子**）。
3. **build**：用映射后的 dfn 数组建立线段树。
4. **路径操作**：比较两点所在重链顶点 topf 的深度，**深者先往上跳**，边跳边用线段树处理该段重链区间。

File: 树链剖分.cpp

```cpp
#include<bits/stdc++.h>
using namespace std;

#define int long long
#define endl '\n'

const int MAXN=1e5+10;

vector<int> e[MAXN];
int w[MAXN];    // w[i]: 原图中节点 i 的初始权值
int wt[MAXN];   // wt[i]: 映射后，DFS序为 i 的位置对应的权值 (用于线段树建树)

// --- 树链剖分核心数组 ---
int fa[MAXN], dep[MAXN], sz[MAXN], son[MAXN];
int topf[MAXN]; // 节点所在重链的顶端节点 (用于路径跳跃)
int dfn[MAXN];  // 树转数组的时间戳下标
int rnk[MAXN];  // 数组转树的逆映射
int timestamp;

// ================== 1. 树剖初始化 ==================
void dfs1(int u, int f, int d)
{
    fa[u]=f; dep[u]=d; sz[u]=1;
    int max_sz=-1;
    for(int v:e[u])
    {
        if(v==f)continue;
        dfs1(v,u,d+1);
        sz[u]+=sz[v];
        if(sz[v]>max_sz)
        {
            max_sz=sz[v];
            son[u]=v;
        }
    }
}

void dfs2(int u, int t)
{
    dfn[u]=++timestamp;
    rnk[timestamp]=u;
    wt[timestamp]=w[u];
    topf[u]=t;

    if(!son[u])return;
    // 【核心】：必须优先走重儿子，保证重链 DFS 序连续
    dfs2(son[u], t);

    // 轻儿子各自成为新重链的顶点
    for(int v:e[u])
    {
        if(v==fa[u]||v==son[u])continue;
        dfs2(v, v);
    }
}

// ================== 2. 线段树维护 ==================
int sum[MAXN*4], add[MAXN*4];

void pushup(int u){ sum[u]=sum[u<<1]+sum[u<<1|1]; }

void pushdown(int u, int l, int r)
{
    if(add[u])
    {
        int mid=l+r>>1;
        sum[u<<1]+=add[u]*(mid-l+1);
        sum[u<<1|1]+=add[u]*(r-mid);
        add[u<<1]+=add[u];
        add[u<<1|1]+=add[u];
        add[u]=0;
    }
}

void build(int u, int l, int r)
{
    if(l==r){ sum[u]=wt[l]; return; }
    int mid=l+r>>1;
    build(u<<1, l, mid);
    build(u<<1|1, mid+1, r);
    pushup(u);
}

void update_seg(int u, int l, int r, int L, int R, int k)
{
    if(L<=l&&r<=R){ sum[u]+=k*(r-l+1); add[u]+=k; return; }
    pushdown(u, l, r);
```

```
    int mid=l+r>>1;
    if(L<=mid)update_seg(u<<1, l, mid, L, R, k);
    if(R>mid)update_seg(u<<1|1, mid+1, r, L, R, k);
    pushup(u);
}

int query_seg(int u, int l, int r, int L, int R)
{
    if(L<=l&&r<=R)return sum[u];
    pushdown(u, l, r);
    int mid=l+r>>1, res=0;
    if(L<=mid)res+=query_seg(u<<1, l, mid, L, R);
    if(R>mid)res+=query_seg(u<<1|1, mid+1, r, L, R);
    return res;
}

// ================== 3. 树上路径与子树操作 ==================
// 路径修改
void update_path(int u, int v, int k, int n)
{
    while(topf[u]!=topf[v])
    {
        if(dep[topf[u]]<dep[topf[v]]) swap(u,v);
    // 谁深谁往上跳
        update_seg(1, 1, n, dfn[topf[u]], dfn[u], k); // 处理这一截重链
        u=fa[topf[u]]; // 跨越轻边
    }
    if(dep[u]>dep[v]) swap(u,v);
    update_seg(1, 1, n, dfn[u], dfn[v], k); // 处理最后在同一条重链上的部分
}

// 路径查询
int query_path(int u, int v, int n)
{
    int res=0;
    while(topf[u]!=topf[v])
    {
        if(dep[topf[u]]<dep[topf[v]]) swap(u,v);
        res+=query_seg(1, 1, n, dfn[topf[u]], dfn[u]);
        u=fa[topf[u]];
    }
    if(dep[u]>dep[v]) swap(u,v);
    res+=query_seg(1, 1, n, dfn[u], dfn[v]);
    return res;
}

// 子树修改
void update_tree(int u, int k, int n) {
    update_seg(1, 1, n, dfn[u], dfn[u]+sz[u]-1, k); }

// 子树查询
int query_tree(int u, int n) { return query_seg(1, 1, n, dfn[u], dfn[u]+sz[u]-1); }

void solve()
{
    /* 实战调用示例
    int n,m,root; cin>>n>>m>>root;
    for(int i=1;i<=n;i++) cin>>w[i];
    // ... 建图 e[u].push_back(v); ...

    dfs1(root,0,1);
    dfs2(root,root);
    build(1,1,n);

    // update_path(u,v,k,n);
    // cout<<query_path(u,v,n)<<endl;
    */
}

signed main()
{
    cin.tie(0); ios::sync_with_stdio(0);
    solve();
    return 0;
}
```

图论 - 欧拉路径与欧拉回路 (Eulerian Path/-Circuit)

算法说明

- **用途**：求解“一笔画”问题，即寻找一条恰好经过图中每条边一次的路径/回路。常用于扫雪路线规划、单词接龙、密码爆破（De Bruijn 序列）等。
- **时间复杂度**：$O(V+E)$。
- **核心思想 (Hierholzer 算法)**：
  1. **存在性判定**：欧拉回路要求所有点入度等于出度；欧拉路径要求仅有两个异常点（起点出度比入度大 1，终点入度比出度大 1）。
  2. **边走边删**：从起点 DFS，只要有未走过的边就往下走，走完立刻物理“删除”该边（使用当前弧优化 `cur` 数组防止超时）。
  3. **死胡同压栈**：当一个点无边可走时压入答案栈，最终将栈**逆序输出**即为正确路径。

File: 欧拉图和欧拉路径.cpp

```
#include <bits/stdc++.h>
using namespace std;
#define int long long
using PII = pair<int, int>;

const int MAXN = 2e5 + 5;
```

```cpp
vector<int> adj[MAXN];
int cur[MAXN];
int in_deg[MAXN], out_deg[MAXN];
vector<int> path;

// Hierholzer 核心 DFS
void dfs(int u)
{
    while(cur[u] < adj[u].size())
    {
        int v = adj[u][cur[u]++]; // 边走边删，指针后移
        dfs(v);
    }
    path.push_back(u); // 陷入死胡同，压入答案栈
}

void solve()
{
    int n, m;
    cin >> n >> m;

    // 1. 多组数据初始化清空
    for(int i = 1; i <= n; i++)
    {
        adj[i].clear();
        cur[i] = in_deg[i] = out_deg[i] = 0;
    }
    path.clear();

    // 2. 读入有向边建图
    for(int i = 1; i <= m; i++)
    {
        int u, v;
        cin >> u >> v;
        adj[u].push_back(v);
        out_deg[u]++;
        in_deg[v]++;
    }

    // 3. 字典序最小：对邻接表进行升序排序
    for(int i = 1; i <= n; i++)
    {
        sort(adj[i].begin(), adj[i].end());
    }

    // 4. 判定奇偶点，寻找起点
    int start_node = 1, odd_cnt = 0;
    bool valid = true;

    for(int i = 1; i <= n; i++)
    {
        if(in_deg[i] != out_deg[i])
        {
            odd_cnt++;
            if(out_deg[i] - in_deg[i] == 1)
start_node = i; // 路径的唯一合法起点
            else if(in_deg[i] - out_deg[i] != 1)
valid = false; // 度数差大于1，直接无解
        }
    }

    // 异常点只能是 0 个(回路) 或 2 个(路径)
    if(!valid || (odd_cnt != 0 && odd_cnt != 2))
    {
        cout << "No\n";
        return;
    }

    // 如果是欧拉回路(异常点为0)，没有绝对起点，随便找一个有出度的点作为起点
    if(odd_cnt == 0)
    {
        for(int i = 1; i <= n; i++)
        {
            if(out_deg[i] > 0)
            {
                start_node = i;
                break;
            }
        }
    }

    // 5. 跑核心算法
    dfs(start_node);

    // 6. 连通性判定：如果跑出来的点数不等于边数+1，说明图不连通，有边没走到
    if(path.size() != m + 1)
    {
        cout << "No\n";
        return;
    }

    // 7. 逆序输出即为最终的一笔画路径
    reverse(path.begin(), path.end());
    for(int i = 0; i < path.size(); i++)
    {
        cout << path[i] << " \n"[i == path.size() - 1];
    }
}
```

网络流 - 最小费用流 (SPFA 增广)

算法说明

- **用途**：求最大流，并使最大流的总费用最小；允许负费用边，初始网络无负费用环。
- **核心思想**：用 SPFA 寻找最小费用增广路，按路径最小剩余容量增广，反向边用于撤回并调整已有流量。

- **使用方法**：调用 `add(u,v,cap,cost)` 建图，再调用 `mcmf(n,s,t)`，返回 {最大流,最小费用}。点编号为 `1..n`，源汇不同。
- **时间复杂度**：$O((A+1)nm)$，A 为增广次数，m 为原图边数。
- **空间复杂度**：模板为 $O(N+m)$。

File: 网络流-最小费用流.cpp

```cpp
#include<bits/stdc++.h>
using namespace std;
#define int long long
#define endl '\n'

const int N=2e5+10,INF=1LL<<60;

struct E
{
    int v,cap,cost; // 终点、剩余容量、单位流量费用
};

vector<E>e; // 正反向边成对存储，反向边编号为id^1
vector<int>g[N]; // g[u]存从u出发的边编号
int dis[N],pre[N]; // 源点到各点的最小费用、最短路上进入各点的边编号
bool st[N]; // 点是否在SPFA队列中

void add(int u,int v,int cap,int cost) // 添加容量为cap、单位费用为cost的有向边
{
    int id=e.size();
    e.push_back({v,cap,cost}); // 正向边用于发送流量
    e.push_back({u,0,-cost}); // 反向边用于退回流量，同时退回对应费用
    g[u].push_back(id);
    g[v].push_back(id^1);
}

bool spfa(int n,int s,int t) // 在残量网络中寻找单位费用最小的增广路
{
    fill(dis+1,dis+n+1,INF);
    fill(st+1,st+n+1,0);
    queue<int>q;
    dis[s]=0;
    q.push(s);
    st[s]=1;

    while(!q.empty())
    {
        int u=q.front();
        q.pop();
        st[u]=0; // 出队后，距离再次变小时可以重新入队

        for(auto id:g[u])
        {
            int v=e[id].v;
            if(e[id].cap==0)continue; // 没有剩余容量，不能继续发送流量
            if(dis[v]>dis[u]+e[id].cost)
            {
                dis[v]=dis[u]+e[id].cost; // 找到到达v的更小费用
                pre[v]=id; // 记录进入v的边，之后沿路径回溯
                if(!st[v])
                {
                    q.push(v);
                    st[v]=1; // 已在队列中的点不重复加入
                }
            }
        }
    }
    return dis[t]!=INF; // 汇点可达，说明还能继续增广
}

pair<int,int>mcmf(int n,int s,int t) // 点编号1..n，源汇不同，初始残量网络无负费用环
{
    int flo=0,cost=0; // 累计流量、累计费用
    while(spfa(n,s,t))
    {
        int f=INF; // 本次增广量为路径上的最小剩余容量
        for(int u=t;u!=s;u=e[pre[u]^1].v) // 反向边的终点就是路径上的前一个点
            f=min(f,e[pre[u]].cap);

        for(int u=t;u!=s;u=e[pre[u]^1].v)
        {
            int id=pre[u];
            e[id].cap-=f; // 正向边消耗容量
            e[id^1].cap+=f; // 反向边增加容量，允许后续调整已有流量
        }

        flo+=f;
        cost+=f*dis[t]; // 本次费用=增广量×路径单位费用
    }
    return {flo,cost}; // 无法继续增广时，得到最大流及其最小费用
}
```

图论 - 网络流 Dinic (最大流)

使用方法

- **初始化**：用 `Dinic dinic(n)` 建立一张有 n 个点的网络，点编号为 $1 \sim n$；指定源点 `s` 和汇点 `t`。
- **加边**：对原图的每条有向边调用 `dinic.add(u,v,ca)`，表示从 `u` 到 `v` 最多通过 `ca` 单位流量。函数会自动添加初始容量为 0 的反向边，用于在增广时调整已有流量。
- **求最大流**：建图后调用 `dinic.maxflo(s,t)`，返回从源点到汇点的最大流。容量和流量使用 `long long`。
- **残量网络**：计算过程中，`e[id].cap` 保存该边的剩余容量；正向边和反向边的编号互为 `id ^ 1`。再次调用 `maxflo(s,t)` 会从当前残量网络继续增广，返回本次新增的流量。

```
Dinic dinic(n); // 初始化 n 个点
dinic.add(u,v,ca); // 对原图的每条有向边执行一次
int ans=dinic.maxflo(s,t); // 得到 s 到 t 的最大流
```

核心步骤

1. **BFS 分层**：通过剩余容量大于 0 的边计算 `dep`，得到源点到各点的层数。
2. **DFS 增广**：只沿层数增加 1 的边发送流量，同时减少正向边容量、增加反向边容量；`cur` 记录当前搜索位置，减少重复扫描。
3. **重复分层**：当前分层图无法继续发送流量后重新 BFS，直到汇点不可达。

复杂度

- **时间复杂度**：一般网络为 $O(V^2E)$，其中 V 为点数，E 为原图边数。
- **空间复杂度**：$O(V+E)$。

File: 网路流-dinic.cpp

```cpp
#include<bits/stdc++.h>
using namespace std;
#define int long long
#define endl '\n'
const int INF=1LL<<60;

struct Dinic
{
    struct E
    {
        int v,cap; // 终点、剩余容量
    };
    vector<E>e; // 正向边与反向边成对存储，编号互为 id^1
    vector<vector<int>>g; // g[u] 存储从 u 出发的边编号
    vector<int>dep,cur; // BFS 层数、DFS 当前搜索到的邻接表位置
    Dinic(int n)
    {
        g.resize(n+1); // 点编号为 1~n
        dep.resize(n+1);
        cur.resize(n+1);
    }
    void add(int u,int v,int ca) // 添加容量为 ca 的有向边 u->v
    {
        int id=e.size();
        e.push_back({v,ca}); // 正向边初始剩余容量为 ca
        e.push_back({u,0}); // 反向边用于退回已发送的流量
        g[u].push_back(id);
        g[v].push_back(id^1);
    }
    bool bfs(int s,int t) // 在残量网络上建立分层图
    {
        fill(dep.begin(),dep.end(),-1); // -1 表示尚未到达
        queue<int>Q;
        dep[s]=0;
        Q.push(s);
        while(Q.size())
        {
            int u=Q.front();
            Q.pop();
            for(auto id:g[u])
            {
                int v=e[id].v;
                if(e[id].cap==0||dep[v]!=-1)
continue; // 只沿有剩余容量的边访问新点
                dep[v]=dep[u]+1;
                Q.push(v);
            }
        }
        return dep[t]!=-1; // 汇点可达时继续增广
    }
    int dfs(int u,int t,int f) // 从 u 向汇点最多发送 f 单位流量
    {
        if(u==t)return f;
        int us=0; // 本次实际发送的流量
        for(int &i=cur[u];i<(int)g[u].size();i++)
 // 当前弧优化，记录已搜索到的位置
        {
            int id=g[u][i];
            int v=e[id].v;
            if(e[id].cap==0||dep[u]+1!=dep[v])
continue; // 只沿分层图的下一层前进
            int re=dfs(v,t,min(f-us,e[id].cap));
// 发送量受剩余需求和边容量限制
            if(re==0)continue;
            e[id].cap-=re; // 消耗正向边的剩余容量
            e[id^1].cap+=re; // 增加反向边的剩余容量
            us+=re; // 累计本次发送量
            if(us==f)break; // 已发送完 f，保留当前弧位置供下次使用
        }
        return us;
```

```
    }
    int maxflo(int s,int t) // 求源点 s 到汇点 t 的最大流
    {
        if(s==t)return 0;
        int an=0;
        while(bfs(s,t)) // 每轮重新建立分层图
        {
            fill(cur.begin(),cur.end(),0); // 新的分层图从头搜索邻接表
            while(1)
            {
                int re=dfs(s,t,INF);
                if(re==0)break; // 当前分层图的阻塞流已求完
                an+=re;
            }
        }
        return an;
    }
};
```

连通 - Kosaraju (强连通分量与缩点)

算法说明

- **用途**：求有向图的强连通分量，并缩点得到有向无环图。
- **核心思想**：第一遍在原图上 DFS，记录完成顺序；第二遍按完成顺序的逆序，在反图上 DFS，每次划分一个分量。
- **使用方法**：调用 `add(u,v)` 添加有向边，再调用 `scc(n)`。`id[u]` 为所属分量，`cnt` 为分量总数，`ne` 为缩点图，平行边不去重。
- **时间复杂度**：$O(n+m)$。
- **空间复杂度**：$O(n+m)$。

File: 连通-Kosaraju.cpp

```
#include<bits/stdc++.h>
using namespace std;
#define int long long
#define endl '\n'

const int N=2e5+10;
vector<int>e[N],re[N],ne[N]; // 原有向图、反图、缩点后的有向无环图
int stk[N],top,id[N],cnt; // 按DFS完成顺序存点、记录数量、所属分量编号、分量数量
bool st[N]; // 第一遍DFS中，点是否已经访问

void add(int u,int v) // 添加有向边u->v，同时建立反图
{
    e[u].push_back(v);
    re[v].push_back(u); // 反图将原图的边方向全部反转
}

void dfs1(int u) // 第一遍在原图上搜索，记录DFS完成顺序
{
    st[u]=1;
    for(auto v:e[u])
        if(!st[v])dfs1(v); // 先处理所有尚未访问的后继
    stk[++top]=u; // 所有出边处理完后记录u，得到后序顺序
}

void dfs2(int u) // 第二遍在反图上搜索，划分当前强连通分量
{
    id[u]=cnt; // 标记所属分量，同时表示第二遍已经访问
    for(auto v:re[u])
        if(!id[v])dfs2(v); // 沿反图搜索，未归属的可达点属于当前分量
}

void scc(int n) // 用add提前建图，其余状态初始为零
{
    for(int i=1;i<=n;i++)
        if(!st[i])dfs1(i); // 原图可能不连通，每个未访问的点都要搜索

    for(int i=top;i>=1;i--) // 按第一遍DFS完成顺序的逆序处理
    {
        int u=stk[i];
        if(!id[u]) // 尚未归属任何分量，从u开始划分新分量
        {
            cnt++;
            dfs2(u); // 本次在反图中访问到的点构成一个强连通分量
        }
    }

    for(int u=1;u<=n;u++) // 遍历原图的边，建立缩点后的新图
    {
        for(auto v:e[u])
        {
            int a=id[u],b=id[v]; // 将原点映射为所属分量
            if(a!=b)ne[a].push_back(b); // 只保留分量之间的边，平行边不去重
        }
    }
}
```

图论 - Tarjan 强连通分量 (SCC) 与缩点

何时使用

有向图中寻找**死循环圈（强连通分量）**，或者需要消除环，将图转化为**有向无环图 (DAG)** 以便跑 DP / 最长路 / 拓扑排序时（即缩点）。

核心步骤

1. **tarjan(u)**：打时间戳 dfn 和追溯值 low，点入栈。
2. **遍历出边**：若未访问，深搜并用子节点 low 更新自己；若已在栈中（发现后向边成环），用其 dfn 更新自己 low。
3. **出栈成团**：当 dfn[u] == low[u]，说明 u 是该环的最高点，不断弹栈直到弹出 u，打包成一个新的超级大点。
4. **重建新图**：遍历原图边，若两端点归属不同超级大点，则在新图间连有向边。

File: 连通-tarjan.cpp

```cpp
#include<bits/stdc++.h>
using namespace std;
#define int long long
#define endl '\n'

const int N=2e5+10;
vector<int>e[N],ne[N]; // 原有向图、缩点后的有向无环图
int dfn[N],low[N],tim; // DFS访问顺序、能够追溯到的最早访问顺序、时间戳
int stk[N],top,id[N],cnt; // 尚未归属分量的点栈、栈顶、所属分量编号、分量数量
bool st[N]; // st[u]表示u是否仍在栈中

void tarjan(int u) // 求有向图的强连通分量
{
    dfn[u]=low[u]=++tim; // 首次访问u，初始化访问顺序和追溯值
    stk[++top]=u; // 将u放入栈中，等待确定所属分量
    st[u]=1;
    for(auto v:e[u]) // 遍历u的所有出边
    {
        if(!dfn[v]) // v尚未访问，沿DFS树继续搜索
        {
            tarjan(v);
            low[u]=min(low[u],low[v]); // 用子节点能够追溯到的位置更新u
        }
        else if(st[v]) // v仍在栈中，可以参与当前尚未划分完的分量
            low[u]=min(low[u],dfn[v]); // 已访问的点用dfn更新，已出栈的点不再考虑
    }
    if(dfn[u]==low[u]) // u是一个强连通分量在DFS树中的根
    {
        cnt++;
        int v;
        do
        {
            v=stk[top--]; // 从栈顶弹出当前分量的点
            st[v]=0; // 已经确定所属分量，不再留在栈中
            id[v]=cnt; // 同一次弹出的点属于同一个强连通分量
        }while(v!=u); // 弹到u时，这个强连通分量恰好全部取出
    }
}

void scc(int n) // e[1..n]提前建好，其余状态初始为零
{
    for(int i=1;i<=n;i++)
        if(!dfn[i])tarjan(i); // 图可能不连通，每个未访问的点都要搜索
    for(int u=1;u<=n;u++)
    {
        for(auto v:e[u])
        {
            int a=id[u],b=id[v]; // 将原边的两个端点映射为分量编号
            if(a!=b)ne[a].push_back(b); // 只保留分量之间的边，平行边不去重
        }
    }
}
```

图论 - 割点与割边 (桥) (Articulation Points & Bridges)

何时使用

在**无向图**中寻找“脆弱点/脆弱边”。

- **割点**：如果删除该节点（及其相连的边），图的连通块数量会增加。
- **割边 (桥)**：如果删除该边，图的连通块数量会增加。

核心步骤

1. **建图**：每条无向边调用一次 add(u,v)，两个方向共用一个边编号。DFS 只跳过进入边的编号，保留其他重边；cut[u] 标记割点，bridge[id] 标记第 id 次加入的边是否为桥。
2. **tarjan(u,from)**：记录 dfn（时间戳）和 low（能回溯到的最早时间）。可调用 cut_bridge(n) 统一处理所有连通块。
3. **判定割边**：若在 DFS 树中发现 dfn[u] < low[v]，说明 v 的子树根本无法回到 u 或更早的节点，此时连接它们的边必然是割边。
4. **判定割点**：
   - **非根节点**：只要存在一个子节点满足 dfn[u] <= low[v]，说明子节点离不开 u，断开 u 必定断层。

- **根节点**：特判，如果在 DFS 树中有 ≥ 2 个分支（子节点），那根节点必是割点。

File: 连通-割点桥.cpp

```cpp
#include<bits/stdc++.h>
using namespace std;
#define int long long
#define endl '\n'

const int N=2e5+10,M=2e5+10;
vector<pair<int,int>>e[N]; // 每条邻接记录存{终点,无向边编号}
int dfn[N],low[N],tim,tot; // DFS访问顺序、最早追溯顺序、时间戳、无向边数量
bool cut[N],bridge[M]; // 点是否为割点、无向边是否为桥

void add(int u,int v) // 添加一条无向边，两个方向使用同一个边编号
{
    tot++;
    e[u].push_back({v,tot});
    e[v].push_back({u,tot}); // bridge[tot]对应本次加入的无向边
}

void tarjan(int u,int from) // from为进入u的无向边编号，DFS根节点传0
{
    dfn[u]=low[u]=++tim; // 首次访问u，初始化访问顺序和追溯值
    int cnt=0; // u在DFS树中的子节点数量，用于判断根节点是否为割点
    for(auto [v,id]:e[u])
    {
        if(id==from)continue; // 只跳过进入边，其他通向父节点的重边仍要处理
        if(!dfn[v]) // v尚未访问，这条边成为DFS树边
        {
            cnt++;
            tarjan(v,id);
            low[u]=min(low[u],low[v]); // 将v子树能够追溯到的位置合并到u
            if(low[v]>dfn[u])bridge[id]=1; // v子树无法绕过这条边回到u或u的祖先，说明它是桥
            if(from&&low[v]>=dfn[u])cut[u]=1; // 非根节点：v子树无法绕过u到达u的祖先
        }
        else
            low[u]=min(low[u],dfn[v]); // 非树边用邻点的dfn更新，重边和自环均可处理
    }
    if(!from&&cnt>=2)cut[u]=1; // 根节点至少有两个DFS子树时才是割点
}

void cut_bridge(int n) // 用add提前建图，其余状态初始为零
{
    for(int i=1;i<=n;i++)
        if(!dfn[i])tarjan(i,0); // 分别处理每个连通块，根节点没有进入边
}
```

连通 - 圆方树 (关闭路口后的连通性)

算法说明

- **用途**：无向图中，每次独立关闭一个点，判断另外两个点是否仍然连通。
- **圆方点**：原图点 `1..n` 为圆点，每个点双新建一个方点，连接其中的所有圆点；`ne` 存圆方树，原图不连通时得到森林。
- **核心判断**：圆点 `x` 在树上 `u→v` 的路径中，当且仅当 `dis(u,x)+dis(x,v)==dis(u,v)`。此时 `x` 是必经点，关闭后不连通；关闭端点、原本不连通等情况由 `check` 单独处理。
- **使用方法**：输入 `n,m,q`，接着输入 `m` 条双向道路，再输入 `q` 组 `u,v,x`。`build(n)` 预处理，`check(u,v,x)` 判断关闭 `x` 后是否连通，输出 `YES/NO`；各次关闭互不累积。
- **时间复杂度**：预处理 $O(n + m + n\log n)$，单次查询 $O(\log n)$。
- **空间复杂度**：$O(m + n\log n)$。

File: 连通-圆方树.cpp

```cpp
#include<bits/stdc++.h>
using namespace std;
#define int long long
#define endl '\n'

const int N=2e5+10,L=20;
vector<pair<int,int>>e[N]; // 原无向图，存{终点,边编号}
vector<int>ne[N*2]; // 圆方树，1..n为圆点，n+1..tot为方点
int dfn[N],low[N],tim,stk[N],top,tot,ec; // Tarjan访问顺序、追溯值、时间戳、点栈、栈顶、总点数、边数
int fa[N*2][L],dep[N*2],bel[N*2]; // 倍增祖先、深度、所在圆方树的根编号

void add(int u,int v) // 添加无向边，两个方向共用边编号
{
    ec++;
    e[u].push_back({v,ec});
    e[v].push_back({u,ec});
}

void tarjan(int u,int from) // 划分点双连通分量，同时建立圆方树
{
```

```
    dfn[u]=low[u]=++tim;
    stk[++top]=u;
    for(auto [v,id]:e[u])
    {
        if(id==from)continue; // 只跳过进入边，保留其他重边
        if(!dfn[v])
        {
            tarjan(v,id);
            low[u]=min(low[u],low[v]);
            if(low[v]>=dfn[u]) // 一个点双在u处划分完成
            {
                tot++; // 新建方点，表示这个点双
                int x;
                do
                {
                    x=stk[top--];
                    ne[tot].push_back(x);
                    ne[x].push_back(tot); // 点双内的圆点连接到对应方点
                }while(x!=v);
                ne[tot].push_back(u);
                ne[u].push_back(tot); // u也属于这个点双，但暂时留在栈中
            }
        }
        else low[u]=min(low[u],dfn[v]);
    }
}

void dfs(int u,int from,int root) // 预处理圆方树的深度、祖先和所属连通块
{
    bel[u]=root;
    dep[u]=dep[from]+1;
    fa[u][0]=from;
    for(int j=1;j<L;j++)fa[u][j]=fa[fa[u][j-1]][j-1]; // 计算向上跳2^j步到达的祖先
    for(auto v:ne[u])
        if(v!=from)dfs(v,u,root);
}

void build(int n)
{
    tot=n; // 保留原图点编号，方点从n+1开始编号
    for(int i=1;i<=n;i++)
    {
        if(!dfn[i])
        {
            tarjan(i,0);
            top--; // 当前连通块只剩DFS根节点留在栈中，将它移除
        }
    }
    for(int i=1;i<=n;i++)
        if(!bel[i])dfs(i,0,i); // 原图可能不连通，分别预处理每棵圆方树
}

int lca(int u,int v) // 查询同一棵树中两个点的最近公共祖先
{
    if(dep[u]<dep[v])swap(u,v);
    int d=dep[u]-dep[v];
    for(int j=L-1;j>=0;j--)
        if(d&(1LL<<j))u=fa[u][j]; // 先将较深的点提升到同一深度
    if(u==v)return u;
    for(int j=L-1;j>=0;j--)
    {
        if(fa[u][j]!=fa[v][j]) // 两点同时向上跳，保持在公共祖先的下方
        {
            u=fa[u][j];
            v=fa[v][j];
        }
    }
    return fa[u][0];
}

int dis(int u,int v) // 圆方树上两点之间的边数
{
    return dep[u]+dep[v]-2*dep[lca(u,v)];
}

bool check(int u,int v,int x) // 关闭原图路口x后，u和v是否仍能互相到达
{
    if(u==x||v==x)return 0; // 起点或终点已经关闭
    if(bel[u]!=bel[v])return 0; // 两个地点原本就不连通
    if(bel[x]!=bel[u])return 1; // 关闭其他连通块的路口，不影响这两个地点
    return dis(u,x)+dis(x,v)!=dis(u,v); // x在树上路径中才是必经点，关闭后才会断开
}

void solve()
{
    int n,m,q;
    cin>>n>>m>>q; // 路口数、双向道路数、询问数
    for(int i=1;i<=m;i++)
    {
        int u,v;
        cin>>u>>v;
        add(u,v);
    }
    build(n); // 一次建树，之后每个询问互相独立
    while(q--)
    {
```

```
        int u,v,x;
        cin>>u>>v>>x; // 判断关闭路口x后，地点u和v是否仍然连通
        cout<<(check(u,v,x)?"YES":"NO")<<endl;
    }
}

signed main()
{
    ios::sync_with_stdio(0);
    cin.tie(0);
    solve();
    return 0;
}
```

字符串

字符串 - AC 自动机 (Aho-Corasick Automaton)

算法说明

- **用途**：多模式串匹配。用于在长文本中高效查找、统计多个短关键词的存在情况。
- **时间复杂度**：构建 $O(\sum |P| + |\Sigma|)$，查询 $O(|T|)$。
- **核心思想：Trie 树 + KMP (fail 指针) + Trie 图优化**。失配时跳 `fail` 指针找最长公共前后缀；利用 `tr[u][i] = tr[fail[u]][i]` 路径压缩，实现 $O(1)$ 状态转移。
- **多组数据避坑：严禁 memset 清空全局数组**，必超时！需精确遍历 `0` 到 `idx` 进行清空。

File: ac 自动机.cpp

```
#include<bits/stdc++.h>
using namespace std;
#define int long long
#define endl '\n'
using PII = pair<int, int>;

const int N = 1e6 + 10;
int tr[N][26] = { 0 }, idx = 0;
int fail[N] = { 0 };
int cnt[N] = { 0 };

// 1. 建 Trie 树
void insert(string s)
{
    int he = 0;
    for(auto ch : s)
    {
        int dd = ch - 'a';
        if(tr[he][dd] == 0) tr[he][dd] = ++idx;
        he = tr[he][dd];
    }
    cnt[he]++; // 标记结尾
}

// 2. BFS 构建 fail 指针与 Trie 图
void build()
{
    queue<int> q;
    for(int i = 0; i < 26; i++)
    {
        if(tr[0][i]) q.push(tr[0][i]);
    }

    while(!q.empty())
    {
        int u = q.front();
        q.pop();
        for(int i = 0; i < 26; i++)
        {
            if(tr[u][i])
            {
                fail[tr[u][i]] = tr[fail[u]][i];
                q.push(tr[u][i]);
            }
            else
            {
                tr[u][i] = tr[fail[u]][i]; // Trie 图路径压缩优化
            }
        }
    }
}

// 3. 匹配文本串 (统计包含多少种模式串)
int query(string s)
{
    int u = 0, res = 0;
    for(auto ch : s)
    {
        int dd = ch - 'a';
        u = tr[u][dd]; // O(1) 状态转移

        // 跳 fail 指针收集所有匹配的模式串
        for(int j = u; j > 0 && cnt[j] != -1; j = fail[j])
        {
            res += cnt[j];
            cnt[j] = -1; // 打标记防重复统计
        }
    }
    return res;
}

/* 使用说明与多组数据排雷技巧：
 * 1. 核心调用流：
 * - for(模式串) insert(s);
 * - build();
```

```
 * - query(主串);
 * 2. 初始化防 TLE (极其关键):
 * 在多组测试数据中，切忌使用 memset 清空整个 tr 数
   组，必会导致超时!
 * 应严格基于使用过的节点数量 idx 进行精准清空:
 * void init() {
 * for(int i = 0; i <= idx; i++) {
 * for(int j = 0; j < 26; j++) tr[i][j] = 0;
 * fail[i] = cnt[i] = 0;
 * }
 * idx = 0;
 * }
 */
```

字符串-KMP

算法说明

- **用途**：在 $O(N+M)$ 的时间复杂度内，高效统计模式串 P 在主串 S 中出现的次数（或查找所有出现的位置）。
- **时间复杂度**：
  - 预处理 ne 数组：$O(M)$，其中 M 为模式串长度。
  - 匹配过程：$O(N)$，其中 N 为主串长度。
- **适用场景**：单模式串匹配问题、统计字符串出现频次、求字符串的最小循环节与公共前后缀。

File: kmp.cpp

```cpp
#include<bits/stdc++.h>
using namespace std;

void solve()
{
    string s,p;
    cin>>s>>p;
    int n=s.size(),m=p.size();
    vector<int>ne(m+1);
    s=" "+s;
    p=" "+p;
    for(int i=2,j=0;i<=m;i++)
    {
        while(j&&p[i]!=p[j+1])j=ne[j];
        if(p[i]==p[j+1])j++;
        ne[i]=j;
    }
    int cn=0;
    for(int i=1,j=0;i<=n;i++)
    {
        while(j&&s[i]!=p[j+1])j=ne[j];
        if(s[i]==p[j+1])j++;
        if(j==m)
        {
            cn++;
            j=ne[j];
        }
    }
}
```

字符串 - Z 函数 (扩展 KMP)

算法说明

- **用途**：在 $O(N)$ 时间内求出一个字符串所有后缀与前缀的最长公共前缀长度。
- **时间复杂度**：$O(N)$。
- **核心思想**：维护一个最靠右的匹配段 (Z-box) $[l,r]$，在该区间内的计算直接“白嫖”前缀 $z[i-l]$ 的值，并卡住边界，超出边界的部分再暴力扩展。

File: kmpz 函数.cpp

```cpp
#include<bits/stdc++.h>
using namespace std;
void solve()
{
    string s;
    cin>>s;
    int n=s.size();
    vector<int>z(n);
    int l=0,r=0;
    for(int i=1;i<n;i++)
    {
        if(i<=r)z[i]=min(z[i-l],r-i+1);
        while(i+z[i]<n&&s[z[i]]==s[i+z[i]])z[i
    ]++;
        if(i+z[i]-1>r)
        {
            l=i;
            r=i+z[i]-1;
        }
    }

}
```

字符串 - Manacher 算法 (马拉车)

算法说明

- **用途**：在 $O(N)$ 时间复杂度内，求出字符串中以每个位置为中心的最长回文子串的半径（从而找到全局最长回文子串）。
- **时间复杂度**：$O(N)$。
- **核心思想**：利用已经求出的大回文串的对称性，直接将左半边的答案镜像拷贝到右半边，避免重复扩展。

File: Manachar.cpp

```cpp
#include<bits/stdc++.h>
using namespace std;
using PII = pair<int,int>;
#define int long long
#define endl '\n'
void solve()
{
```

```
    string s;
    cin>>s;
    string t="$#";
    for(auto c:s)
    {
        t+=c;
        t+="#";
    }
    t+='^';
    int n=t.size();
    vector<int>p(n+1);
    int C=0,R=0;//中心，最远R
    int mxle=0;
    for(int i=1;i<n-1;i++)
    {
        if(i<=R)p[i]=min(p[2*C-i],R-i+1);
        else p[i]=1;
        while(t[i-p[i]]==t[i+p[i]])p[i]++;
        if(i+p[i]-1>R)
        {
            C=i;
            R=i+p[i]-1;
        }
        if(p[i]-1>mxle)mxle=p[i]-1;
    }
    cout<<mxle<<endl;

}
```

字符串 - 后缀数组 (倍增与计数排序)

算法说明

- **用途**：将所有后缀按字典序排序。例如 banana 排序后为 a、ana、anana、banana、na、nana。
- **sa[i]**：第 i 名后缀的起点，如 sa[2]=4，对应 ana。
- **rk[i]**：从位置 i 开始的后缀排名，满足 rk[sa[i]]=i。
- **height[i]**：第 i 名与第 i-1 名后缀的最长公共前缀长度，规定 height[1]=0。它按排名索引，不按原串位置索引。
- **复杂度**：构建 $O(n \log n)$，计算 height 为 $O(n)$，工作空间 $O(n)$。

核心过程

每轮把比较长度从 k 扩展到 2k，用 (rk[i],rk[i+k]) 作为两个关键字，越界排名为 0。y 先排好第二关键字，再用 cnt 稳定排序第一关键字，最后用 tmp 重新编号。

计算 height 时，按起点顺序比较当前后缀与其字典序前驱；起点右移一位后，复用上次公共前缀长度减 1，再继续向后比较。

使用与例子

调用 suffix_array(s)，使用 sa[1..n]、rk[1..n]、height[1..n]。模板支持长度不超过 20 万的非空串，按字节处理字符。

CPP 输入 banana，输出三行，依次为 sa、rk、height：

```
6 4 2 1 5 3
4 3 6 2 5 1
0 1 3 0 0 2
```

常用结论

- **最长重复子串长度**：max(height)，上例为 3，对应 ana。
- **不同子串数量**：$n(n+1)/2 - \sum height[i]$，上例为 15。
- **两个不同后缀的 LCP**：若排名 a<b，答案为 height[a+1..b] 的最小值，可配合 ST 表。

File: 后缀数组.cpp

```
#include<bits/stdc++.h>
using namespace std;
#define int long long
#define endl '\n'

const int N=2e5+10;
int sa[N],rk[N],height[N]; // 排名第i的后缀起点、起点为i的后缀排名、相邻排名后缀的最长公共前缀长度
int y[N],tmp[N],cnt[N]; // 按第二关键字排好的起点、新排名、计数排序的桶

void suffix_array(const string &s) // 构建非空字符串的后缀数组，结果下标从1开始
{
    int n=s.size(),m=256; // 字符串长度、当前排名的最大值
    fill(cnt+1,cnt+m+1,0);
    for(int i=1;i<=n;i++)cnt[rk[i]=(unsigned char)s[i-1]+1]++; // 初始按单个字符排名，0留给越界部分
    for(int i=1;i<=m;i++)cnt[i]+=cnt[i-1]; // 桶的前缀和，确定每种排名的结束位置
    for(int i=n;i>=1;i--)sa[cnt[rk[i]]--]=i; // 按第一字符进行计数排序

    for(int k=1;k<n;k<<=1) // 已按前k个字符排序，本轮扩展到前2k个字符
    {
        int p=0;
        for(int i=n-k+1;i<=n;i++)y[++p]=i; // 第二段越界，第二关键字为0，排在最前面
        for(int i=1;i<=n;i++)
            if(sa[i]>k)y[++p]=sa[i]-k; // 利用原sa顺序，得到按第二关键字排好的起点

        fill(cnt+1,cnt+m+1,0);
        for(int i=1;i<=n;i++)cnt[rk[y[i]]]++;
        for(int i=1;i<=m;i++)cnt[i]+=cnt[i-1];
        for(int i=n;i>=1;i--)sa[cnt[rk[y[i]]]--]=y[i]; // 稳定排序第一关键字，得到两关键字的顺序

        p=1;
        tmp[sa[1]]=1;
```

```
    for(int i=2;i<=n;i++)
    {
        int a=sa[i-1],b=sa[i]; // 比较排序后相邻的两个起点
        int u=a+k<=n?rk[a+k]:0,v=b+k<=n?rk[b+k]:0; // 两个起点的第二关键字
        if(rk[a]!=rk[b]||u!=v)p++; // 任一关键字不同，就划分一个新排名
        tmp[b]=p;
    }
    for(int i=1;i<=n;i++)rk[i]=tmp[i]; // 用新排名替换旧排名
    if(p==n)break; // 所有后缀已经区分，排序完成
    m=p; // 下一轮计数排序只需处理当前排名范围
}

for(int i=1;i<=n;i++)rk[sa[i]]=i; // 最终排名为1..n，兼容只有一个字符的情况
int h=0;
height[1]=0; // 排名第一的后缀没有前驱
for(int i=1;i<=n;i++) // 按起点顺序，线性计算height
{
    if(rk[i]==1)
    {
        h=0;
        continue;
    }
    if(h)h--; // 起点右移一位，可保留至少h-1个相同字符
    int j=sa[rk[i]-1]; // 当前后缀在字典序中的前一个后缀
    while(i+h<=n&&j+h<=n&&s[i+h-1]==s[j+h-1])h++; // 从已知公共前缀后继续比较
    height[rk[i]]=h;
}
}

void solve()
{
    string s;
    cin>>s; // 输入一个非空字符串，示例为banana
    suffix_array(s);
    int n=s.size();
    for(int i=1;i<=n;i++)cout<<sa[i]<<" "; // 第一行：按字典序排列后的后缀起点
    cout<<endl;
    for(int i=1;i<=n;i++)cout<<rk[i]<<" "; // 第二行：每个起点对应的后缀排名
    cout<<endl;
    for(int i=1;i<=n;i++)cout<<height[i]<<" "; // 第三行：相邻排名后缀的最长公共前缀长度
    cout<<endl;
}

signed main()
{
    ios::sync_with_stdio(0);
    cin.tie(0);
    solve();
    return 0;
}
```

字符串 - 后缀自动机 (SAM)

算法说明

- **用途**：维护一个串的所有子串，支持统计不同子串数量、查询子串出现次数。
- **状态含义**：结束位置集合相同的子串合并为一个状态。例如 ababa 中，b 和 ab 都在位置 2、4 结束，所以共用状态。
- **复杂度**：固定 26 个小写字母时，构建 $O(n)$，查询长度为 m 的串为 $O(m)$，转移表空间 $O(26n)$。

关键字段

- **ch[c]**：追加字符 c 后到达的状态，0 表示不存在。
- **len**：当前状态代表的最长子串长度。
- **link**：指向最长的、属于另一个状态的真后缀所在状态。
- **occ**：完成次数传播后，当前状态中每个子串的出现次数。

1 为根，last 为整个当前字符串对应的状态。非根状态 p 代表长度在 len[link[p]]+1..len[p] 之间的子串，每个长度恰好一个。

核心过程

1. 添加字符时建立新状态 cur，令 len 加 1、occ=1，沿后缀链接补齐缺失转移。
2. 遇到已有转移 p→q：若 len[p]+1==len[q]，直接连接；否则克隆 q，缩短克隆的 len，调整转移和后缀链接。
3. 克隆的 occ=0，因为它只拆分状态，没有新增结束位置。构建后按 len 从长到短执行 occ[link[p]]+=occ[p]，统计真实出现次数。

克隆例子：ab 中的 b、ab 都只在位置 2 结束；追加 b 后，b 在位置 2、3 结束，ab 仍只在位置 2 结束，因此要拆成不同状态。

使用与例子

模板处理长度不超过 20 万的非空小写字母串，查询串也须非空且为小写字母。

```
build("ababa"); // 构建并完成次数统计
cout<<distinct()<<endl; // 9种不同的非空子串
cout<<query("aba")<<endl; // 出现2次，重叠也计算
cout<<query("c")<<endl; // 没有出现，输出0
```

CPP 输入原串、询问数和各查询串；先输出不同子串数量，再逐行输出各串的出现次数。

常用结论

- **不同子串数量**：对非根状态累加 len[p]-len[link[p]]。

- **最长重复子串长度**：在 occ>=2 的状态中取最大的 len。

File: 后缀自动机.cpp

```cpp
#include<bits/stdc++.h>
using namespace std;
#define int long long
#define endl '\n'

const int N=2e5+10;
struct T
{
    int ch[26],len,link,occ; // 字符转移、当前状态的最长长度、后缀链接、出现次数
}tr[N*2]; // 一个状态可能代表多个子串，最多预留约2n个状态
int tot=1,last=1; // 状态总数、整个当前字符串对应的状态，1为根，0表示不存在
int cnt[N],ord[N*2]; // 按len计数的桶、按len从小到大排列的状态编号

void extend(int c) // 在当前字符串末尾添加一个字符，c为0..25
{
    int cur=++tot,p=last; // cur代表新前缀的结束位置，p沿后缀链接向前寻找
    tr[cur].len=tr[last].len+1;
    tr[cur].occ=1; // 新增一个实际结束位置，先在新状态中记录一次
    while(p&&!tr[p].ch[c]) // 当前后缀还不能接字符c，补上通向cur的转移
    {
        tr[p].ch[c]=cur;
        p=tr[p].link; // 继续处理更短的后缀
    }

    if(!p)tr[cur].link=1; // 所有后缀都没有原有转移，后缀链接指向根
    else
    {
        int q=tr[p].ch[c]; // 找到原来已经存在的字符c转移
        if(tr[p].len+1==tr[q].len)
            tr[cur].link=q; // 长度刚好衔接，q就是新状态的后缀链接目标
        else
        {
            int clone=++tot; // 原状态需要拆分，新建克隆状态承接较短的一组子串
            tr[clone]=tr[q]; // 保留原状态的字符转移和后缀链接
            tr[clone].len=tr[p].len+1; // 克隆状态的最长长度改为当前需要的长度
            tr[clone].occ=0; // 克隆只拆分状态，没有新增实际结束位置
            while(p&&tr[p].ch[c]==q)
            {
                tr[p].ch[c]=clone; // 将较短后缀原本通向q的转移改为通向clone
                p=tr[p].link;
            }
            tr[q].link=tr[cur].link=clone; // 原状态和新状态都通过clone连接更短的后缀
        }
    }
    last=cur; // 整个当前字符串对应的新状态
}

void build(const string &s) // 输入非空小写字母串，构建并统计出现次数，支持重新构建
{
    for(int i=1;i<=tot;i++)tr[i]={}; // 清空上次实际用到的状态
    tot=last=1;
    for(char c:s)extend(c-'a'); // 从左到右逐个加入字符

    int n=s.size();
    fill(cnt,cnt+n+1,0);
    for(int i=1;i<=tot;i++)cnt[tr[i].len]++; // 按状态最长长度进行计数排序
    for(int i=1;i<=n;i++)cnt[i]+=cnt[i-1];
    for(int i=tot;i>=1;i--)ord[cnt[tr[i].len]--]=i;
    for(int i=tot;i>=2;i--) // 从长状态向短状态传播，根位于ord[1]，不再向外传播
    {
        int p=ord[i];
        tr[tr[p].link].occ+=tr[p].occ; // 当前串每次出现，它的后缀也会在同一位置结束
    }
}

int query(const string &s) // 查询非空小写字母串的出现次数，允许重叠
{
    int p=1; // 从根开始，依次沿字符转移
    for(char c:s)
    {
        int x=c-'a';
        if(!tr[p].ch[x])return 0; // 没有对应转移，说明这个串不是原串的子串
        p=tr[p].ch[x];
    }
    return tr[p].occ; // 到达的状态中，所有子串具有相同的出现次数
}

int distinct() // 统计不同的非空子串数量
{
    int an=0;
    for(int p=2;p<=tot;p++) // 根代表空串，不参与统计
```

```
        an+=tr[p].len-tr[tr[p].link].len; // 当前状态每种合法长度恰好代表一个子串
    return an;
}

void solve()
{
    string s;
    cin>>s; // 输入非空小写字母串，示例为ababa
    build(s);
    cout<<distinct()<<endl; // 第一行输出不同的非空子串数量
    int q;
    cin>>q; // 接下来输入q个要查询的非空小写字母串
    while(q--)
    {
        string t;
        cin>>t;
        cout<<query(t)<<endl; // 输出t在原串中的出现次数，重叠出现也计算
    }
}

signed main()
{
    ios::sync_with_stdio(0);
    cin.tie(0);
    solve();
    return 0;
}
```

字符串 - 字典树 (Trie)

算法说明

- **用途**：高效存储和查询字符串集合（前缀树）。将字符串的公共前缀合并，最大限度减少无谓的字符串比较。
- **时间复杂度**：插入与查询均为 $O(|S|)$，其中 $|S|$ 为单条字符串的长度，与字典中字符串的数量无关。
- **适用场景**：词频统计、前缀匹配查询、字符串去重/判重、自动机预处理。
- **核心思想**：将字符隐式地映射为树上的边。每个节点代表一个前缀状态，`nex[p][c]` 相当于在节点 `p` 后追加字符 `c` 转移到的下一个状态节点。
- **常见模型**：
  1. **点名/签到系统**：记录字符串是否合法及是否被重复访问（如本模板的 `OK/REPEAT/WRONG`）。
  2. **前缀统计**：给经过的每个节点打上 `cnt++` 标记，用于查询“以某个字符串为前缀的单词数量”。
  3. **01 Trie**：将数字转为二进制位插入，用于解决最大异或和等极值问题

File: 字典树.cpp

```
#include<bits/stdc++.h>
using namespace std;
#define int long long
using PII = pair<int,int>;

const int MAXN = 5e5 + 5; // 节点总数极限，通常为：字符串个数 * 字符串最大长度

struct Trie {
    int nex[MAXN][26]; // nex[p][c]：节点 p 的字符 c 连向的子节点编号
    int st[MAXN];      // 状态标记：0=非结尾/不存在，1=存在且未访问，>1=已访问
    int id = 1;        // 节点分配器，0为根节点，从1开始分配新节点

    // 插入字符串
    void insert(const string& s)
    {
        int p = 0;
        for(char ch : s)
        {
            int c = ch - 'a';
            if(!nex[p][c]) nex[p][c] = id++;
            p = nex[p][c];
        }
        st[p] = 1; // 标记该节点为某个字符串的结尾
    }

    // 查询字符串状态
    string find(const string& s)
    {
        int p = 0;
        for(char ch : s)
        {
            int c = ch - 'a';
            if(!nex[p][c]) return "WRONG"; // 连前缀都匹配不上
            p = nex[p][c];
        }

        // 匹配到了目标节点，根据状态标记判断
        if(st[p] == 0) return "WRONG";      // 是别人的前缀，但自身不是完整单词
        else if(st[p] == 1)
        {
            st[p]++;
            return "OK";                    // 首次合法访问
        }
        else return "REPEAT";               // 合法但重复访问
    }
};

/* 核心调用与拓展逻辑：
 * 1. 实例化与使用：
```

```
 * Trie tr;
 * tr.insert(s);
 * cout << tr.find(s) << '\n';
 * * 2. 多组数据清空防 TLE 优化：
 * 绝不能 memset 整个 nex 数组（极大可能会超时）。
 * 只需按分配出的 id 数量进行精准清空：
 * void clear() {
 * for(int i = 0; i < id; i++) {
 * for(int c = 0; c < 26; c++) nex[i][c] = 0;
 * st[i] = 0;
 * }
 * id = 1;
 * }
 */
```

数学

博弈论 - SG 函数与 Sprague-Grundy 定理

解决什么实际问题 (实战映射)

在算竞的博弈论（尤其是公平组合游戏 Impartial Game）中，当你遇到以下特征的题目时，直接掏出 SG 定理：

1. **多线并行的独立游戏 (复合博弈)**：桌子上有 N 堆石子，或者有 N 张图，每次只能选择**其中一堆/一张图**进行操作。**【核心痛点】**：多线操作的分支极度爆炸，无法直接算。但 SG 定理能将它们“降维打击”，各自算各自的，最后异或起来就能得到全局胜负。
2. **规则古怪的取石子/移动棋子**：比如“每次只能取斐波那契数个石子”、“每次可以把一堆石子分裂成不相等的两堆”。这类问题无法用传统 Nim 游戏（直接异或石子数）的结论秒杀，必须手推它们的状态转移图，求出每个状态的 SG 值。
3. **“无法操作者判负”(正常规则)**：这是使用 SG 函数的绝对大前提。

核心结论与数学性质 (实战必背)

所有的公平组合博弈，最终都可以等效映射为经典的 Nim 游戏。

1. mex **运算 (Minimum Excluded Value)**：
   - 定义：施加于一个集合，求出**不属于该集合的最小非负整数**。
   - 举例：$mex(\{0,1,3\}) = 2$；$mex(\{1,2\}) = 0$；$mex(\emptyset) = 0$。
2. **状态的 SG 值定义**：
   - 一个状态 x 的 SG 值，等于它**所有后继状态**的 SG 值构成的集合的 mex。
   - 公式：$SG(x) = mex(\{SG(y) \mid x \to y\})$。
   - **必败态结论**：所有无法继续操作的终点状态，其后继为空集，所以 $SG(\text{终点}) = mex(\emptyset) = 0$。反之，如果 $SG(x) = 0$，当前玩家面对 x 必败；$SG(x) > 0$，必胜。
3. **Sprague-Grundy 定理 (大一统缝合怪)**：
   - 如果一个大游戏由 N 个互相独立的小游戏组成（每次只能动其中一个），那么这个大游戏的全局状态的 SG 值，等于所有小游戏当前状态的 SG 值的**异或和 (XOR)**。
   - $SG_{total} = SG(x_1) \oplus SG(x_2) \oplus \cdots \oplus SG(x_n)$
   - **终极判决**：若 $SG_{total} > 0$，先手必胜；若 $SG_{total} = 0$，先手必败。

File: 博弈论-sg 函数.cpp

```cpp
#include<bits/stdc++.h>
using namespace std;

#define int long long
#define endl '\n'

const int MAXN=10010; // 状态的最大规模 (例如单堆石子的最大数量)

int k;                // 允许的操作种类数
int f[110];           // f[i]: 每次允许取走的石子数量集合
int sg[MAXN];         // sg[x]: 状态 x 的 SG 值 (记忆化数组)

// --- 核心：记忆化搜索求 SG 值 ---
int get_sg(int x)
{
    // 如果该状态已经算过，直接查表返回 O(1)
    if(sg[x]!=-1) return sg[x];

    // S 集合用来记录当前状态 x 所有合法后继状态的 SG 值
    unordered_set<int> S;

    // 遍历所有合法的下一步操作
    for(int i=1;i<=k;i++)
    {
        int next_state = x - f[i];
        // 如果这个操作合法 (剩下的石子数不能为负)
        if(next_state >= 0)
        {
            S.insert(get_sg(next_state)); // 递归求子状态的 SG 值并扔进集合
        }
    }

    // --- 模拟 mex 运算 ---
    // 从 0 开始往上枚举，找到第一个不在集合 S 中的数
    for(int i=0;;i++)
    {
        if(!S.count(i))
        {
            return sg[x]=i; // 记忆化并返回
        }
```

```
    }
}

void solve()
{
    /* 实战调用示例
    // 题目背景：有 n 堆石子，每次从一堆中取石子，取的数量必须在集合 {f[1], f[2] ... f[k]} 中。
    // 问先手是否必胜？

    cin>>k;
    for(int i=1;i<=k;i++) cin>>f[i]; // 读入允许取走的石子数集合

    // 1. 初始化记忆化数组为 -1 (代表还没算过)
    memset(sg, -1, sizeof sg);

    int n;
    cin>>n;

    int res = 0; // 记录异或和 SG_total
    for(int i=1;i<=n;i++)
    {
        int x;
        cin>>x; // 读入每一堆初始的石子数

        // 2. 利用 SG 定理，把所有独立游戏的 SG 值异或起来
        res ^= get_sg(x);
    }

    // 3. 终极判决
    if(res > 0) cout<<"Yes"<<endl; // 先手必胜
    else cout<<"No"<<endl;         // 先手必败
    */
}

signed main()
{
    cin.tie(0);
    ios::sync_with_stdio(0);
    int T=1;
    // cin>>T;
    while(T--) solve();
    return 0;
}
```

多项式 - FFT (快速傅里叶变换)

算法说明

- **用途**：快速计算多项式乘积，本模板返回整数系数卷积。
- **核心思想**：系数转为点值，对应相乘，再逆变换恢复系数。
- **复杂度**：时间 $O(n \log n)$，空间 $O(n)$，n 为补齐后的变换长度。

使用方法

- 系数从常数项到最高次项存入非空数组，调用 `mul(a,b)`，返回 `a.size()+b.size()-1` 个系数。
- 函数自动补 0 到 2 的幂，使用 `double` 计算，最后四舍五入为整数；支持负系数。

```
vector<int>a={1,2},b={3,4}; // 1+2x与3+4x相乘
vector<int>ans=mul(a,b); // {3,10,8}，即3+10x+8x²
```

与 NTT 的区别

- **运算方式**：FFT 使用复数和浮点运算，NTT 使用模运算，模意义下没有浮点误差。
- **结果含义**：本模板返回普通整数系数，NTT 返回系数对 998244353 取模的结果。
- **如何选择**：普通整数卷积且精度足够时用 FFT；题目要求模 998244353 时用 NTT。FFT 的系数或结果较大时可能舍入失准，不能保证任意 `long long` 系数都精确。

File: 多项式-fft.cpp

```
#include<bits/stdc++.h>
using namespace std;
#define int long long
#define endl '\n'

using cp=complex<double>;
const double pi=acos(-1);

void fft(vector<cp>&a,int op) // op=1为正变换，op=-1为逆变换，数组长度必须是2的幂
{
    int n=a.size();
    for(int i=1,j=0;i<n;i++) // 将下标调整为二进制位翻转后的顺序
    {
        int k=n>>1;
        for(;j&k;k>>=1)j^=k; // 从高位开始处理进位
        j^=k;
        if(i<j)swap(a[i],a[j]); // 每对位置只交换一次
    }

    for(int len=2;len<=n;len<<=1) // 从长度2开始，逐层合并两个较小的变换
    {
        double ang=2*pi/len*op;
        cp wn(cos(ang),sin(ang)); // 当前层的单位根
        for(int i=0;i<n;i+=len) // 每次处理一个长度为len的区间
        {
            cp w(1,0); // 单位根从0次幂开始
            for(int j=0;j<len/2;j++)
            {
                cp u=a[i+j],v=w*a[i+j+len/2]; // 左半部分、乘上单位根后的右半部分
```

```
                a[i+j]=u+v; // 蝶形合并的前半部分
                a[i+j+len/2]=u-v; // 蝶形合并的后半部分
                w*=wn; // 转到单位根的下一次幂
            }
        }
    }

    if(op==-1)
        for(auto &x:a)x/=n; // 逆变换最后除以长度，恢复原系数
}

vector<int>mul(const vector<int>&a,const vector<int>&b) // 两个非空整数系数多项式相乘
{
    int m=a.size()+b.size()-1,n=1; // 乘积的系数数量、FFT使用的长度
    while(n<m)n<<=1; // 补到不小于m的最小2次幂，避免循环卷积混叠

    vector<cp>x(n),y(n); // 其余位置自动补0
    for(int i=0;i<(int)a.size();i++)x[i]=a[i];
    for(int i=0;i<(int)b.size();i++)y[i]=b[i];

    fft(x,1);
    fft(y,1); // 将两个多项式从系数表示转换为点值表示
    for(int i=0;i<n;i++)x[i]*=y[i]; // 点值对应相乘，得到乘积的点值表示
    fft(x,-1); // 逆变换得到乘积的系数

    vector<int>ans(m);
    for(int i=0;i<m;i++)ans[i]=llround(x[i].real()); // 四舍五入为整数，支持负系数
    return ans;
}
```

多项式 - NTT (快速数论变换)

算法说明

- **用途**：快速计算多项式乘积，结果对 998244353 取模。
- **核心思想**：用模意义下的单位根完成正逆变换，点值相乘后恢复系数。
- **复杂度**：时间 $O(n \log n)$，空间 $O(n)$，n 为补齐后的变换长度。

使用方法

- 系数从常数项到最高次项存入非空数组，调用 `mul(a,b)`，返回 `a.size()+b.size()-1` 个系数，负系数自动转为合法余数。
- 固定 `mod=998244353`、原根 `g=3`，补齐后的变换长度必须不超过 2^{23}。

```
vector<int>a={1,2},b={3,4}; // 1+2x与3+4x相乘
vector<int>ans=mul(a,b); // {3,10,8}，所有系数对mod取模
```

与 FFT 的区别

- **精度与结果**：NTT 做整数模运算，模意义下结果精确；FFT 做浮点运算，本仓库 FFT 模板将结果四舍五入为普通整数。
- **模数限制**：变换长度必须整除 `mod-1`。当前模数满足 $998244353 - 1 = 119 \times 2^{23}$，不能直接改成 `1e9+7` 沿用本模板。
- **如何选择**：要求模 998244353 时用 NTT；需要普通整数卷积且精度足够时用 FFT。单模 NTT 不能直接恢复超过模数范围的普通整数系数。

File: 多项式-ntt.cpp

```
#include<bits/stdc++.h>
using namespace std;
#define int long long
#define endl '\n'

const int mod=998244353,g=3;

int qpow(int a,int b) // 快速计算a^b mod mod
{
    int an=1;
    while(b)
    {
        if(b&1)an=an*a%mod;
        a=a*a%mod;
        b>>=1;
    }
    return an;
}

void ntt(vector<int>&a,int op) // op=1为正变换，op=-1为逆变换，长度为2的幂且不超过2^23
{
    int n=a.size();
    for(int i=1,j=0;i<n;i++) // 将下标调整为二进制位翻转后的顺序
    {
        int k=n>>1;
        for(;j&k;k>>=1)j^=k; // 从高位开始处理进位
        j^=k;
        if(i<j)swap(a[i],a[j]); // 每对位置只交换一次
    }

    for(int len=2;len<=n;len<<=1) // 逐层合并两个较小的变换
    {
        int wn=qpow(g,(mod-1)/len); // 当前层的单位根，满足wn^len=1
        if(op==-1)wn=qpow(wn,mod-2); // 逆变换使用单位根的逆元
        for(int i=0;i<n;i+=len) // 每次处理一个长度为len的区间
        {
```

```
            int w=1; // 单位根从0次幂开始
            for(int j=0;j<len/2;j++)
            {
                int u=a[i+j],v=w*a[i+j+len/2]%mod;
                a[i+j]=(u+v)%mod; // 蝶形合并的前半部分
                a[i+j+len/2]=(u-v+mod)%mod; // 蝶形合并的后半部分
                w=w*wn%mod; // 转到单位根的下一次幂
            }
        }
    }

    if(op==-1)
    {
        int inv=qpow(n,mod-2); // 长度n在模意义下的逆元
        for(auto &x:a)x=x*inv%mod; // 逆变换最后除以长度，恢复原系数
    }
}

vector<int>mul(const vector<int>&a,const vector<int>&b) // 两个非空多项式相乘，返回系数模mod的结果
{
    int m=a.size()+b.size()-1,n=1; // 乘积的系数数量、NTT使用的长度
    while(n<m)n<<=1; // 补到不小于m的最小2次幂

    vector<int>x(n),y(n); // 其余位置自动补0
    for(int i=0;i<(int)a.size();i++)x[i]=(a[i]%mod+mod)%mod;
    for(int i=0;i<(int)b.size();i++)y[i]=(b[i]%mod+mod)%mod; // 将负系数也转为合法余数

    ntt(x,1);
    ntt(y,1); // 从系数表示转换为点值表示
    for(int i=0;i<n;i++)x[i]=x[i]*y[i]%mod; // 对应点值相乘
    ntt(x,-1); // 逆变换得到乘积的系数

    x.resize(m); // 去掉补0产生的多余位置
    return x;
}
```

数学 - 卢卡斯定理与扩展卢卡斯定理 (Lucas & exLucas)

解决什么实际问题 (实战映射)

在算竞中，看到以下几种经典题面，直接套用卢卡斯全家桶：

1. **天文数字的组合数（选数/网格图路径）：**题面要求从极大的集合中选数，或者在一个长宽达到 10^{18} 的网格图中求路径数（本质也是组合数）。**特征：**N, M 极大（远超 10^7，无法预处理阶乘），但模数 P 相对较小（通常 $\leq 10^5$）。P 为质数直接用 Lucas，P 为合数用 exLucas。
2. **阶乘末尾的零 / 整除质数的最高次幂：**题目问“$N!$ 在 P 进制下末尾有多少个 0”或者“$N!$ 能被 p^k 整除的最大 k 是多少”。**破解：**直接抠出板子里的**勒让德定理**（即那个 `while(n) cnt += n / p, n /= p;` 循环），几行代码瞬间秒杀。
3. **组合数的奇偶性判断 / 规律分形：**题目让你判断 $\binom{N}{M}$ 是奇数还是偶数，或者多次询问要求 $O(1)$ 回答。**破解：**千万不要算！直接利用推论判断 `(N & M) == M`，如果是 `true` 结果就是奇数，`false` 就是偶数。这在 CF 的思维题和打表找规律题中是绝杀技巧。

核心结论与数学性质 (实战必背)

1. **卢卡斯定理 (Lucas Theorem)：**
   - **前提条件：**模数 P **必须是质数。**
   - **核心公式：**$\binom{n}{m} \equiv \binom{n \bmod p}{m \bmod p} \times \binom{n/p}{m/p} \pmod p$
   - **进制推论 (极为有用)：**如果把 n 和 m 都转化为 p 进制数，那么 $\binom{n}{m} \pmod p$ 等价于对应位上的组合数连乘。只要某一位上出现 $m_i > n_i$，直接返回 0。
2. **勒让德定理 (Legendre's Formula)：**
   - **用途：**计算 $n!$ 质因数分解后，质数 p 的指数。
   - **结论：**指数 $cnt = \sum_{i=1}^{\infty} \lfloor \frac{n}{p^i} \rfloor$。代码中通常写成 `while(n) cnt += n / p, n /= p;`。这是扩展卢卡斯中提纯因子的核心。
3. **库默尔定理 (Kummer's Theorem)：**
   - **结论：**$\binom{n}{m}$ 中包含质数 p 的个数，等于在 p 进制下计算 $m + (n - m)$ 时**发生进位的次数**。
   - **奇偶性推论：**$\binom{n}{m}$ 为奇数（即 $\bmod 2 \neq 0$）的充要条件是 $n \ \& \ m == m$（在二进制下 m 是 n 的子集）。
4. **扩展卢卡斯定理 (exLucas)：**
   - **前提条件：**模数 P 是**合数**（不一定是质数）。
   - **核心思想：**大一统缝合怪。先用**唯一分解定理**把 P 拆成若干个 $p_i^{k_i}$ 的乘积；然后分别求出 $\binom{n}{m} \pmod{p_i^{k_i}}$；最后用**中国剩余定理 (CRT)** 把所有结果拼起来。
   - **提纯操作：**在求 $\binom{n}{m} \pmod{p^k}$ 时，因为 p 的倍数没有逆元，必须用勒让德定理把 $n!, m!, (n-m)!$ 里所有的 p 因子全部抠出来，剩下互质的部分求逆元，最后再乘上 $p^{\Delta cnt}$。

File: 数论-卢卡斯定理-扩展.cpp

```
#include<bits/stdc++.h>
using namespace std;

#define int long long
#define endl '\n'

// --- 通用工具函数 ---
int qmi(int a,int k,int p)
{
    int res=1;
```

```
    a%=p;
    while(k)
    {
        if(k&1)res=res*a%p;
        a=a*a%p;
        k>>=1;
    }
    return res;
}

void exgcd(int a,int b,int &x,int &y)
{
    if(!b){x=1;y=0;return;}
    exgcd(b,a%b,y,x);
    y-=a/b*x;
}

int inv(int a,int p)
{
    int x,y;
    exgcd(a,p,x,y);
    return (x%p+p)%p; // 保证逆元为正
}


// ================= 1. 普通卢卡斯定理 (模数 p 必须是质数) =================
// 求解组合数 C(a, b) % p
int C(int a,int b,int p)
{
    if(a<b)return 0;
    int res=1;
    for(int i=1,j=a;i<=b;i++,j--)
    {
        res=res*j%p;
        res=res*qmi(i,p-2,p)%p; // 质数求逆元直接用费马小定理
    }
    return res;
}

int lucas(int a,int b,int p)
{
    if(a<p&&b<p)return C(a,b,p);
    return C(a%p,b%p,p)*lucas(a/p,b/p,p)%p;
}
/*  提示：如果有多组询问且 p <= 1e5，请把 C 函数改成预处理阶乘和阶乘逆元以做到 O(1) 查询。 */


// ================= 2. 扩展卢卡斯定理 (模数 P 可以是合数) =================
// 求解 n! % pk 孤立的部分 (即剔除掉所有 p 的倍数)
int fac(int n,int p,int pk)
{
    if(!n)return 1;
    int res=1;
    // 循环节部分
    for(int i=1;i<=pk;i++)
        if(i%p)res=res*i%pk;
    res=qmi(res,n/pk,pk);
    // 余数部分
    for(int i=1;i<=n%pk;i++)
        if(i%p)res=res*i%pk;
    return res*fac(n/p,p,pk)%pk; // 递归处理提取出来的 p 的倍数
}

// 求解组合数 C(n, m) % pk (pk = p^k)
int exC(int n,int m,int p,int pk)
{
    if(n<m)return 0;
    int f1=fac(n,p,pk),f2=fac(m,p,pk),f3=fac(n-m,p,pk);
    int cnt=0;
    // 勒让德定理：计算抠出来的 p 的总个数
    for(int i=n;i;i/=p)cnt+=i/p;
    for(int i=m;i;i/=p)cnt-=i/p;
    for(int i=n-m;i;i/=p)cnt-=i/p;

    // 互质部分相乘，因为此时互质，可以用 exgcd 求逆元
    return f1*inv(f2,pk)%pk*inv(f3,pk)%pk*qmi(p,cnt,pk)%pk;
}

int exlucas(int n,int m,int P)
{
    int res=0,tmp=P;
    // 唯一分解定理拆解模数 P
    for(int i=2;i*i<=tmp;i++)
    {
        if(tmp%i==0)
        {
            int pk=1;
            while(tmp%i==0)pk*=i,tmp/=i;
            // 求出在 pk 模下的答案 a
            int a=exC(n,m,i,pk);
            // CRT 中国剩余定理合并
            int Mi=P/pk;
            res=(res+a*Mi%P*inv(Mi,pk)%P)%P;
        }
    }
    // 扫尾处理最后一个大于 sqrt(P) 的质因子
    if(tmp>1)
    {
        int a=exC(n,m,tmp,tmp);
        int Mi=P/tmp;
        res=(res+a*Mi%P*inv(Mi,tmp)%P)%P;
    }
    return res;
```

```
}

void solve()
{
    /* 实战调用示例
    int n,m,p;
    cin>>n>>m>>p;
    // 如果 p 是质数
    // cout<<lucas(n,m,p)<<endl;
    // 如果 p 可能是合数
    // cout<<exlucas(n,m,p)<<endl;
    */
}
```

数论 - 中国剩余定理与扩展 (CRT & EXCRT)

算法说明

- **用途**：求解一元线性同余方程组 $x \equiv a_i \pmod{m_i}$。
- **CRT 核心思想**：构造法。对于每一个方程构造一个 ans_i，使得它在对 m_i 取模时余数为 a_i，对其他模数取模时余数为 0。最后全部加起来求和取总模数。**（前提：m_i 必须两两互质）**
- **EXCRT 核心思想**：合并法（数学归纳）。每次将两个方程通过 exgcd 合并成一个新的同余方程，直到只剩下一个方程。**（无互质限制，无解会返回 -1）**
- **避坑指南**：
  1. EXCRT 在合并过程中，中间变量相乘极容易产生大于 1e18 的数值导致溢出，实战中强烈建议用 __int128 强转。
  2. 无论加减乘，运算完一定要立刻 % M 并 (x % M + M) % M 变为正数，同余世界里负数是万恶之源。

File: 数论-扩展中国剩余定理.cpp

```
#include<bits/stdc++.h>
using namespace std;

#define int long long
#define endl '\n'

const int MAXN=1e5+10;

int a[MAXN]; // 余数数组 (x % m[i] = a[i])
int m[MAXN]; // 模数数组
int n;

// 扩展欧几里得算法 (求 ax + by = gcd(a,b))
int exgcd(int a,int b,int &x,int &y)
{
    if(!b){x=1;y=0;return a;}
    int d=exgcd(b,a%b,y,x);
    y-=a/b*x;
    return d;
}

// ---------------- 1. 普通中国剩余定理 (CRT) ----------------
// 要求：m[1]~m[n] 必须两两互质！
int CRT(int n,int *a,int *m)
{
    int M=1,ans=0;
    for(int i=1;i<=n;i++)M*=m[i];
    for(int i=1;i<=n;i++)
    {
        int Mi=M/m[i]; // 其他所有模数的乘积
        int x,y;
        exgcd(Mi,m[i],x,y); // 求 Mi 在模 m[i] 下的逆元 x
        x=(x%m[i]+m[i])%m[i]; // 保证逆元为正

        // ans = (ans + a[i] * Mi * x) % M
        // 为防溢出强转 __int128
        ans=(ans+(int)((__int128)a[i]*Mi%M*x%M))%M;
    }
    return (ans%M+M)%M;
}

// ---------------- 2. 扩展中国剩余定理 (EXCRT) ----------------
// 通杀所有情况，m[i] 不互质也能做，发生矛盾无解时返回 -1
int EXCRT(int n,int *a,int *m)
{
    int M=m[1],ans=a[1]; // 初始维护第一个方程
    for(int i=2;i<=n;i++)
    {
        int x,y;
        int A=M,B=m[i],C=(a[i]-ans%B+B)%B;
        int d=exgcd(A,B,x,y); // 解方程 Ax + By = C

        if(C%d!=0)return -1; // gcd 不能整除 C，说明方程组内部自相矛盾，无解

        int bg=B/d;
        // x = x * (C / d) % (B / d)，强转防爆 long long
        x=(int)((__int128)x*(C/d)%bg);
        x=(x%bg+bg)%bg;

        // ans = ans + x * M
        ans=(ans+(int)((__int128)x*M));
        M=M/d*B; // M 更新为 lcm(M, m[i])
        ans=(ans%M+M)%M;
    }
    return (ans%M+M)%M;
}

void solve()
```

```
{
    /* 实战调用示例
    cin>>n;
    for(int i=1;i<=n;i++)cin>>m[i]>>a[i]; // 注意题目给的读入顺序是 m 还是 a
    int ans=EXCRT(n,a,m);
    cout<<ans<<endl;
    */
}
```

数论 - 欧拉函数 & 扩展欧拉定理

核心结论与数学性质 (实战必背)

在算竞中，欧拉函数 $\phi(N)$ 不仅用来求“$1 \sim N$ 中与 N 互质的数的个数”，它的各种推论更是解数论题的题眼。以下结论务必牢记：

1. **质数性质**：若 p 为质数，则 $\phi(p) = p-1$。（显然 $1 \sim p-1$ 都与 p 互质）
2. **质数幂性质**：若 p 为质数，则 $\phi(p^k) = p^k - p^{k-1} = p^{k-1}(p-1)$。
3. **积性函数**：若 a, b 互质（即 $\gcd(a,b)=1$），则 $\phi(a \times b) = \phi(a) \times \phi(b)$。这是线性筛欧拉函数的核心基石。
4. **互质和推论**：$1 \sim N$ 中所有与 N 互质的数的**总和**为 $N \times \frac{\phi(N)}{2}$（当 $N > 1$ 时）。
5. **欧拉定理**：若 $\gcd(a,m)=1$，则 $a^{\phi(m)} \equiv 1 \pmod m$。
   - **特例（费马小定理）**：若 m 为质数，$\phi(m) = m-1$，即 $a^{m-1} \equiv 1 \pmod m$。
6. **扩展欧拉定理 (降幂公式)**：当指数极大（如 10^{100000}）无法用 `long long` 存储时，强制降维：

$$a^b \equiv a^{b \pmod{\phi(m)} + \phi(m)} \pmod m \quad (\text{前提: } b \geq \phi(m))$$

$$a^b \equiv a^b \pmod m \quad (\text{前提: } b < \phi(m))$$

算法说明

- **求单个数的 ϕ**：利用分解质因数，复杂度 $O(\sqrt{N})$。公式为 $N \times \prod(1 - \frac{1}{p})$。
- **求 $1 \sim N$ 的 ϕ 数组**：利用上述的**积性函数**结论配合线性筛，复杂度 $O(N)$。

File: 数论-欧拉定理扩展.cpp

```cpp
#include<bits/stdc++.h>
using namespace std;

#define int long long
#define endl '\n'

const int MAXN=1e6+10;

// --- 1. 求单个数的欧拉函数 O(sqrt(N)) ---
int get_phi(int n)
{
    int res=n;
    for(int i=2;i*i<=n;i++)
    {
        if(n%i==0)
        {
            // 发现质因数 i，利用公式：res = res * (1 - 1/i)
            res=res/i*(i-1);
            // 榨干这个质因数，保证以后找到的因数必定是质数
            while(n%i==0)n/=i;
        }
    }
    // n 最终没有被除尽，说明还有一个大于 sqrt(N) 的质因数
    if(n>1)res=res/n*(n-1);
    return res;
}

// --- 2. 线性筛求 1~N 所有的欧拉函数 O(N) ---
int primes[MAXN],cnt;
int phi[MAXN];
bool st[MAXN];

void get_phi_sieve(int n)
{
    phi[1]=1; // 1 的欧拉函数规定为 1
    for(int i=2;i<=n;i++)
    {
        if(!st[i])
        {
            primes[++cnt]=i;
            phi[i]=i-1; // 质数 p 的欧拉函数显然是 p-1
        }
        for(int j=1;primes[j]*i<=n;j++)
        {
            st[primes[j]*i]=true;
            if(i%primes[j]==0)
            {
                // 【核心逻辑 1】：primes[j] 已经是 i 的质因数了。
                // 根据欧拉函数质因数分解公式，新增加的这个 primes[j] 不会引入新的 (1 - 1/p) 项。
                // 所以直接纯乘进去即可。
                phi[primes[j]*i]=phi[i]*primes[j];
                break;
            }
            // 【核心逻辑 2】：primes[j] 是一个新的质因数。
            // 它是质数，由于 i 不包含它，两者互质，利用欧拉函数的积性：phi(a*b) = phi(a)*phi(b)
            // 即 phi[i * p] = phi[i] * phi[p] = phi[i] * (p - 1)
            phi[primes[j]*i]=phi[i]*(primes[j]-1);
        }
    }
}
```

```

// --- 3. 扩展欧拉定理 (处理超大指数降幂) ---
int qmi(int a,int k,int p)
{
    int res=1;
    a%=p; // 底数先取模，防爆 long long
    while(k)
    {
        if(k&1)res=res*a%p;
        a=a*a%p;
        k>>=1;
    }
    return res;
}

// 参数说明：底数 a，极大的指数字符串 b，模数 m
int ex_euler(int a,string b,int m)
{
    int p=get_phi(m); // 求出模数的欧拉函数 phi(m)
    int exp=0;
    bool flag=false;  // 标记真实的 b 是否 >= phi(m)

    // 边读入边取模，模拟高精度除法
    for(char c:b)
    {
        exp=exp*10+(c-'0');
        if(exp>=p)
        {
            flag=true; // 触发降幂条件
            exp%=p;
        }
    }

    // 【最致命的判断】：只有当指数 b 确实大于等于 phi(m) 时，指数才需要加上 phi(m)
    if(flag) exp+=p;

    // 最后套用普通的快速幂即可
    return qmi(a,exp,m);
}

void solve()
{
    /* 实战调用示例
    // 1. 求单个数
    // cout<<get_phi(12)<<endl;

    // 2. 线性筛
    // get_phi_sieve(1000000);

    // 3. 超大指数降幂：求 a^b % m
    // int a, m; string b;
    // cin>>a>>m>>b;
    // cout<<ex_euler(a, b, m)<<endl;
    */
}
```

数论 - 莫比乌斯反演

核心结论

- **莫比乌斯函数**：$\mu(1)=1$；含有平方质因子的数，$\mu(n)=0$；其余数若有 t 个不同质因子，则 $\mu(n)=(-1)^t$。
- **约数求和**：$\sum_{d|n}\mu(d)$ 在 $n=1$ 时为 1，在 $n>1$ 时为 0。例如 $\mu(6)=1$、$\mu(12)=0$。
- **反演公式**：若 $F(n)=\sum_{d|n}f(d)$，则 $f(n)=\sum_{d|n}\mu(d)F(n/d)$。

使用方法

- **初始化**：调用 `init(N)`，线性筛出 $1\sim N$ 的 `mu`，同时计算前缀和 `su`；`N` 为正整数，且至少覆盖后续调用中最大的 `min(n,m)`。多次查询可以统一预处理一次。
- **统计互质数对**：调用 `cal(n,m)`，返回满足 $1\le a\le n$、$1\le b\le m$、$\gcd(a,b)=1$ 的有序数对数量。例如 `cal(6,6)` 返回 23，其中 `(1,2)` 与 `(2,1)` 分别计数。
- **统计最大公约数为 k 的数对**：先令 `n/=k`、`m/=k`，再调用 `cal(n,m)`。代码中的 `solve()` 读入正整数 `n,m,k` 并输出结果；缩小后的任意范围为 0 时，直接输出 0。

```
init(min(n,m)); // 预处理本次互质计数所需的范围
int an=cal(n,m); // 统计两个范围内的互质有序数对
```

计数原理

利用约数求和性质，对每个数对枚举其公约数 d：互质时贡献 1，不互质时总贡献为 0。交换求和顺序后得到：

$$\mathrm{cal}(n,m)=\sum_{d=1}^{\min(n,m)}\mu(d)\left\lfloor\frac{n}{d}\right\rfloor\left\lfloor\frac{m}{d}\right\rfloor$$

固定区间左端点 `l`，令 `x=n/l`、`y=m/l`，则两个商同时不变的最远右端点为 `r=min(n/x,m/y)`。这一段的贡献合并为 `(su[r]-su[l-1])*x*y`，然后跳到 `r+1`。

复杂度

- **预处理**：时间 $O(N)$，空间 $O(N)$。
- **单次查询**：整除分块，时间 $O(\sqrt{n}+\sqrt{m})$，额外空间 $O(1)$；这里的 n,m 为传入 `cal` 的范围。

File: 数论-莫比乌斯反演.cpp

```
#include<bits/stdc++.h>
using namespace std;
#define int long long
#define endl '\n'
vector<int>mu,pri,su,st; // mu为莫比乌斯函数，pri存质数，su为mu的前缀和，st标记合数
void init(int n)
{
    mu.assign(n+1,0); // 初始化mu
    st.assign(n+1,0); // 初始化合数标记
    su.assign(n+1,0); // 初始化前缀和，su[0]=0
    pri.clear(); // 清空质数，支持重复调用
    mu[1]=1; // 特殊定义：mu(1)=1
```

```
    for(int i=2;i<=n;i++)
    {
        if(!st[i]) // 没有被更小的数筛到，说明i是质数
        {
            pri.push_back(i); // 记录质数，保持从小到大
            mu[i]=-1; // 只有一个质因子，所以mu(i)=-1
        }
        for(auto p:pri) // 用质数p筛出合数i*p
        {
            if(p>n/i)break; // 等价于i*p>n，同时避免乘法溢出
            st[i*p]=1; // 标记i*p为合数
            if(i%p==0) // i已经含有p，再乘p就含有p²
            {
                mu[i*p]=0; // 含有平方质因子，mu为0
                break; // 保证每个合数只由它的最小质因子筛一次
            }
            mu[i*p]=-mu[i]; // 新增一个不同质因子，正负翻转；原来为0则仍为0
        }
    }
    for(int i=1;i<=n;i++)su[i]=su[i-1]+mu[i]; // 区间mu之和为su[r]-su[l-1]
}
int cal(int n,int m) // 统计1<=a<=n、1<=b<=m且gcd(a,b)=1的有序数对
{
    int an=0,lim=min(n,m); // 公约数d不会超过两个范围的较小值
    for(int l=1,r;l<=lim;l=r+1) // 每次处理一段两个商都不变的区间
    {
        int x=n/l,y=m/l; // 当前区间内，两个范围中d的倍数分别有x、y个
        r=min(n/x,m/y); // 两个商分别不变到n/x、m/y，取较小的右端点
        an+=(su[r]-su[l-1])*x*y; // 合并这一段的mu[d]*(n/d)*(m/d)
    }
    return an; // 互质数对最终贡献1，不互质数对的容斥贡献抵消为0
}
void solve()
{
    int n,m,k;
    cin>>n>>m>>k; // 输入正整数n、m、k，要求gcd(a,b)=k
    n/=k; // 令a=k*x，则1<=x<=原来的n/k
    m/=k; // 令b=k*y，原条件等价于gcd(x,y)=1
    if(!n||!m) // 某个范围内没有正的k的倍数
    {
        cout<<0<<endl;
        return;
    }
    init(min(n,m)); // 只需预处理求和中会用到的mu
    cout<<cal(n,m)<<endl; // 输入6 6 1时，输出23
}
signed main()
{
    ios::sync_with_stdio(false);
    cin.tie(0);
    solve();
}
```

数论 - 整除分块 (数论分块)

算法说明

- **用途**：快速计算形如 $\sum_{i=1}^{n}\lfloor\frac{k}{i}\rfloor$ 的区间取整求和问题。
- **时间复杂度**：$O(\sqrt{n})$。
- **适用场景**：适用于 n 极大（如 10^9 级别）导致 $O(n)$ 遍历超时的情况。利用 $\lfloor\frac{k}{i}\rfloor$ 取值成块连续相同的特性，每次直接计算并跳过一整段区间，极大提升效率。常作为莫比乌斯反演、杜教筛的底层组件。

File: 整除分块.cpp

```
#include<bits/stdc++.h>
using namespace std;
void solve()
{
    int n;
    cin>>n;
    int ans=0;
    for(int l=0,r;l<=n;l=r+1)
    {
        r=n/(n/l);
        ans+=(r-l+1)*(n/l);
        l=r+1;
    }
}
```

数学 - 矩阵运算全家桶 (无重载 N*M 版)

算法说明

- **用途**：任意维度矩阵的加、减、乘，以及方阵的求逆。采用普通函数调用，逻辑清晰，不易与基本数据类型混淆。
- **复杂度**：乘法 $O(N\times M\times P)$，求逆 $O(N^3)$。
- **避坑**：
  - 调用 `mul(A, B)` 时，必须保证 `A.c == B.r`，否则乘法无意义（代码中默认保证合法，无断言以追求极限常数）。
  - 调用 `inv(A)` 时，`A` 必须是方阵。不可逆时返回空矩阵。

File: 矩阵 1 运算.cpp

```cpp

#include<bits/stdc++.h>
using namespace std;

#define int long long
#define endl '\n'

const int MOD=1e9+7;
const int SZ=105; // 矩阵最大边长

int qmi(int a,int k)
{
    int res=1;
    while(k)
    {
        if(k&1)res=res*a%MOD;
        a=a*a%MOD;
        k>>=1;
    }
    return res;
}

struct Mat
{
    int r,c; // r:行数, c:列数
    int m[SZ][SZ];

    Mat(int r=0,int c=0):r(r),c(c)
    {
        memset(m,0,sizeof(m));
    }
    void eye() // 生成单位矩阵 (必须在方阵下调用)
    {
        for(int i=1;i<=r;i++)m[i][i]=1;
    }
};

// 1. 矩阵加法 (要求 A.r == B.r && A.c == B.c)
Mat add(Mat a,Mat b)
{
    Mat res(a.r,a.c);
    for(int i=1;i<=a.r;i++)
        for(int j=1;j<=a.c;j++)
            res.m[i][j]=(a.m[i][j]+b.m[i][j])%MOD;
    return res;
}

// 2. 矩阵减法 (要求 A.r == B.r && A.c == B.c)
Mat sub(Mat a,Mat b)
{
    Mat res(a.r,a.c);
    for(int i=1;i<=a.r;i++)
        for(int j=1;j<=a.c;j++)
            res.m[i][j]=(a.m[i][j]-b.m[i][j]+MOD)%MOD;
    return res;
}

// 3. 矩阵乘法 (要求 A.c == B.r, 结果维度为 A.r * B.c)
Mat mul(Mat a,Mat b)
{
    Mat res(a.r,b.c);
    for(int i=1;i<=a.r;i++)
        for(int k=1;k<=a.c;k++)
        {
            if(!a.m[i][k])continue; // 常数级剪枝
            for(int j=1;j<=b.c;j++)
                res.m[i][j]=(res.m[i][j]+a.m[i][k]*b.m[k][j])%MOD;
        }
    return res;
}

// 4. 矩阵求逆 (要求 A 必须是方阵 A.r == A.c)
Mat inv(Mat a)
{
    int n=a.r;
    Mat res(n,n);
    res.eye();

    for(int i=1;i<=n;i++)
    {
        int pivot=i;
        for(int j=i;j<=n;j++)if(a.m[j][i]){pivot=j;break;}

        if(!a.m[pivot][i])return Mat(0,0); // 不可逆返回空矩阵

        if(pivot!=i)
        {
            for(int j=1;j<=n;j++)
            {
                swap(a.m[i][j],a.m[pivot][j]);
                swap(res.m[i][j],res.m[pivot][j]);
            }
        }

        int iv=qmi(a.m[i][i],MOD-2);
        for(int j=1;j<=n;j++)
        {
            a.m[i][j]=a.m[i][j]*iv%MOD;
            res.m[i][j]=res.m[i][j]*iv%MOD;
        }

        for(int j=1;j<=n;j++)
        {
```

```
            if(i==j||!a.m[j][i])continue;
            int factor=a.m[j][i];
            for(int k=1;k<=n;k++)
            {
                a.m[j][k]=(a.m[j][k]-factor*a.m[i][k])%MOD;
                if(a.m[j][k]<0)a.m[j][k]+=MOD;

                res.m[j][k]=(res.m[j][k]-factor*res.m[i][k])%MOD;
                if(res.m[j][k]<0)res.m[j][k]+=MOD;
            }
        }
    }
    return res;
}

/*  使用提示：
计算 (A + B) * C：
Mat ans = mul(add(A, B), C);

计算 A / B：
Mat ans = mul(A, inv(B));
*/
```

矩阵与递推 - 矩阵快速幂

算法说明

- **用途**：快速求解线性递推数列的第 n 项（如斐波那契数列、方案数统计等），以及图论中求解定长路径的方案数。
- **时间复杂度**：$O(M^3 \log k)$，其中 M 为矩阵的大小（状态维度），k 为递推的项数或幂次。
- **适用场景**：适用于递推项数极大（如 $k \geq 10^{18}$）且状态转移呈线性的场景。纯全局数组写法避开了 `struct` 和运算符重载的开销，结合了极其关键的 `if(!A[i][k]) continue;` 稀疏矩阵常数剪枝，运行极快。

File: 矩阵快速幂.cpp

```
#include<bits/stdc++.h>
using namespace std;
const int N = 4;
const int MOD  = 1e9+10;

int a[N][N];
int res[N][N];

void mul(int c[][N],int A[][N],int B[][N])
{
    int tmp[N][N]={0};
    for(int i=0;i<N;i++)
    {
        for(int j=0;j<N;j++)
        {
            if(!A[i][j])continue;
            for(int k=0;k<N;k++)
            {
                tmp[i][k]=(tmp[i][k]+A[i][j]*B[j][k])%MOD;
            }
        }

    }
    memcpy(c,tmp,sizeof tmp);
}

void qmi(int k)
{
    memset(res,0,sizeof res);
    for(int i=0;i<N;i++)res[i][i]=1;
    while(k)
    {
        if(k&1)mul(res,res,a);
        mul(a,a,a);
        k>>=1;
    }
}
//使用方法
int main()
{
    //1初始化底数，使得A*n=n+1
    a[0][0]=1;a[0][1]=1;
    a[1][0]=1;
    a[1][1]=0;
    qmi(10);
    cout<<res[0][0]<<endl;
    return 0;
}
```

数据结构 - 异或线性基 (XOR Linear Basis)

算法说明

- **用途**：处理异或空间问题。能在 $O(\log V)$ 时间内插入元素，支持查询原数组子集的最大异或和、最小异或和、第 k 小异或和，以及判断某个数能否被子集异或表示。
- **核心性质**：线性基中的元素互相异或绝对得不到 0；`d[i]` 存在时，其二进制最高位必然是第 i 位。
- **避坑指南**：求第 k 小前**必须**调用 `rebuild()` 进行高斯消元对角化；判断位运算时必须写 `(x>>i)&1`。

File: 线性代数-线性基.cpp

```
#include<bits/stdc++.h>
using namespace std;

#define int long long
#define endl '\n'

const int MAXB=62; // 值域上限，1e18 对应 62

int d[65]; // 线性基主数组
```

```cpp
int p[65]; // 重构后的数组
int cnt;   // 重构后的有效基底个数
bool zer;  // 标记原序列能否异或出 0

// 1. 插入元素
void insert(int x)
{
    for(int i=MAXB;i>=0;i--)
    {
        if(!((x>>i)&1))continue;
        if(!d[i])
        {
            d[i]=x;
            return ;
        }
        x^=d[i];
    }
    zer=1; // 如果被完全消掉，说明能异或出0
}

// 2. 判断 x 能否被线性基表示
bool check(int x)
{
    if(x==0)return zer;
    for(int i=MAXB;i>=0;i--)
    {
        if((x>>i)&1)x^=d[i];
    }
    return x==0;
}

// 3. 查最大异或和
int qmax()
{
    int res=0;
    for(int i=MAXB;i>=0;i--)
    {
        res=max(res,res^d[i]);
    }
    return res;
}

// 4. 查最小异或和 (非空子集)
int qmin()
{
    if(zer)return 0; // 如果能异或出0，最小就是0
    for(int i=0;i<=MAXB;i++)
    {
        if(d[i])return d[i]; // 找到最底层的基底就是大于0的最小值
    }
    return 0;
}

// 5. 线性基重构 (求第 k 小的前置操作)
void rebuild()
{
    cnt=0;
    for(int i=MAXB;i>=0;i--)
    {
        for(int j=i-1;j>=0;j--)
        {
            if((d[i]>>j)&1)d[i]^=d[j];
        }
    }
    for(int i=0;i<=MAXB;i++)
    {
        if(d[i])p[cnt++]=d[i];
    }
}

// 6. 查第 k 小异或和 (必须先 rebuild)
int kth(int k)
{
    if(zer)k--;
    if(k==0)return 0;
    if(k>=(1LL<<cnt))return -1; // 1LL防止爆int
    int res=0;
    for(int i=0;i<cnt;i++)
    {
        if((k>>i)&1)res^=p[i];
    }
    return res;
}
```

组合数学 - 卡特兰数 (Catalan Number)

算法说明

- **用途**：用于解决“在特定规则下，两种操作数量始终满足偏序关系”的计数问题。
- **时间复杂度**：预处理阶乘与逆元 $O(N)$，单次查询 $O(1)$。
- **适用场景**：题目模型可转化为两种互相抵消的操作（如 $+1$ 和 -1），且限制任意时刻的前缀和必须 ≥ 0 时。核心是 **容斥原理 + 折线法（翻转法）**，推导出公式 $C_n = \binom{2n}{n} - \binom{2n}{n-1} = \frac{1}{n+1}\binom{2n}{n}$。
- **常见模型**：
  1. **括号匹配**：n 对括号，合法的括号序列总数。
  2. **出栈序列**：长度为 n 的序列依次进栈，不同的合法出栈序列总数。
  3. **网格路径**：从 $(0,0)$ 到 (n,n)，只能向右或向上走，且不越过对角线 $y=x$ 的路径总数。
  4. **二叉树形态**：n 个结点的二叉树，共有多少种不同的形态。
  5. **多边形剖分**：凸 $n+2$ 边形通过对角线划分为三角形的方案数。
  6. **找零问题**：n 个人拿 50 元，n 个人拿 100 元排队买票，售票员不会找不开钱的排队方案数。

File: 组合数学-卡特兰数.cpp

```cpp
#include <iostream>
using namespace std;

#define int long long
using PII = pair<int, int>;

const int MOD = 1e9 + 7;
const int MAXN = 2e5 + 5; // 计算 C_n 需要用到 2n 的阶乘，注意开两倍大小

int fact[MAXN], invFact[MAXN];

int qpow(int a, int b){
    int res = 1;
    while(b){
        if(b & 1)res = res * a % MOD;
        a = a * a % MOD;
        b >>= 1;
    }
    return res;
}

void init(){
    fact[0] = 1;invFact[0] = 1;
    for(int i = 1;i < MAXN;i++)fact[i] = fact[i - 1] * i % MOD;
    invFact[MAXN - 1] = qpow(fact[MAXN - 1], MOD - 2);
    for(int i = MAXN - 2;i >= 1;i--)invFact[i] = invFact[i + 1] * (i + 1) % MOD;
}

int C(int n, int m){
    if(m<0 || m>n)return 0;
    return fact[n] * invFact[m] % MOD * invFact[n - m] % MOD;
}

int get_catalan(int n){
    if(n < 0)return 0;
    return C(2 * n, n) * qpow(n + 1, MOD - 2) % MOD;
}
// O(1) 求广义卡特兰数：n 个操作A，m 个操作B，保证任意时刻 A的出现次数 >= B的出现次数
int get_generalized_catalan(int n, int m){
    if(n < m) return 0; // B操作比A操作多，绝对无法满足限制

    // Ans = C(n+m, m) - C(n+m, m-1)
    int total = C(n + m, m);
    int invalid = C(n + m, m - 1);

    // 注意取模时的减法防负数处理
    return (total - invalid + MOD) % MOD;
}
void solve(){
    int n;
    cin >> n;
    cout << get_catalan(n) << "\n";
}

signed main(){
    ios::sync_with_stdio(false);cin.tie(nullptr);
    init();
    int t = 1;
    cin >> t;
    while(t--)solve();
    return 0;
}
```

组合数学 - 多重集组合数

算法说明

- **用途**：有 k 种物品，第 i 种最多选 n_i 个，求一共选 r 个的方案数。
- **时间复杂度**：$O(2^k \cdot k)$。
- **适用场景**：种类 k 极小（$k \le 20$），总数 r 极大（无法用多重背包 DP）时。核心是 **容斥原理** + **隔板法**。

File: 组合数学-多重组合数.cpp

```cpp
#include<bits/stdc++.h>>
using namespace std;
using namespace std;
#define int long long
const int MOD = 998244353;
const int N = 4e5+10;

int fa[N],infa[N];


int qmi(int a,int k)
{
    int res=1;
    while(k)
    {
        if(k&1)res=res*a%MOD;
        k>>=1;
        a=a*a%MOD;
    }
    return res;
}
void init()
{
    fa[0]=infa[0]=1;
    for(int i=1;i<N;i++)fa[i]=i*fa[i-1]%MOD;
    infa[N-1]=qmi(fa[N-1],MOD-2);
    for(int i=N-2;i>0;i--)infa[i]=infa[i+1]*(i+1)
```

```
    %MOD;
}

int C(int a,int b)
{
    if(a<0||b<0||b>a)return 0;
    return fa[a]*infa[b]%MOD*infa[a-b]%MOD;
}

int mulC(int k,int r,vector<int>&cn)
{
    int ans=0;
    for(int i=0;i<(1LL<<k);i++)
    {
        int lr=r;
        int pan=1;
        for(int j=0;j<k;j++)
        {
            if((i<<j)&1)
            {
                lr-=cn[j]+1;
                pan=-pan;
            }
        }
        if(lr<0)continue;
        ans=(ans+pan*C(lr+k-1,k-1)%MOD+MOD)%MOD;
    }
    return ans;
}
```

组合数学 - 广义卡特兰数

计数的问题

余额从 0 开始，安排下面两种操作的顺序：

- **A：加 1**，共 (k-1)*n 次。
- **B：减去 k-1**，共 n 次。
- **要求**：任何时刻余额都不能为负。同类操作不区分，问有多少种合法顺序。

当 k=2 时，两种操作就是 +1、-1，退化为普通卡特兰数。一般的答案为：

$$F_k(n)=\frac{1}{(k-1)n+1}\binom{kn}{n}$$

一个例子

n=2、k=3：4 次加 1、2 次减 2，合法顺序有 3 种：

```
AAAABB
AAABAB
AABAAB
```

例如 AABAAB 的余额依次为 1、2、0、1、2、0，始终非负；以 AB 开头则会变成 1、-1，不合法。

使用方法

```
init(6); // 预处理到最大k*n，范围小于固定质数模1e9+7
int a=catalan(2); // k默认是2，答案2
int b=catalan(2,3); // 对应上例，答案3
```

要求 n>=0、k>=2，结果对 1e9+7 取模。预处理 $O(M+\log mod)$，单次计算 $O(\log mod)$，空间 $O(M)$。

参考：广义卡特兰公式 (https://arxiv.org/abs/0711.0906)。

File: 组合数学-广义卡特兰数.cpp

```
#include<bits/stdc++.h>
using namespace std;
#define int long long
#define endl '\n'

const int mod=1e9+7;
vector<int>fac,inv; // 阶乘、阶乘的逆元

int qpow(int a,int b)
{
    int an=1;
    while(b)
    {
        if(b&1)an=an*a%mod;
        a=a*a%mod;
        b>>=1;
    }
    return an;
}

void init(int n) // 预处理到最大k*n，预处理范围小于mod
{
    fac.assign(n+1,1);
    inv.assign(n+1,1);
    for(int i=1;i<=n;i++)fac[i]=fac[i-1]*i%mod;
    inv[n]=qpow(fac[n],mod-2);
    for(int i=n;i>=1;i--)inv[i-1]=inv[i]*i%mod;
}

int C(int n,int m)
{
    if(m<0||m>n)return 0;
    return fac[n]*inv[m]%mod*inv[n-m]%mod;
}

int catalan(int n,int k=2) // 广义卡特兰数，n>=0、k>=2，提前预处理到k*n
{
    return C(k*n,n)*qpow((k-1)*n+1,mod-2)%mod; // C(k*n,n)/((k-1)*n+1)，k=2即普通卡特兰数
}
```

组合数学 - k 部分拆数 (k-Part Integer Partition)

算法说明

- **用途**：求将自然数 n 分拆为恰好 k 个正整数之和的方案

数（不考虑顺序）。

- **时间复杂度**：全局预处理 $O(n \cdot k)$，单次查询 $O(1)$。
- **核心推导**：将分拆方案分为两类：
  1. **至少包含一个 1**：将这个 1 拿走，等价于把 $n-1$ 拆分成 $k-1$ 个部分，方案数为 $p(n-1,k-1)$。
  2. **所有部分都 ≥ 2**：将这 k 个部分每个都减去 1（总共减去 k），剩下的每个数依然 ≥ 1，等价于把 $n-k$ 拆分成 k 个部分，方案数为 $p(n-k,k)$。
- **状态转移方程**：$p(n,k) = p(n-1,k-1) + p(n-k,k)$

File: 组合数学-拆分数.cpp

```cpp
#include <iostream>
#include <cstring>
using namespace std;

#define int long long
using PII = pair<int, int>;

const int MOD = 1000007;
const int MAXN = 10005;
const int MAXK = 1005;

int p[MAXN][MAXK]; // p[i][j] 表示将数字 i 拆成恰好 j 个部分的方案数

// 全局预处理，时间复杂度 O(N * K)
void init(){
    memset(p, 0, sizeof(p));
    p[0][0] = 1;
    for(int i = 1;i < MAXN;++i){
        for(int j = 1;j < MAXK;++j){
            if(i - j >= 0){
                // p[i-j][j]: 所有部分 >= 2
                // p[i-1][j-1]: 至少有一个部分是 1
                p[i][j] = (p[i - j][j] + p[i - 1][j - 1]) % MOD;
            }
        }
    }
}

void solve(){
    int n, k;
    // 多组输入处理
    if(cin >> n >> k){
        cout << p[n][k] << "\n";
    }
    else{
        exit(0);
    }
}
```

组合数学 - 互异分拆数 (Distinct Integer Partition)

算法说明

- **用途**：求将自然数 n 分拆为若干个**互不相同**的正整数之和的方案数。
- **时间复杂度**：利用分拆部分数的上限约为 $\sqrt{2n}$，时间复杂度降至 $O(n\sqrt{n})$。使用滚动数组优化，空间复杂度为 $O(n)$。
- **核心推导**：假设把 n 拆成 k 个互不相同的部分，将其从大到小排列 $r_1 > r_2 > \cdots > r_k \geq 1$。给这 k 个数**同时减去 1**（总共减去 k），它们依然严格递减：
  1. 如果原本最小的数 $r_k = 1$，减 1 后变成 0（消失了），剩下 $k-1$ 堆互不相同的数。方案数为 $pd(n-k,k-1)$。
  2. 如果原本最小的数 $r_k \geq 2$，减 1 后 ≥ 1（没消失），依然是 k 堆互不相同的数。方案数为 $pd(n-k,k)$。
- **状态转移方程**：$pd(n,k) = pd(n-k,k-1) + pd(n-k,k)$

File: 组合数学-拆分数互异.cpp

```cpp
#include <iostream>
#include <cstring>
using namespace std;

#define int long long
using PII = pair<int, int>;

const int MOD = 1000007;
const int MAXN = 50005;

// 使用滚动数组压缩 k 维度，pd[i][j&1] 表示将 i 拆为 j 个互异部分的方案数
int pd[MAXN][2];

void solve(){
    int n;
    if(!(cin >> n))exit(0);

    memset(pd, 0, sizeof(pd));
    pd[0][0] = 1;
    int ans = 0;

    // 互异分拆的极端情况是 1+2+...+k <= n。
    // 当 n=50000 时，k 的最大值约为 315，所以循环到 350 即可。
    for(int j = 1;j < 350;++j){
        // 每次进入新的一层 j，先清空当前层
        for(int i = 0;i <= n;++i)pd[i][j & 1] = 0;

        for(int i = 0;i <= n;++i){
            if(i - j >= 0){
                // pd[i-j][j&1]: 抽底后原本最小的数依然 >=1 (共 j 堆)
                // pd[i-j][(j-1)&1]: 抽底后原本最小的数变成 0 (剩 j-1 堆)
                pd[i][j & 1] = (pd[i - j][j & 1] + pd[i - j][(j - 1) & 1]) % MOD;
```

```
            }
        }
        // 将恰好拆成 j 个互异部分的方案数累加到最终答案
        ans = (ans + pd[n][j & 1]) % MOD;
    }
    cout << ans << "\n";
}
```

数学 - 斐波那契快速倍增法 (Fast Doubling)

算法说明

- **用途：**在 $O(\log N)$ 时间复杂度内，求出斐波那契数列的第 N 项对特定模数取模的结果，常数极小，全面碾压矩阵快速幂。
- **时间复杂度：**$O(\log N)$。
- **核心思想：**利用分治思想与恒等式 $F_{2k} = F_k(2F_{k+1} - F_k)$ 和 $F_{2k+1} = F_k^2 + F_{k+1}^2$，每次将所求项的下标减半，递归求解。

File: 组合数学-斐波那.cpp

```
#include<bits/stdc++.h>
using namespace std;
using PII = pair<int,int>;
#define int long long
const int MOD = 1e9+10;

PII fib(int n)
{
    if(n==0)return {0,1};
    auto p=fib(n>>1);
    int fk=p.first,fk1=p.second;
    int c=(2*fk1*fk%MOD-fk*fk%MOD+MOD)%MOD;
    int d=(fk*fk%MOD+fk1*fk1%MOD)%MOD;
    if(n&1)return {d,(c+d)%MOD};
    return {c,d};
}
//使用 fin(20)代表求第20项
```

组合数学 - 斯特林数 (Stirling Number)

算法说明

- **用途：**解决两类经典的元素划分排列问题，并在多项式推导中用于普通幂与阶乘幂的相互转化。
  1. **第二类斯特林数** $S(n,k)$ 或 $\left\{ {n \atop k} \right\}$：将 n 个**两两不同**的元素，划分到 k 个**互不区分**且**非空**的**集合**（分宿舍问题，不看顺序）。
  2. **第一类斯特林数** $s(n,k)$ 或 $\left[{n \atop k} \right]$：将 n 个**两两不同**的元素，划分成 k 个**互不区分**且**非空**的**轮换**（圆桌会议问题，旋转视为同一种，翻转视为不同）。
- **时间复杂度：**
  - **预处理 DP 表：**$O(N^2)$，适用于 $N, K \le 5000$ 的频繁查询。
  - **求单个第二类斯特林数：**$O(K \log N)$，适用于 N 极大（如 10^9），$K \le 10^5$ 的单次计算。

File: 组合数学-斯特林数.cpp

```
#include <iostream>
using namespace std;

#define int long long
using PII = pair<int, int>;

const int MOD = 1e9 + 7;
const int MAXN = 2e5 + 5; // 用于阶乘和单项容斥
const int MAX_DP = 5005;  // 用于 O(N^2) 预处理的二维表大小

int fact[MAXN], invFact[MAXN];
int S1[MAX_DP][MAX_DP]; // 第一类斯特林数二维表
int S2[MAX_DP][MAX_DP]; // 第二类斯特林数二维表

int qpow(int a, int b){
    int res = 1;
    while(b){
        if(b & 1)res = res * a % MOD;
        a = a * a % MOD;
        b >>= 1;
    }
    return res;
}

void init(){
    // 1. 初始化阶乘与逆元
    fact[0] = 1; invFact[0] = 1;
    for(int i = 1; i < MAXN; i++) fact[i] = fact[i - 1] * i % MOD;
    invFact[MAXN - 1] = qpow(fact[MAXN - 1], MOD - 2);
    for(int i = MAXN - 2; i >= 1; i--) invFact[i] = invFact[i + 1] * (i + 1) % MOD;

    // 2. 初始化斯特林数二维 DP 表
    S1[0][0] = 1;
    S2[0][0] = 1;
    for(int i = 1; i < MAX_DP; i++){
        for(int j = 1; j <= i; j++){
            // 第一类：新开一桌 + 坐前面(i-1)人中任意一人的左边
            S1[i][j] = (S1[i - 1][j - 1] + (i - 1) * S1[i - 1][j] % MOD) % MOD;
            // 第二类：新开一间房 + 挤进前面已有的 j 间房
            S2[i][j] = (S2[i - 1][j - 1] + j * S2[i - 1][j] % MOD) % MOD;
        }
    }
```

```
}

int C(int n, int m){
    if(m < 0 || m > n) return 0;
    return fact[n] * invFact[m] % MOD * invFact[n
     - m] % MOD;
}

// O(k log n) 容斥原理求单个第二类斯特林数 S(n, k)
// 允许 n 极大，k 需要 <= MAXN
int gets2(int n, int k){
    if(k < 0 || k > n) return 0;
    if(n == 0 && k == 0) return 1;

    int res = 0;
    for(int i = 0; i <= k; i++){
        int term = C(k, i) * qpow(i, n) % MOD;
        if((k - i) & 1){ // 奇数减
            res = (res - term + MOD) % MOD;
        }else{           // 偶数加
            res = (res + term) % MOD;
        }
    }
    return res * invFact[k] % MOD;
}

void solve(){
    int n, k;
    cin >> n >> k;
    // n, k 较小时可以直接查表 O(1)
    // cout << S2[n][k] << "\n";
    // n 极大时使用单次计算 O(k log n)
    cout << gets2(n, k) << "\n";
}
```

数学 - 范德蒙德卷积 (Vandermonde's Convolution)

解决什么实际问题 (实战映射)

在算竞的组合数学与期望 DP 中，当你遇到以下情况时，直接祭出范德蒙德卷积：

1. $O(N)$ **暴力求和化** $O(1)$ **的化简神器：**题目让你求一个长得像 $\sum C(n,i) \times C(m,k-i)$ 的式子，如果暴力循环求和会 TLE，利用范德蒙德卷积可以直接把它压缩成**一个组合数**，做到 $O(1)$ 极速求解。
2. **多项式系数合并：**在推导生成函数或多项式乘法 $(1+x)^n \times (1+x)^m$ 的某一项系数时，范德蒙德卷积是其在组合意义上的等价表达。
3. **"分堆选人"的组合背景：**例如"班里有 N 个男生 M 个女生，要选出 K 个人，求各种男女比例情况下的方案数总和"，利用它的组合意义瞬间秒杀。

核心结论与数学性质 (实战必背)

所有的范德蒙德卷积，本质上都是在讲一句话：**"从 n 里面选 i 个，再从 m 里面选 $k-i$ 个，穷举所有的 i，就等同于直接把 n 和 m 倒在一起，从 $n+m$ 里面选 k 个。"**

1. **基础公式 (标准范德蒙德卷积)：**
$$\sum_{i=0}^{k} \binom{n}{i}\binom{m}{k-i} = \binom{n+m}{k}$$
(注：当 $i>n$ 或 $k-i>m$ 时，组合数为 0，不影响等式成立)
2. **平方和推论 (极为高频)：**当 $n=m=k$ 时，配合 $\binom{n}{i}=\binom{n}{n-i}$，可以推导出组合数的平方和公式：
$$\sum_{i=0}^{n} \binom{n}{i}^2 = \sum_{i=0}^{n} \binom{n}{i}\binom{n}{n-i} = \binom{2n}{n}$$
实战映射：遇到 $\sum C(n,i)^2$，直接输出 $C(2n,n)$！
3. **错位平移推论 (进阶版)：**
$$\sum_{i=0}^{\min(n,m-k)} \binom{n}{i}\binom{m}{i+k} = \binom{n+m}{n+k}$$
推导技巧：利用对称性把第一项变成 $\binom{n}{n-i}$，此时上下标之和 $(n-i)+(i+k)=n+k$ 变为定值，即可套用基础公式。
4. **多项式意义证明：**考虑等式 $(1+x)^n \times (1+x)^m = (1+x)^{n+m}$。等式右边 x^k 的系数是 $\binom{n+m}{k}$。等式左边展开后，要凑出 x^k，必然是由前者的 x^i 和后者的 x^{k-i} 相乘得到。因此两边系数必须相等，得证。

File: 组合数学-范德蒙德卷积.cpp

```
#include<bits/stdc++.h>
using namespace std;

#define int long long
#define endl '\n'

const int MOD=1e9+7;
const int MAXN=2e6+10; // 空间注意开到 n+m 的上限

int fact[MAXN];    // fact[i]: i! % MOD
int infact[MAXN];  // infact[i]: (i!)^(-1) % MOD

// --- 快速幂求逆元 ---
int qmi(int a,int k,int p)
{
    int res=1;
    a%=p;
    while(k)
    {
        if(k&1)res=res*a%p;
        a=a*a%p;
        k>>=1;
    }
    return res;
}

// --- 预处理阶乘与阶乘逆元 O(N) ---
void init_comb(int n)
```

```
{
    fact[0]=infact[0]=1;
    for(int i=1;i<=n;i++)
    {
        fact[i]=fact[i-1]*i%MOD;
    }

    // 根据费马小定理，求 n! 的逆元
    infact[n]=qmi(fact[n],MOD-2,MOD);

    // 倒推预处理所有的阶乘逆元：1/(i-1)! = 1/i! * i
    for(int i=n-1;i>=1;i--)
    {
        infact[i]=infact[i+1]*(i+1)%MOD;
    }
}

// --- O(1) 组合数查询 ---
int C(int n,int m)
{
    // 不合法判断，极为重要
    if(n<0||m<0||n<m)return 0;

    // C(n, m) = n! / (m! * (n-m)!)
    int res=fact[n]*infact[m]%MOD*infact[n-m]%MOD;
    return res;
}

void solve()
{
    /* 实战调用示例
    // 必须在主程序最开头初始化，范围开到可能出现的最大下标 (n+m)
    init_comb(2000000);

    // 示例 1: 求 sum(i=0 to k) C(n, i) * C(m, k-i)
    int n=1000, m=2000, k=500;
    // 暴力求和复杂度 O(K) 甚至 O(N)，化简后直接 O(1) 秒杀：
    int ans1 = C(n + m, k);
    cout << ans1 << endl;

    // 示例 2: 求 sum(i=0 to n) C(n, i)^2
    // 结论：等于 C(2n, n)
    int n2=100000;
    int ans2 = C(2 * n2, n2);
    cout << ans2 << endl;
    */
}

signed main()
{
    cin.tie(0);
    ios::sync_with_stdio(0);
    int T=1;
    // cin>>T;
    while(T--) solve();
    return 0;
}
```

数学 - 皮萨诺周期 (Pisano Period) 与超大数降维

算法说明

- **用途**：当 N 是高精度超大数（如长度达 10^5 的数字串）时，求 $F_N \pmod m$。通过计算模 m 下的周期 P，将原问题强行降维为 $F_{N \bmod P} \pmod m$。
- **时间复杂度**：求周期 $O(\sqrt{m})$，大数取模 $O(|N|)$，最后倍增求值 $O(\log m)$。
- **核心思想 (周期分类查表法则)**：斐波那契通项公式中包含 $\sqrt{5}$。在模质数 p 的世界里，$\sqrt{5}$ 是否存在（二次剩余）决定了周期的长度。利用二次互反律，我们只需看 p 除以 5 的余数（即个位特征）来查表：
  1. **特例**：$p = 2$ 时基数为 3；$p = 5$ 时基数为 20。
  2. **第一派** ($p \pmod 5 = 1$ **或** 4)：即个位为 1 或 9。此时模 p 意义下存在 $\sqrt{5}$，公式正常运转，周期基数取 $p - 1$。
  3. **第二派** ($p \pmod 5 = 2$ **或** 3)：即个位为 3 或 7。此时模 p 意义下不存在 $\sqrt{5}$，进入二维扩域，周期基数取 $2(p + 1)$。

  将 m 分解质因数后，分别计算上述基数并求最小公倍数 (LCM)，即可得到总周期。

File: 组合数学-费波纳周期.cpp

```
#include<bits/stdc++.h>
using namespace std;
#define int long long
using PII = pair<int,int>;
int MOD;

PII fib(int n)
{
    if(n==0)return {0,1};
    auto p=fib(n>>1);
    int fk=p.first,fk1=p.second;
    int c=fk*((2*fk1-fk+MOD)%MOD)%MOD;
    int d=(fk*fk%MOD+fk1*fk1%MOD)%MOD;
    if(n&1)return {d,(c+d)%MOD};
    return {c,d};
}

int qpow(int a,int k)
{
    int re=1;
    while(k)
```

```
    {
        if(k&1)re=a*re;
        a=a*a;
        k>>=1;
    }
    return re;
}

int lcm(int a,int b)
{
    return a/__gcd(a,b)*b;
}

int peropd(int m)
{
    int re=1,tem=m;
    for(int i=2;i*i<=tem;i++)
    {
        if(tem%i==0)
        {
            int p=i,e=0;
            while(tem%i==0)
            {
                tem/=i;
                e++;
            }
            int cur=qpow(p,e-1);
            if(p==2)cur*=3;
            else if(p==5)cur*=20;
            else if(p%5==1||p%5==4)cur*=(p-1);
            else cur*=2*(p+1);
            re=lcm(re,cur);
        }
    }
    if(tem>1)
    {
        int p=tem;
        int cur=1;
        if(p==2)cur*=3;
        else if(p==5)cur*=20;
        else if(p%5==1||p%5==4)cur*=(p-1);
        else cur*=2*(p+1);
        re=lcm(re,cur);
    }
    return re;
}
/*使用 求周期P=peropd( MOD)
知道周期后
int n_mod = 0;
    for(char c : n_str) {
        n_mod = (n_mod * 10 + (c - '0')) % P;
    }
    cout<<fib(n_mod).first
*/
```

组合数学 - 错位排列 (Derangement)

算法说明

- **用途**：计算 n 个元素的全错位排列方案数（没有任何一个元素在自己原来的位置上）。
- **时间复杂度**：预处理 $O(N)$，单次查询 $O(1)$。
- **核心思想**：动态规划/递推。考虑第 n 个元素放在位置 k，而第 k 个元素是否放在位置 n，可划分为两个子问题。
- **核心公式**：

$$D_n = (n-1)(D_{n-1} + D_{n-2})$$

File: 组合数学-错排.cpp

```
#include<bits/stdc++.h>
using namespace std;
#define int long long
const int N = 1e6+10;
const int MOD = 1e9+7;
int d[N];
void pre()
{
    d[1]=0,d[2]=1;
    for(int i=3;i<N;i++)d[i]=(i-1)*(d[i-1]+d[i
    -2])%MOD;
}
```

数论 - 线性筛质数 (欧拉筛)

算法说明

- **用途**：快速筛选出 $1 \sim n$ 范围内的所有质数。
- **时间复杂度**：$O(n)$。
- **适用场景**：适用于 n 较大（如 10^7 级别）的情况。相比埃氏筛，它保证每个合数只被其**最小质因子**筛去一次，因此是线性的，效率极高。

File: 质数筛.cpp

```
const int N = 1e7 + 10;
int pri[N], cnt;
bool st[N];
void getpri(int n)
{
    for(int i = 2;i <= n;i++)
    {
        if(!st[i])pri[cnt++] = i;
        for(int j = 0;pri[j] <= n / i;j++)
        {
            st[pri[j] * i] = 1;
            if(i % pri[j] == 0)break;
        }
    }
}
```

数学 - 高斯消元 (高斯-若尔当消元法)

算法说明

- **用途**：求解 N 元一次线性方程组。
- **时间复杂度**：$O(N^3)$。
- **核心步骤**：
  1. 找主元：为了防止浮点数精度爆炸，必须在当前列找一个绝对值最大的数作为主元，并把它换到当前行。
  2. 判自由变元：如果当前列全是 0（小于 eps），说明该变量是自由的，直接跳过处理下一列。
  3. 消元：用当前行去消掉**其他所有行**的这一列，将其化为 0。
- **避坑指南**：消元结束后，如果秩（非全零行的数量）小于 N，需要进一步判断：如果等号左边全为 0，但等号右边不为 0，则**无解**；如果等号右边也全为 0，则有**无穷多解**。

File: 高斯消元.cpp

```cpp
#include<bits/stdc++.h>
using namespace std;

#define int long long
#define endl '\n'

const double eps=1e-8;
const int N=105; // 方程组的最大未知数个数

double a[N][N];  // 增广矩阵，1~n列为系数，n+1列为常数项
double ans[N];   // 存储唯一解时的答案

// 传入未知数个数 n
// 返回值: 0 表示唯一解, 1 表示无穷多解, -1 表示无解
int gauss(int n)
{
    int r=1; // r 记录当前正在构造对角线的第几行
    for(int c=1;c<=n;c++) // 枚举每一列
    {
        // 1. 找主元：在当前列中，找绝对值最大的行
        int pivot=r;
        for(int i=r+1;i<=n;i++)
        {
            if(fabs(a[i][c])>fabs(a[pivot][c]))pivot=i;
        }

        // 如果主元接近 0，说明这一列对应的变量是自由变量，跳过处理下一列
        if(fabs(a[pivot][c])<eps)continue;

        // 2. 将主元所在的行交换到当前处理行 r
        for(int j=c;j<=n+1;j++)swap(a[r][j],a[pivot][j]);

        // 3. 将其余行的当前列全部消为 0
        for(int i=1;i<=n;i++)
        {
            if(i==r)continue; // 留着自己不消
            if(fabs(a[i][c])>eps) // 常数优化，不为 0 才消
            {
                double factor=a[i][c]/a[r][c];
                for(int j=c;j<=n+1;j++)
                {
                    a[i][j]-=factor*a[r][j];
                }
            }
        }
        r++; // 当前行处理完毕，准星下移
    }

    // 4. 判解的情况
    if(r<=n)
    {
        // 说明出现了全零行，逐个检查常数项
        for(int i=r;i<=n;i++)
        {
            if(fabs(a[i][n+1])>eps)return -1; // 0 = 常数，矛盾，无解
        }
        return 1; // 0 = 0，无穷多解
    }

    // 5. 唯一解，得出最终答案
    for(int i=1;i<=n;i++)
    {
        ans[i]=a[i][n+1]/a[i][i];
    }
    return 0;
}

void solve()
{
    /* 实战调用示例：
    int n;
    cin>>n;
    for(int i=1;i<=n;i++)
        for(int j=1;j<=n+1;j++)
            cin>>a[i][j];

    int res=gauss(n);
    if(res==-1) cout<<"No Solution"<<endl;
    else if(res==1) cout<<"Infinite Group Solutions"<<endl;
    else
    {
        for(int i=1;i<=n;i++)
            cout<<fixed<<setprecision(2)<<ans[i]<<endl;
    }
    */
```

```
}
/*
###  实战小贴士
如果你遇到的题目要求在**模运算（整数域）**下解线性方程组（比如 `a[i][j]` 都是整数且要求模 $998244353$），只需要把这个模板里的：
1. `double` 改回 `int`。
2. 找主元时去掉 `fabs`。
3. 把除法 `factor = a[i][c] / a[r][c]` 替换成**乘法逆元**：`factor = a[i][c] * qmi(a[r][c], MOD-2) % MOD` 即可完美复用！
*/
```

数据结构

数据结构 - CDQ 分治 (三维偏序)

算法说明

- **用途:**统计每个三维点之前满足 $x_j \le x_i$、$y_j \le y_i$、$z_j \le z_i$ 的点数。
- **核心思想：**按照 (x, y, z) 字典序排序，在第一维下标上进行 CDQ 分治；第二维排序扫描，第三维使用树状数组维护前缀和。
- **复杂度：**每层分别排序左右区间，时间复杂度 $O(N \log^2 N)$，空间复杂度 $O(N)$。

核心流程

1. 按 (x, y, z) 排序，将完全相同的点合并，记录重复次数 `cnt`。
2. 对第三维 `z` 离散化。
3. `cdq(l,r)` 先递归左区间，再处理左区间对右区间的贡献。
4. 左右区间按 `y` 排序，用双指针加入 `y[left] <= y[right]` 的点。
5. 在树状数组中加入左侧点的 `cnt`，查询 `z[left] <= z[right]` 的数量。
6. 当前层结束后用相反增量撤销树状数组修改，再递归右区间。
7. 最终答案为 `ans + cnt`，并恢复到原输入顺序。

注意事项

- 初始排序必须是 `(x,y,z)` 字典序，否则相同 `x` 时可能漏掉合法贡献。
- 本模板是“小于等于”偏序，第二维和第三维都要保留等号。
- 树状数组维护计数，清除时应执行 `add(z,-cnt)`，不能直接把树状数组节点清零。
- 如果题目要求严格偏序、最大值转移或答案直方图，需要相应修改比较边界和树状数组维护内容。

使用方式

输入第一行是 `N`，接下来 `N` 行每行输入 `x y z`。程序按照原输入顺序逐行输出每个点的被支配点数量，包含点自身和完全相同的重复点。

File: cdq.cpp

```cpp
#include<bits/stdc++.h>
using namespace std;

#define int long long
#define endl '\n'

// 合并后的点：相同三元组只保留一个节点
struct Node
{
    int x,y,z;
    int cnt; // 当前三元组出现次数
    int ans; // 其他三元组对当前点的贡献
};

// 原始点：id 用于最后恢复输入顺序
struct Point
{
    int x,y,z,id;
};

vector<Node> a;
vector<int> p; // CDQ 中按第二维排序时保存下标
vector<int> tr; // 树状数组，维护第三维前缀和
int m; // 树状数组大小

// ================== 1. 树状数组 ==================

void add(int x,int v)
{
    for(;x<=m;x+=x&-x)
        tr[x]+=v;
}

// 查询第三维 <= x 的点数
int ask(int x)
{
    int ans=0;
    for(;x;x-=x&-x)
        ans+=tr[x];
    return ans;
}

// 当前 CDQ 层结束后，使用负值撤销 add 的修改
void clear_bit(int x,int v)
{
    add(x,-v);
}

// ================== 2. 排序比较 ==================

// 初始按照 (x,y,z) 字典序排序
```

```cpp
bool cmp_xyz(const Point &u,const Point &v)
{
    if(u.x!=v.x)return u.x<v.x;
    if(u.y!=v.y)return u.y<v.y;
    if(u.z!=v.z)return u.z<v.z;
    return u.id<v.id;
}

// CDQ 合并时按照第二维 y 排序
bool cmp_y(int x,int y)
{
    if(a[x].y!=a[y].y)return a[x].y<a[y].y;
    if(a[x].z!=a[y].z)return a[x].z<a[y].z;
    return x<y;
}

// ================== 3. CDQ 分治 ==================

void cdq(int l,int r)
{
    if(l>=r)return;

    int mid=(l+r)>>1;
    cdq(l,mid);

    // 左右区间分别按 y 排序，p[i] 保存合并点编号
    for(int i=l;i<=r;i++)p[i]=i;
    sort(p.begin()+l,p.begin()+mid+1,cmp_y);
    sort(p.begin()+mid+1,p.begin()+r+1,cmp_y);

    int j=l;
    for(int k=mid+1;k<=r;k++)
    {
        int id=p[k];

        // 第一维由 CDQ 区间保证，第二维用双指针处理
        while(j<=mid&&a[p[j]].y<=a[id].y)
        {
            int q=p[j++];
            add(a[q].z,a[q].cnt);
        }

        // 树状数组再限制第三维 z <= a[id].z
        a[id].ans+=ask(a[id].z);
    }

    // 树状数组必须按加入时的权值撤销，不能直接清空节点
    for(int k=l;k<j;k++)
    {
        int q=p[k];
        clear_bit(a[q].z,a[q].cnt);
    }

    cdq(mid+1,r);
}

void solve()
{
    // 方便以后改成多组数据时复用全局模板
    a.clear();
    p.clear();
    tr.clear();

    int n;
    if(!(cin>>n))return;

    vector<Point> q(n);
    for(int i=0;i<n;i++)
    {
        cin>>q[i].x>>q[i].y>>q[i].z;
        q[i].id=i;
    }

    // 1. 按三维字典序排序，并将相同点合并
    sort(q.begin(),q.end(),cmp_xyz);
    vector<int> belong(n);

    for(int i=0;i<n;)
    {
        int j=i+1;
        while(j<n&&q[j].x==q[i].x&&q[j].y==q[i].y&&q[j].z==q[i].z)
            j++;

        int id=a.size();
        a.push_back({q[i].x,q[i].y,q[i].z,j-i,0});
        for(int k=i;k<j;k++)belong[q[k].id]=id;

        i=j;
    }

    // 2. 离散化第三维
    vector<int> z;
    for(Node &v:a)z.push_back(v.z);
    sort(z.begin(),z.end());
    z.erase(unique(z.begin(),z.end()),z.end());

    for(Node &v:a)
        v.z=lower_bound(z.begin(),z.end(),v.z)-z.begin()+1;

    // 3. CDQ 计算不同三元组之间的贡献
    m=z.size();
    tr.assign(m+1,0);
    p.resize(a.size());
    if(!a.empty())cdq(0,a.size()-1);

    // 当前点的重复点也满足三维偏序，需要补上 cnt
```

```
    for(int i=0;i<n;i++)
        cout<<a[belong[i]].ans+a[belong[i]].cnt<<
    endl;
}

signed main()
{
    cin.tie(0);
    ios::sync_with_stdio(0);

    int T=1;
    // cin>>T;
    while(T--)solve();

    return 0;
}
```

数据结构 - ST 表 (RMQ)

算法说明

- **用途**：解决静态数组的“可重复贡献问题”区间查询（如最值、GCD 等），不支持修改。
- **时间复杂度**：
  - 预处理：$O(N \log N)$。
  - 单次查询：$O(1)$。
- **适用场景**：数组无动态修改且查询极频繁（$M \geq 10^6$）时，$O(1)$ 速度平替线段树。
- **注意事项**：
  1. `pre()` 预处理 `logn` 必须在 `main` 最开头仅调用一次，严防多测 TLE。
  2. 建表外层倍增循环界限写为 `(1<<j) <= n` 进行常数级剪枝。
  3. 查询区间 $[L, R]$ 时，步长 $s = \log n[R - L + 1]$，结果由两端覆盖：$\max(f[L][s], f[R - (1 \ll s) + 1][s])$。

File: st 表.cpp

```
#include<bits/stdc++.h>
using namespace std;

const int N = 2e5+10;
const int logN = 22;//LOG(n)
int f[N][logN],logn[N];

void pre()
{

    logn[1]=0;
    logn[2]=1;
    for(int i=3;i<N;i++)logn[i]=logn[i/2]+1;
}

void solve()
{
    int n;
    cin>>n;
    for(int i=1;i<=n;i++)cin>>f[i][0];
    pre();
    for(int j=1;(1<<j)<=n;j++)
    {
        for(int i=1;i+(1<<j)-1<=n;i++)
        {
            f[i][j]=max(f[i][j-1],f[i+(1<<(j-1))
    ][j-1]);
        }
    }
    int l,r;
    cin>>l>>r;
    int s=logn[r-l+1];
    cout<<max(f[l][s],f[r-(1<<s)+1][s])<<endl;
}
```

分块 - 区间加 & 区间秩查询

算法说明

- **用途**：在 $O(\sqrt{N} \log \sqrt{N})$ 的时间内维护序列的区间增量修改，并支持查询区间内小于特定值 x 的元素个数。
- **时间复杂度**：
  - 建块：$O(N \log \sqrt{N})$。
  - 区间修改：$O(\sqrt{N} \log \sqrt{N})$。
  - 区间查询：$O(\sqrt{N} \log \sqrt{N})$。
- **适用场景**：处理无法直接通过线段树或树状数组维护的非线性区间问题（如区间秩、区间众数、区间第 k 大），常用于处理带区间加法的统计问题。

File: 分块 1.cpp

```
#include <bits/stdc++.h>
using namespace std;

#define int long long
#define endl '\n'

const int N = 1e5 + 10, BSI = 400;

int a[N], sora[N], lazy[BSI];
int L[BSI], R[BSI], pos[N];
int n, q, blosize, blonum;
// 重构函数：散块修改后同步 sora 镜像并排序
void rebuild(int id)
{
    for (int i = L[id]; i <= R[id]; i++)sora[i] =
     a[i];
    sort(sora + L[id], sora + R[id] + 1);
}

// 操作 1：区间增量
void update(int l, int r, int x)
{
    int bl = pos[l], br = pos[r];
```

```
    if (bl == br)
    {
        for (int i = l; i <= r; i++) a[i] += x;
        rebuild(bl);
    }
    else
    {
        // 左散块处理
        for (int i = l; i <= R[bl]; i++) a[i] += x;
        rebuild(bl);
        // 中间整块处理标记
        for (int i = bl + 1; i <= br - 1; i++) lazy[i] += x;
        // 右散块处理
        for (int i = L[br]; i <= r; i++) a[i] += x;
        rebuild(br);
    }
}
// 操作 2：区间小于 x 计数
int query(int l, int r, int x)
{
    int bl = pos[l], br = pos[r];
    int ans = 0;
    if (bl == br)
    {
        for (int i = l; i <= r; i++)
        {
            if (a[i] + lazy[bl] < x) ans++;
        }
    }
    else
    {
        // 左右散块暴力匹配
        for (int i = l; i <= R[bl]; i++) if (a[i] + lazy[bl] < x) ans++;
        for (int i = L[br]; i <= r; i++) if (a[i] + lazy[br] < x) ans++;
        // 中间整块二分查找
        for (int i = bl + 1; i <= br - 1; i++)
        {
            int target = x - lazy[i];
            ans += lower_bound(sora + L[i], sora + R[i] + 1, target) - (sora + L[i]);
        }
    }
    return ans;
}

void solve()
{
    if (!(cin >> n >> q)) return;
    blosize = sqrt(n);
    blonum = (n - 1) / blosize + 1;
    for (int i = 1; i <= n; i++)
    {
        cin >> a[i];
        pos[i] = (i - 1) / blosize + 1;
        sora[i] = a[i];
    }
    // 初始化边界并对块内进行首次预排序
    for (int i = 1; i <= blonum; i++)
    {
        L[i] = (i - 1) * blosize + 1;
        R[i] = min(i * blosize, n);
        sort(sora + L[i], sora + R[i] + 1);
    }

    while (q--)
    {
        int op, l, r, x;
        cin >> op >> l >> r >> x;
        if (op == 1) update(l, r, x);
        else cout << query(l, r, x) << endl;
    }
}
```

分块 - 回滚莫队 (只增不删)

算法说明

- **用途**：处理容易加入、难以删除维护答案的离线区间查询，模板求区间内某个数的最大出现次数。
- **核心思想**：左端点按块排序，块内右端点升序。块右侧永久加入，块内部分临时加入；查询后回滚次数并恢复保存的答案，换块时清空。
- **使用方法**：填入 `a[1..n]`、`q[1..m]` 和原编号 `id`，`cnt` 初始为零，调用 `mo(n,m)` 后读取 `ans[id]`。数值用作 `cnt` 下标，必要时先离散化。
- **时间复杂度**：$O(m \log m + (n+m)\sqrt{n})$，其中单次加入、回滚为 $O(1)$。
- **空间复杂度**：模板固定数组为 $O(N)$。

File: 分块-回滚莫队.cpp

```
#include<bits/stdc++.h>
using namespace std;
#define int long long
#define endl '\n'

const int N=2e5+10;
int a[N],cnt[N],ans[N]; // 原数组、出现次数、询问答案，a中的值作为cnt下标
int len,an; // 分块长度、当前区间内某个数的最大出现次数

struct Q
{
    int l,r,id; // 询问区间[l,r]、询问原编号
}q[N];
```

```
void add(int x) // 加入位置x，最大出现次数只会增加
{
    cnt[a[x]]++;
    an=max(an,cnt[a[x]]); // 更新当前最大出现次数
}

void del(int x) // 回滚位置x，只恢复出现次数
{
    cnt[a[x]]--; // an通过之前保存的值恢复，不在这里重新计算
}

bool cmp(const Q &x,const Q &y)
{
    int u=(x.l-1)/len,v=(y.l-1)/len;
    if(u!=v)return u<v; // 左端点按块编号排序
    return x.r<y.r; // 同一块内右端点升序，保证右端点只向右移动
}

void mo(int n,int m) // a[1..n]、q[1..m]提前填好，cnt初始为0
{
    len=max(1LL,(int)sqrt(n));
    sort(q+1,q+m+1,cmp);
    int last=-1,ed=0,r=0; // 上一个块编号、当前块右边界、永久区间右端点
    for(int i=1;i<=m;i++)
    {
        int u=(q[i].l-1)/len; // 当前询问左端点所在块
        if(u!=last) // 换块后，重新建立永久区间
        {
            while(r>ed)del(r--); // 清空上一块的永久区间[ed+1,r]
            an=0; // 当前区间已清空，答案重新初始化
            ed=min(n,(u+1)*len); // 当前块的右边界
            r=ed; // 永久区间从[ed+1,ed]这个空区间开始
            last=u;
        }

        while(r<q[i].r)add(++r); // 永久加入块右侧的位置，同一块内不回退
        int re=an; // 保存永久区间的答案，供临时加入后恢复
        int t=min(ed,q[i].r); // 同块询问只加到右端点，跨块询问加到块边界

        for(int j=t;j>=q[i].l;j--)add(j); // 临时加入块内部分，与永久区间组成询问区间
        ans[q[i].id]=an; // 保存当前询问的答案
        for(int j=q[i].l;j<=t;j++)del(j); // 按加入的逆序回滚块内部分
        an=re; // 恢复永久区间的答案，准备下一个询问
    }
}
```

分块 - 莫队 (离线区间查询)

算法说明

- **用途**：处理静态数组的离线区间查询，模板维护区间内不同数的个数。
- **核心思想**：左端点按块排序，块内右端点奇偶交替排序；移动左右端点，通过 add/del 维护答案。
- **使用方法**：填入 a[1..n]、q[1..m]，设置询问原编号 id，调用 mo(n,m)，从 ans[id] 取答案。cnt、an 初始为零，数值用作 cnt 下标，必要时先离散化。
- **时间复杂度**：$O(m \log m + (n+m)\sqrt{n})$，其中单次 add/del 为 $O(1)$。
- **空间复杂度**：模板固定数组为 $O(N)$。

File: 分块-莫队.cpp

```
#include<bits/stdc++.h>
using namespace std;
#define int long long
#define endl '\n'

const int N=2e5+10;
int a[N],cnt[N],ans[N]; // 原数组、当前区间内每个值的出现次数、每个询问的答案，a中的值作为cnt下标
int len,an; // 分块长度、当前区间内不同数的个数

struct Q
{
    int l,r,id; // 询问区间[l,r]、询问的原始编号
}q[N];

void add(int x) // 将位置x加入当前区间，按题目修改维护逻辑
{
    if(cnt[a[x]]==0)an++; // 加入前没有这个值，不同数个数加1
    cnt[a[x]]++; // 这个值的出现次数加1
}

void del(int x) // 将位置x移出当前区间，按题目修改维护逻辑
{
    cnt[a[x]]--; // 这个值的出现次数减1
    if(cnt[a[x]]==0)an--; // 最后一次出现被移除，不同数个数减1
}

bool cmp(const Q &x,const Q &y) // 调整询问顺序，减少区间端点移动次数
{
    int u=(x.l-1)/len,v=(y.l-1)/len; // 两个询问的左端点所在块，块编号从0开始
```

```
    if(u!=v)return u<v; // 不同块按块编号升序排列
    if(u&1)return x.r>y.r; // 同一奇数块内，右端点降序排列
    return x.r<y.r; // 同一偶数块内，右端点升序排列，形成交替移动顺序
}

void mo(int n,int m) // 数组长度n、询问数量m，处理静态数组的离线区间查询
{
    len=max(1LL,(int)sqrt(n)); // 块长取根号n，至少为1
    sort(q+1,q+m+1,cmp); // a[1..n]、q[1..m]提前填好，q[i].id记录原编号
    int l=1,r=0; // 当前维护区间为[l,r]，初始为空，cnt和an需为0
    for(int i=1;i<=m;i++) // 按莫队顺序处理询问
    {
        while(l>q[i].l)add(--l); // 左端点向左移动，先移动再加入新位置
        while(r<q[i].r)add(++r); // 右端点向右移动，先移动再加入新位置
        while(l<q[i].l)del(l++); // 左端点向右移动，先移除旧位置再移动
        while(r>q[i].r)del(r--); // 右端点向左移动，先移除旧位置再移动
        ans[q[i].id]=an; // 当前区间已变为询问区间，按原编号保存答案
    }
}
```

数据结构 - 带删除并查集

算法说明

- **用途**：合并、查询、删除（将指定单点剥离为孤立点）。
- **复杂度**：均摊 $O(\alpha(N))$，极小常数。
- **核心思想：虚拟节点（替身术）**。真实节点 x 映射到虚拟节点 `id[x]`。删除 x 时，为其分配新编号 `id[x] = ++tot`，旧节点留在原树充当“空壳”维持连通性不断裂。
- **避坑**：每次删除必生新节点，**相关数组务必开到** $N+Q$（节点数 + 操作数），且初始化需覆盖至上限！

File: 并查集-删除集合单节点.cpp

```
#include<bits/stdc++.h>
using namespace std;

#define int long long
#define endl '\n'

// MAXN 必须开到 N + 最大删除操作次数
const int MAXN=5e5+10;

int fa[MAXN]; // 实际构建树结构的虚拟节点数组
int id[MAXN]; // 真实节点 x 映射到的虚拟节点编号
int sz[MAXN]; // 维护集合大小
int tot;      // 虚拟节点总数

// 1. 初始化
void init(int n)
{
    tot=n;
    // 注意：必须初始化到 MAXN，因为后续会不断分配新节点
    for(int i=1;i<MAXN;i++)
    {
        fa[i]=i;
        sz[i]=1;
        id[i]=i;
    }
}

// 2. 查 (带路径压缩，注意传进来的是虚拟节点)
int find(int x)
{
    return fa[x]==x?x:fa[x]=find(fa[x]);
}

// 3. 并
void merge(int x,int y)
{
    // 操作的必须是 x 和 y 当前对应的虚拟节点
    int fx=find(id[x]),fy=find(id[y]);
    if(fx!=fy)
    {
        fa[fx]=fy;
        sz[fy]+=sz[fx];
    }
}

// 4. 删 (把 x 从当前集合中剥离，变成孤立点)
void del(int x)
{
    int fx=find(id[x]);
    sz[fx]--;         // 原本所在的集合大小减 1

    id[x]=++tot;      // 给 x 分配一个新的虚拟节点作为新身份
    fa[id[x]]=id[x]; // 新身份自成一派
    sz[id[x]]=1;      // 新身份所在集合大小为 1
}
```

数据结构 - 带权并查集 (Weighted DSU)

算法说明

- **用途**：在维护连通性的基础上，动态维护节点间的相对关系（如 x 到根节点的距离、相对偏置或某种异或关系）。
- **复杂度**：均摊 $O(\alpha(N))$。
- **核心逻辑**：
  1. **路径压缩 (懒更新)**：在执行 `find(x)` 时，递归更新 `d`

[x]。通过 d[x] = d[x] + d[fa[x]]（这里的 + 可以是加法、异或或取模运算），将 x 到原父亲的关系转化为 x 到新根的关系。

2. **合并 (计算权值)**：当合并两个集合时，已知 x 与 y 的关系为 v，通过已有的 $d[x]$ 和 $d[y]$ 推导出两个根节点之间的权重。

- **避坑**：在 merge 时，必须严格按照向量合成公式推导根节点的更新，防止由于方向问题导致的权值错误。

File: 并查集-懒标记.cpp

```cpp
#include<bits/stdc++.h>
using namespace std;

#define int long long
#define endl '\n'

const int MAXN=2e5+10;

int fa[MAXN],d[MAXN]; // d[x] 维护 x 到 fa[x] 的权值

// 1. 初始化
void init(int n)
{
    for(int i=1;i<=n;i++)
    {
        fa[i]=i;
        d[i]=0; // 初始到自身的权值为 0
    }
}

// 2. 查 (带权路径压缩)
int find(int x)
{
    if(fa[x]==x)return x;
    int root=find(fa[x]); // 先递归找到根
    d[x]+=d[fa[x]];       // 懒标记下传：将父亲到根的权值累加到当前节点
    return fa[x]=root;    // 路径压缩
}

// 3. 并 (合并 x 和 y，并已知 x 到 y 的权值为 v)
// 即满足：d_actual[x] - d_actual[y] = v
void merge(int x,int y,int v)
{
    int fx=find(x),fy=find(y);
    if(fx!=fy)
    {
        fa[fx]=fy;
        // 向量推导：d[fx] = d[y] + v - d[x]
        d[fx]=d[y]+v-d[x];
    }
}

/*  使用提示：
计算 x 和 y 的相对权值（距离）：
if(find(x) == find(y))
    int dist = d[x] - d[y];
*/
```

数据结构/图论 - LCA 转 RMQ (欧拉序/DFN + ST 表)

何时使用

当题目需要**极其频繁地查询最近公共祖先 (LCA)** 时（例如查询量 M 达到 10^6 级别，或者需要构建**虚树**）。普通的倍增 LCA 查询复杂度是 $O(\log N)$，而欧拉序 +ST 表可以做到**预处理** $O(N \log N)$，**查询严格** $O(1)$。

核心步骤

1. **dfs (求欧拉序)**：不仅在向下走时记录节点，在**回溯时也要记录**。这样一棵 N 个节点的树会生成长度为 $2N-1$ 的欧拉序数组 seq。同时记录每个节点第一次出现的位置 first 和深度 dep。
2. **结论映射**：节点 u 和 v 的 LCA，就是欧拉序数组中，从 first[u] 到 first[v] 这段区间内，**深度最小的那个节点**。
3. **build_ST (建 ST 表)**：在欧拉序数组上跑一遍 ST 表算法，维护区间最小深度对应的节点。
4. **query_LCA**：取出 u 和 v 的 first 下标构成区间 $[L,R]$，直接 $O(1)$ 查 ST 表。

File: 树-公共祖先 dfn.cpp

```cpp
#include<bits/stdc++.h>
using namespace std;

#define int long long
#define endl '\n'

const int MAXN=5e5+10; // 注意：由于是欧拉序，ST表相关数组要开到两倍！

vector<int> e[MAXN];

// --- 欧拉序与 LCA 核心数组 ---
int dep[MAXN];          // dep[u]: 节点 u 的深度
int seq[MAXN * 2];      // seq[i]: 欧拉序中第 i 个访问的节点编号
int first[MAXN];        // first[u]: 节点 u 在欧拉序中第一次出现的下标
int tot;                // 欧拉序长度计数器

// --- ST 表 ---
int st[MAXN * 2][21];  // st[i][j]: 从下标 i 开始，长度为 2^j 的区间内，深度最小的【节点编号】
int lg[MAXN * 2];       // log2 预处理数组，用于 O(1) 查询

// ================== 1. DFS 求欧拉序 ==================
```

```
void dfs(int u, int f, int d)
{
    dep[u] = d;
    seq[++tot] = u;     // 第一次访问该节点，加入欧拉序
    first[u] = tot;     // 记录第一次出现的下标

    for(int v : e[u])
    {
        if(v == f) continue;
        dfs(v, u, d + 1);
        seq[++tot] = u; // 【核心】：从子节点回溯回来后，再次将自己加入欧拉序
    }
}

// ================== 2. 初始化 ST 表 ==================
// 比较两个节点哪个深度更浅，返回深度更浅的节点编号
int get_min(int u, int v)
{
    return dep[u] < dep[v] ? u : v;
}

void build_ST()
{
    // 预处理 log2 数组
    lg[1] = 0;
    for(int i = 2; i <= tot; i++) lg[i] = lg[i / 2] + 1;

    // 初始化 ST 表第 0 列 (长度为 2^0 = 1 的区间)
    for(int i = 1; i <= tot; i++) st[i][0] = seq[i];

    // 倍增递推 ST 表
    for(int j = 1; (1 << j) <= tot; j++)
    {
        for(int i = 1; i + (1 << j) - 1 <= tot; i++)
        {
            // 区间 [i, i + 2^j - 1] 的最小深度节点，等于两个半区间的最小深度节点的较浅者
            st[i][j] = get_min(st[i][j - 1], st[i + (1 << (j - 1))][j - 1]);
        }
    }
}

// ================== 3. O(1) 查询 LCA ==================
int query_lca(int u, int v)
{
    int L = first[u], R = first[v];
    if(L > R) swap(L, R); // 保证 L <= R

    int k = lg[R - L + 1];

    // 查询区间 [L, R] 的最小深度节点，拆分为两段长度为 2^k 的重叠区间
    return get_min(st[L][k], st[R - (1 << k) + 1][k]);
}

void solve()
{
    /* 实战调用示例
    int n, m, root;
    cin >> n >> m >> root;
    for(int i = 1; i < n; i++)
    {
        int u, v; cin >> u >> v;
        e[u].push_back(v);
        e[v].push_back(u);
    }

    // 1. 求欧拉序
    tot = 0;
    dfs(root, 0, 1);

    // 2. 建 ST 表
    build_ST();

    // 3. O(1) 极速响应查询
    while(m--)
    {
        int u, v; cin >> u >> v;
        cout << query_lca(u, v) << endl;
    }
    */
}
```

树 - 最近公共祖先 (LCA) - 倍增法

算法说明

- **用途**：在有根树中，快速查询两个节点的最近公共祖先。
- **时间复杂度**：
  - 预处理：$O(N \log N)$。
  - 单次查询：$O(\log N)$。
- **适用场景**：适用于 N, M 较大（如 $10^5 \sim 10^6$）且需要在线查询的情况。
- **注意事项**：
  1. `f` 数组第二维的大小由 N 决定，通常 $2^{19} > 5 \times 10^5$，所以开 `20` 足够。
  2. 利用 `de[0] = 0` (全局变量默认值) 作为哨兵，防止倍增跳越界。

File: 树-公共祖先 LCA.cpp

```
#include<bits/stdc++.h>
```

```cpp
using namespace std;

const int N = 500010;

vector<int> g[N];

int f[N][20];  // f[u][i] 表示 u 的第 2^i 个祖先
int de[N]; //深度数组，0号点深度为0作为哨兵
/**
 * DFS 预处理倍增数组和深度
 * @param u 当前节点
 * @param fa 父节点 (根节点调用时 fa 传 0)
 */
void dfs(int u, int fa) {
    de[u] = de[fa] + 1;
    f[u][0] = fa;

    // 倍增核心逻辑
    for(int i = 1; i < 20; i++) {
        f[u][i] = f[f[u][i - 1]][i - 1];
    }
    for(int v : g[u])
    {
        if(v == fa) continue;
        dfs(v, u);
    }
}

/**
 * 查询 LCA
 */
int lca(int a, int b) {
    if(de[a] < de[b]) swap(a, b);
    // 1. 将 a 跳到和 b 同一深度
    for(int i = 19; i >= 0; i--) {
        if(de[f[a][i]] >= de[b]) a = f[a][i];
    }
    if(a == b) return a;
    // 2. a 和 b 一起向上跳，直到父亲相同
    for(int i = 19; i >= 0; i--) {
        if(f[a][i] != f[b][i]) {
            a = f[a][i];
            b = f[b][i];
        }
    }
    return f[a][0]; // 返回父节点
}
int main()
{
    return 0;
}
```

数据结构 - 离散化权值树状数组 (支持查排名/第 k 小/前驱/后继)

算法说明

- **用途**：可以代替普通平衡树（如 Treap、Splay）的绝大多数功能。维护一个可重集，支持插入、删除、查询数值的排名、查询第 k 小、求前驱和后继。
- **核心思想**：先将所有可能出现的值离散化（映射为 $1 \sim N$ 的相对大小），然后将树状数组当作“桶”来用，`tr[i]` 维护的是离散化排名在某段区间内的数字一共出现了几次。结合树上倍增，可以将查询第 k 小的复杂度优化至 $O(\log N)$。
- **时间复杂度**：所有操作均在 $O(\log N)$ 内完成。常数极小，吊打普通平衡树

File: 树状数组-离散化.cpp

```cpp
#include<bits/stdc++.h>
using namespace std;

const int MAXN=2e5+10;
int tr[MAXN];
int b[MAXN]; // 离散化数组
int tot;     // 去重后的元素总数

// 1. 离散化 (传入原数组和长度)
void di(int *a,int n)
{
    for(int i=1;i<=n;i++)b[i]=a[i];
    sort(b+1,b+n+1);
    tot=unique(b+1,b+n+1)-(b+1);
}

// 2. 获取离散化后的排名 (1-based)
int getid(int v)
{
    return lower_bound(b+1,b+tot+1,v)-b;
}

int low(int x)
{
    return x&-x;
}

// 3. 增加/减少元素个数 (x为离散化后的排名，v为变化量，通常传1或-1)
void add(int x,int v)
{
    // 注意：这里是 +=v，绝对不能写成 +=x
    for(int i=x;i<=tot;i+=low(i))tr[i]+=v;
}

// 4. 查询排名 (查小于等于排名 x 的数字总个数)
int query(int x)
{
    int res=0;
    for(int i=x;i;i-=low(i))res+=tr[i];
    return res;
```

```
}

// 5. 树上倍增求第 k 小 (返回的是离散化后的排名 idx)
int kth(int k)
{
    int idx=0;
    for(int i=18;i>=0;i--)
    {
        int nx=idx+(1<<i);
        if(nx<=tot&&tr[nx]<k)
        {
            idx=nx;
            k-=tr[nx];
        }
    }
    return idx+1;
}

// 6. 查前驱 (严格小于 x 的最大元素的排名)
int pre(int x)
{
    // 先查有多少个数严格小于 x
    int cnt=query(getid(x)-1);
    if(cnt==0) return -1; // 不存在前驱
    return kth(cnt);
}

// 7. 查后继 (严格大于 x 的最小元素的排名)
int suf(int x)
{
    // 先查有多少个数小于等于 x
    int cnt=query(getid(x));
    if(cnt==query(tot)) return -1; // 不存在后继
    return kth(cnt+1);
}

/*  使用提示:
pre(x) 和 suf(x) 返回的都是离散化后的排名!
如果你想输出真实的原数值，请在主函数中套一层 b[]:
int ans = b[pre(x)];
*/
```

树 - 笛卡尔树

算法说明

- **用途**：将序列转化为同时满足“二叉搜索树（按下标）”和“堆（按权值）”性质的二叉树。
- **时间复杂度**：
  – 建树：$O(N)$。
- **适用场景**：将序列区间问题转化为树上问题（如 RMQ 转 LCA），或求解直方图最大矩形/子段问题。本质是单调栈的树形化。
- **注意事项**：
  1. 多组数据时**务必清空单调栈指针** `top = 0` 及左右儿子数组 `ls, rs`。
  2. 单调栈 `st` 中存储的是数组的**下标**，比较大小时需代入原数组判定（如 `ar[st[top]] > ar[i]`）。
  3. 线性建树完毕后，整棵树的根节点固定为栈底元素 `st[1]`。

File: 树-笛卡尔树.cpp

```
#include<bits/stdc++.h>
using namespace std;

const int N = 1e5 + 10;
vector<int>ar(N), ls(N), rs(N);
vector<int>st(N);
int top;
int build(int n)
{
    top = 0;
    for(int i = 1;i <= n;i++)ls[i] = rs[i] = 0;
    for(int i = 1;i <= n;i++)
    {
        int la = 0;
        while(top && ar[st[top]] > ar[i])
        {
            la = st[top];
            top--;
        }
        if(top)rs[st[top]] = i;
        ls[i] = la;
        st[++top] = i;
    }
    return st[1];
}
```

线段树 - 主席树 (可持久化权值线段树)

算法说明

- **用途**：查询静态数组区间第 k 小，重复值分别计数。
- **核心思想**：每个前缀建立一个版本，插入时复制路径，其余节点共用。两个前缀版本的数量相减，得到区间内的值域分布。
- **建立版本**：将数值离散化到 `1..s`，`rt[0]=0` 表示空树，依次执行 `rt[i]=update(rt[i-1],1,s,a[i])`。
- **查询方法**：`query(rt[l-1],rt[r],1,s,k)` 返回区间 `[l,r]` 第 k 小的离散化编号，再映射回原值，$1 \le k \le r-l+1$。
- **时间复杂度**：单次插入、查询为 $O(\log s)$，建立全部版本为 $O(n \log s)$。
- **空间复杂度**：$O(n \log s)$，模板预留 `N*20` 个节点。

File: 线段树-主席树.cpp

```
#include<bits/stdc++.h>
using namespace std;
#define int long long
#define endl '\n'
```

```

const int N=2e5+10;
int rt[N],tot; // rt[i]表示前i个数形成的版本，tot为已创建的节点数

struct T
{
    int ls,rs,sum; // 左儿子、右儿子、当前值域内的数的个数
}tr[N*20]; // 每次插入只新建根到叶子的一条链，零号节点表示空树

int update(int pre,int l,int r,int x) // 在旧版本pre上插入值x，返回新版本的根
{
    int p=++tot; // 新建节点，避免修改旧版本
    tr[p]=tr[pre]; // 复制旧节点的信息，未修改的子树继续共用
    tr[p].sum++; // 插入一个数，当前值域内的数量加1
    if(l==r)return p; // 到达值x对应的叶子，结束插入

    int mid=(l+r)>>1;
    if(x<=mid) // x在左半值域，只新建左侧路径
        tr[p].ls=update(tr[pre].ls,l,mid,x);
    else // x在右半值域，只新建右侧路径
        tr[p].rs=update(tr[pre].rs,mid+1,r,x);

    return p; // 返回新节点，供上一层连接
}

int query(int u,int v,int l,int r,int k) // 查询版本v减去版本u后，第k小的值
{
    if(l==r)return l; // 值域只剩一个值，它就是答案

    int mid=(l+r)>>1;
    int cnt=tr[tr[v].ls].sum-tr[tr[u].ls].sum; // 两个版本相减，得到查询区间落在左半值域的数量

    if(k<=cnt) // 左半值域已有至少k个数，第k小在左侧
        return query(tr[u].ls,tr[v].ls,l,mid,k);
    else // 跳过左侧cnt个数，寻找右侧第k-cnt小
        return query(tr[u].rs,tr[v].rs,mid+1,r,k-cnt);
}
```

数据结构 - 懒标记线段树

算法说明

- **用途**：维护一个长度为 N 的数组，支持单次将区间 $[L,R]$ 内的所有元素加上 v，以及查询区间 $[L,R]$ 内的最大值。
- **时间复杂度**：建树 $O(N)$，单次区间修改 $O(\log N)$，单次区间查询 $O(\log N)$。
- **适用场景**：需要频繁交替进行“区间修改”和“区间查询”时（普通差分数组无法处理边修改边查询的情况）。核心是 **分治思想 + 延迟更新（Lazy Tag）**。s

File: 线段树-懒标记线段树.cpp

```
#include<bits/stdc++.h>
using namespace std;
#define int long long
const int N = 1e5+10;

int w[N];
int tr[N<<2];
int add[N<<2];
void pushup(int u)
{
    tr[u]=max(tr[u<<1],tr[u<<1|1]);
}
void pushdown(int u,int l,int r)
{
    if(add[u])
    {
        tr[u<<1]+=add[u];
        tr[u<<1|1]+=add[u];
        add[u<<1]+=add[u];
        add[u<<1|1]+=add[u];
        add[u]=0;
    }
}

void build(int u,int l,int r)
{
    if(l==r)
    {
        tr[u]=w[l];
        return ;
    }
    int mid=l+r>>1;
    build(u<<1,l,mid);
    build(u<<1|1,mid+1,r);
    pushup(u);
}

void modify(int u,int l,int r,int L,int R,int v)
{
    if(L<=l&&r<=R)
    {
        tr[u]+=v;
        add[u]+=v;
        return ;
    }
    pushdown(u,l,r);
    int mid=l+r>>1;
    if(L<=mid)modify(u<<1,l,mid,L,R,v);
    if(R>mid)modify(u<<1|1,mid+1,r,L,R,v);
    pushup(u);
}
```

```
int query(int u,int l,int r,int L,int R)
{
    if(l>=L&&r<=R)return tr[u];
    pushdown(u,l,r);
    int mid=l+r>>1;
    int res=-4e18;
    if(L<=mid)res=max(res,query(u<<1,l,mid,L,R));
    if(R>mid)res=max(res,query(u<<1|1,mid+1,r,L,R
    ));
    return res;
}

// //1初始化原数组 1 2 3 4 5
// for(int i=1;i<=n;i++)w[i]=i;
// //2建树
// build(1,1,n);
// //3区间修改：给区间 [2,4] 加上 10
// modify(1,1,n,2,4,10);
// //4区间查询：输出区间 [1,5] 的max
// cout<<query(1,1,n,1,5)<<endl;
```

杂项

杂项 - 多进程模板

使用说明

1. 评测环境启动多个进程，并将当前进程的编号作为第一个命令行参数传入。模板在 `main()` 中用 `atoi(argv[1])` 读取编号，保存到全局变量 `id`。
2. 每个进程都会调用一次 `solve()`。根据题目要求，在函数中用 `id` 分配工作：模板示例将 `id == 0` 的逻辑和其他进程分开；若要区分其他进程，可继续判断 `id` 的值。
3. 按题目规定的协议进行输入、输出和进程间通信。需要立即发送输出时，调用 `cout.flush()`，或使用 `endl` 刷新缓冲区。

File: 多进程.cpp

```
#include<bits/stdc++.h>
using namespace std;

int id;

void solve()
{
    if(id==0)
    {
        // 编写进程 0 的逻辑
    }
    else
    {
        // 编写其他进程的逻辑；需要区分时可继续判断
    id
    }
}

int main(int argc,char *argv[])
{
    // 进程编号由启动程序时传入的第一个命令行参数提供
    id=atoi(argv[1]);
    solve();
    return 0;
}
```

数据结构 - 珂朵莉树 (ODT / Old Driver Tree)

算法说明

- **用途**：维护序列的区间操作。擅长处理具有**“区间推平（区间赋值）”**操作且数据有一定随机性的题目。
- **时间复杂度**：在有大量随机区间赋值操作的前提下，set 中的节点数会被极大地压缩，所有区间操作的均摊时间复杂度趋近于 $O(m \log n)$。若缺乏推平操作，最坏会退化到 $O(mn)$。
- **核心思想**：利用 `std::set` 维护一段段值相同的连续区间。核心操作只有两个：`split`（将一个大区间按指定位置劈开）和 `assign`（将一段区间内的所有碎块删掉，替换成一整块）。
- **避坑铁律**：在执行区间操作时，**必须先 `split(r + 1)`，然后再 `split(l)`**，否则可能会导致右侧的迭代器失效，引发 RE（段错误）。

File: 珂朵莉树.cpp

```
#include<bits/stdc++.h>
using namespace std;

struct ODT{
    int l,r;
    mutable int v;//允许我们在不删节点的情况下直接修改 set 内部元素的值
    bool operator<(const ODT& x)const{
        return l<x.l;
    }
};
set<ODT>odt;
auto split(int pos)
{
    auto it=odt.lower_bound({pos,0,0});
    if(it!=odt.end()&&it->l==pos)return it;
    --it;
    int l=it->l,r=it->r;
    int v=it->v;
    odt.erase(it);
    odt.insert({l,pos-1,v});
    return odt.insert({pos,r,v}).first;
}
void assign(int l,int r,int v)
{
```

```
    auto itr=split(r+1);
    auto itl=split(l);
    odt.erase(itl,itr);
    odt.insert({l,r,v});
}

void init(int n)
{
    odt.clear();
    odt.insert({1,n,0});
}

// 扩展：区间加法
void add(int l, int r, long long val) {
    auto itr = split(r + 1);
    auto itl = split(l);
    for (auto it = itl; it != itr; ++it) {
        it->v += val; // mutable 发挥作用，直接修改值
    }
}

// 扩展：区间统计 / 暴力遍历
// 统计区间 [l, r] 内有多少个满足某种条件的值
long long query(int l, int r) {
    auto itr = split(r + 1);
    auto itl = split(l);
    long long res = 0;
    for (auto it = itl; it != itr; ++it) {
        // it->r - it->l + 1 就是该碎块的长度
        // 可以在这里做任何暴力统计，比如求和：res += (it->r - it->l + 1) * it->v;
    }
    return res;
}
```

算法基础

算法 - 三分查找 (二分斜率/导数逼近)

算法说明

- **用途**：求单峰/单谷函数极值。
- **核心技巧**：
  - **浮点域**：取中点 m，向两侧偏移 $\epsilon/3$，比较两点大小直接砍掉一半区间。
  - **整数域**：直接比较 m 和 $m+1$（相当于整数域的最小偏移量），将三分降维成标准二分。

File: 三分.cpp

```
#include<bits/stdc++.h>
using namespace std;

#define int long long
#define endl '\n'

using LD = long double;

const LD eps = 1e-8;

// 目标函数 (以求单峰函数最大值为例)
LD f_d(LD x)
{
    // return ...
    return 0;
}

int f_i(int x)
{
    // return ...
    return 0;
}

// ---------------- 1. 浮点数极值 (偏移中点法) ----------------
LD ts_d(LD l, LD r)
{
    while(r-l>eps)
    {
        LD m=(l+r)/2.0;
        LD lm=m-eps/3.0, rm=m+eps/3.0;

        // 若左侧低于右侧，说明上升，极大值在右侧
        // 若求极小值，把 < 改成 >
        if(f_d(lm)<f_d(rm)) l=m;
        else r=m;
    }
    return l;
}


// ---------------- 2. 整数极值 (二分斜率法) ----------------
int ts_i(int l, int r)
{
    while(l<r)
    {
        int m=l+r>>1;
        // 直接比较 m 和 m+1 (相当于整数的微小偏移)
        // f(m) < f(m+1) 说明处于上升期，极值还在右边
        // 若求极小值，把 < 改成 >
        if(f_i(m)<f_i(m+1)) l=m+1;
        else r=m; // 下降期或平缓期，极值在 m 或 m 的左边
    }
    return l; // 返回极大值对应的 x 坐标
}
```

算法基础 - 点差分

算法说明

- **用途**：多次给树上一条路径的所有点加权，最后统一求每个点的增量。
- **标记**：令 p=lca(u,v)，两端各加 w，p 和 p 的父节点各减 w。
- **汇总**：所有修改结束后调用一次 collect()，子节点贡献累加给父节点，最终 d[u] 就是点 u 的增量。
- **复杂度**：预处理 $O(n \log n)$，单次修改 $O(\log n)$，最后汇总 $O(n)$。

使用与例子

建好邻接表 e 后调用 init()；代码已包含 LCA 预处理，fa[u][0] 为父节点，d 初始为 0。

```
  1
 / \
2   3
```

```
init(); // 已建好下面的小树
update(2,3,5); // 路径2→1→3上的点全部加5
collect(); // 修改全部结束后汇总，得到d[1]=d[2]=d[3]=5
```

为什么要减父节点？ LCA 本身需要保留一份；在其父节点再减一份，才不会继续影响路径以外的祖先。

CPP 输入 n,m、n-1 条边，再输入 m 组 u,v,w；输出各点的最终权值。

参考：OI Wiki：树上差分 (https://oi-wiki.org/basic/prefix-sum/#树上差分)。

File: 点差分.cpp

```cpp
#include<bits/stdc++.h>
using namespace std;
#define int long long
#define endl '\n'

const int N=2e5+10,L=20;
vector<int>e[N],ord; // 原树、父节点排在子节点之前的遍历顺序
int fa[N][L],dep[N],d[N]; // 倍增祖先、深度、差分标记，汇总后d保存最终增量

void init(int root=1) // 提前建好树，预处理祖先和遍历顺序
{
    ord.clear();
    ord.push_back(root);
    dep[root]=1;
    for(int j=0;j<L;j++)fa[root][j]=0; // 根的祖先为0号哨兵
    for(int i=0;i<(int)ord.size();i++) // 按遍历顺序展开，避免递归遍历长链
    {
        int u=ord[i];
        for(auto v:e[u])
        {
            if(v==fa[u][0])continue;
            fa[v][0]=u;
            dep[v]=dep[u]+1;
            for(int j=1;j<L;j++)fa[v][j]=fa[fa[v][j-1]][j-1];
            ord.push_back(v);
        }
    }
}

int lca(int u,int v) // 求u、v的最近公共祖先
{
    if(dep[u]<dep[v])swap(u,v);
    int dif=dep[u]-dep[v];
    for(int j=L-1;j>=0;j--)
        if(dif&(1LL<<j))u=fa[u][j]; // 先将两点提升到同一深度
    if(u==v)return u;
    for(int j=L-1;j>=0;j--)
    {
        if(fa[u][j]!=fa[v][j])
        {
            u=fa[u][j];
            v=fa[v][j];
        }
    }
    return fa[u][0];
}

void update(int u,int v,int w) // 将路径u到v上的每个点加w，包含两个端点
{
    int p=lca(u,v);
    d[u]+=w;
    d[v]+=w; // 两个端点分别向祖先贡献w
    d[p]-=w; // LCA被计算两次，减去一次
    d[fa[p][0]]-=w; // 抵消LCA父节点及以上的贡献，0号哨兵不作为答案
}

void collect() // 所有修改结束后调用一次，将子节点贡献累加给父节点
{
    for(int i=(int)ord.size()-1;i>0;i--) // 逆序保证先处理子节点，根无需再向上累加
    {
        int u=ord[i];
        d[fa[u][0]]+=d[u];
    }
}

void solve()
{
    int n,m;
    cin>>n>>m; // 点数、路径修改数量，初始点权均为0
```

```
    for(int i=1;i<n;i++)
    {
        int u,v;
        cin>>u>>v;
        e[u].push_back(v);
        e[v].push_back(u);
    }
    init();
    while(m--)
    {
        int u,v,w;
        cin>>u>>v>>w;
        update(u,v,w);
    }
    collect();
    for(int i=1;i<=n;i++)cout<<d[i]<<" "; // 输出每个点的最终权值
    cout<<endl;
}

signed main()
{
    ios::sync_with_stdio(0);
    cin.tie(0);
    solve();
    return 0;
}
```

算法基础 - 边差分

算法说明

- **用途**：多次给树上一条路径的所有边加权，最后统一求每条边的增量。
- **标记**：令 p=lca(u,v)，两端各加 w，p 减 2*w。
- **汇总**：所有修改结束后调用一次 collect()，最终 d[u] 表示边 (u,fa[u][0]) 的增量，根没有父边。
- **复杂度**：预处理 $O(n \log n)$，单次修改 $O(\log n)$，最后汇总 $O(n)$。

使用与例子

建好邻接表 e 后调用 init()；代码已包含 LCA 预处理，d 初始为 0，用较深的端点表示一条边。

```
  1
 / \
2   3
```

```
init(); // 已建好下面的小树
update(2,3,5); // 路径2→1→3上的两条边各加5
collect(); // 得到d[2]=5、d[3]=5，分别对应1—2、1—3；d[1]=0
```

为什么减两份？两端的贡献在 LCA 处汇合，需要全部抵消，因为 LCA 通向父节点的边不在路径上。u=v 时没有边，贡献为 0。

CPP 输入 n,m、n-1 条边，再输入 m 组 u,v,w；按道路输入顺序逐行输出最终边权。

参考：OI Wiki：树上差分 (https://oi-wiki.org/basic/prefix-sum/#树上差分)。

File: 边差分.cpp

```
#include<bits/stdc++.h>
using namespace std;
#define int long long
#define endl '\n'

const int N=2e5+10,L=20;
vector<int>e[N],ord; // 原树、父节点排在子节点之前的遍历顺序
int fa[N][L],dep[N],d[N]; // 倍增祖先、深度、差分标记，汇总后d保存最终增量

void init(int root=1) // 提前建好树，预处理祖先和遍历顺序
{
    ord.clear();
    ord.push_back(root);
    dep[root]=1;
    for(int j=0;j<L;j++)fa[root][j]=0; // 根的祖先为0号哨兵
    for(int i=0;i<(int)ord.size();i++) // 按遍历顺序展开，避免递归遍历长链
    {
        int u=ord[i];
        for(auto v:e[u])
        {
            if(v==fa[u][0])continue;
            fa[v][0]=u;
            dep[v]=dep[u]+1;
            for(int j=1;j<L;j++)fa[v][j]=fa[fa[v][j-1]][j-1];
            ord.push_back(v);
        }
    }
}

int lca(int u,int v) // 求u、v的最近公共祖先
{
    if(dep[u]<dep[v])swap(u,v);
    int dif=dep[u]-dep[v];
    for(int j=L-1;j>=0;j--)
        if(dif&(1LL<<j))u=fa[u][j]; // 先将两点提升到同一深度
    if(u==v)return u;
    for(int j=L-1;j>=0;j--)
    {
        if(fa[u][j]!=fa[v][j])
        {
            u=fa[u][j];
            v=fa[v][j];
        }
    }
```

```
    return fa[u][0];
}

void update(int u,int v,int w) // 将路径u到v上的每条边加w
{
    int p=lca(u,v);
    d[u]+=w;
    d[v]+=w; // 将边权记在较深的端点上，两端分别向祖先贡献w
    d[p]-=2*w; // LCA没有通向父亲的路径边，抵消在这里汇合的两份贡献
}

void collect() // 所有修改结束后调用一次，d[u]表示u到父节点的边权增量
{
    for(int i=(int)ord.size()-1;i>0;i--)
    {
        int u=ord[i];
        d[fa[u][0]]+=d[u]; // 从子节点向父节点累加，根不对应任何父边
    }
}

void solve()
{
    int n,m;
    cin>>n>>m; // 点数、路径修改数量，初始边权均为0
    vector<pair<int,int>>edges; // 保存道路输入顺序，最后按这个顺序输出
    for(int i=1;i<n;i++)
    {
        int u,v;
        cin>>u>>v;
        e[u].push_back(v);
        e[v].push_back(u);
        edges.push_back({u,v});
    }
    init();
    while(m--)
    {
        int u,v,w;
        cin>>u>>v>>w;
        update(u,v,w);
    }
    collect();
    for(auto [u,v]:edges)
    {
        int x=dep[u]>dep[v]?u:v; // 每条边的答案存在较深的端点上
        cout<<d[x]<<endl;
    }
}

signed main()
{
    ios::sync_with_stdio(0);
    cin.tie(0);
    solve();
    return 0;
}
```

基础算法 - 高精度运算 (BigInt)

算法说明

- **核心思路**：使用 `std::vector<int>` 或 `string` 逆序存储每一位数字（个位在下标 0）。逆序存储是为了在加法进位或乘法时，方便在 `vector` 末尾追加高位，而不需要频繁移动数组。
- **复杂度**：加减法 $O(N)$，乘法 $O(N \times M)$，除法 $O(N \times M)$。
- **优化建议**：对于极高要求的题目（位数上万），乘法需要使用 **FFT/NTT** 优化到 $O(N \log N)$，但基础比赛中使用模拟即可。

File: 高精度.cpp

```cpp
#include<bits/stdc++.h>
using namespace std;

#define int long long
#define endl '\n'

typedef vector<int> VI;

// 判断 A >= B
bool cmp(VI &A, VI &B) {
    if (A.size() != B.size()) return A.size() > B.size();
    for (int i = A.size() - 1; i >= 0; i -- )
        if (A[i] != B[i]) return A[i] > B[i];
    return true;
}

// 1. 高精度加法：C = A + B
VI add(VI &A, VI &B) {
    if (A.size() < B.size()) return add(B, A);
    VI C;
    int t = 0;
    for (int i = 0; i < A.size() || t; i ++ ) {
        if (i < A.size()) t += A[i];
        if (i < B.size()) t += B[i];
        C.push_back(t % 10);
        t /= 10;
    }
    return C;
}

// 2. 高精度减法：C = A - B (需保证 A >= B)
VI sub(VI &A, VI &B) {
    VI C;
    for (int i = 0, t = 0; i < A.size(); i ++ ) {
```

```
        t = A[i] - t;
        if (i < B.size()) t -= B[i];
        C.push_back((t + 10) % 10);
        if (t < 0) t = 1;
        else t = 0;
    }
    while (C.size() > 1 && C.back() == 0) C.
    pop_back(); // 去除前导0
    return C;
}

// 3. 高精度乘低精度：C = A * b
VI mul(VI &A, int b) {
    VI C;
    int t = 0;
    for (int i = 0; i < A.size() || t; i ++ ) {
        if (i < A.size()) t += A[i] * b;
        C.push_back(t % 10);
        t /= 10;
    }
    while (C.size() > 1 && C.back() == 0) C.
    pop_back();
    return C;
}

// 4. 高精度除以低精度：C = A / b, r 是余数
VI div(VI &A, int b, int &r) {
    VI C;
    r = 0;
    for (int i = A.size() - 1; i >= 0; i -- ) {
        r = r * 10 + A[i];
        C.push_back(r / b);
        r %= b;
    }
    reverse(C.begin(), C.end()); // 除法是从高位开始算的，存入后需翻转回逆序
    while (C.size() > 1 && C.back() == 0) C.
    pop_back();
    return C;
}

// 5. 高精度乘高精度：C = A * B (模拟竖式)
VI mul_huge(VI &A, VI &B) {
    VI C(A.size() + B.size() + 7, 0);
    for (int i = 0; i < A.size(); i++) {
        for (int j = 0; j < B.size(); j++) {
            C[i + j] += A[i] * B[j];
        }
    }
    int t = 0;
    for (int i = 0; i < C.size(); i++) {
        t += C[i];
        C[i] = t % 10;
        t /= 10;
    }
    while (C.size() > 1 && C.back() == 0) C.
    pop_back();
    return C;
}

void solve() {
    string s1, s2;
    cin >> s1 >> s2;
    VI A, B;
    for (int i = s1.size() - 1; i >= 0; i -- ) A.
    push_back(s1[i] - '0');
    for (int i = s2.size() - 1; i >= 0; i -- ) B.
    push_back(s2[i] - '0');

    // 加法示例
    VI C = add(A, B);
    for (int i = C.size() - 1; i >= 0; i -- )
    cout << C[i];
    cout << endl;
}

signed main() {
    cin.tie(0); ios::sync_with_stdio(0);
    solve();
    return 0;
}
```

计算几何

计算几何 - 二维点与向量基础 (2D Geometry)

算法说明

- **用途**：计算点与向量的距离、叉积、点积。所有二维几何（凸包、相交、面积）的地基。
- **复杂度**：$O(1)$。
- **核心思想**：全整数运算，拒用 double 和 sqrt，**彻底杜绝精度误差**。
- **叉积 cro**：$\vec{AB} \times \vec{AC}$。> 0 左转（逆时针）；< 0 右转（顺时针）；$= 0$ 共线。绝对值为平行四边形面积。
- **点积 dot**：$\vec{AB} \cdot \vec{AC}$。> 0 锐角；< 0 钝角；$= 0$ 垂直。

File: 二维点与向量基础.cpp

```
#include<bits/stdc++.h>
using namespace std;
#define int long long
#define endl '\n'

struct po {
    int x, y;
};

// 叉积: 向量AB × 向量AC (>0逆时针, <0顺时针, =0共线)
int cro(po a, po b, po c)
```

```
{
    return (b.x - a.x) * (c.y - a.y) - (c.x - a.x
    ) * (b.y - a.y);
}

// 点积: 向量AB · 向量AC (>0锐角, <0钝角, =0垂直)
int dot(po a, po b, po c)
{
    return (b.x - a.x) * (c.x - a.x) + (b.y - a.y
    ) * (c.y - a.y);
}

// 距离平方: 省略 sqrt 防止浮点误差
int dis(po a, po b)
{
    return (a.x - b.x) * (a.x - b.x) + (a.y - b.y
    ) * (a.y - b.y);
}

/* 避坑指南:
 * 若坐标绝对值达 1e9, 叉积/点积相乘会爆 long long。
 * 此时必须将返回值和内部强转为 __int128。
 */
```

计算几何 - 凸包 Graham 扫描法 (Convex Hull - Graham Scan)

算法说明

- **用途**：在二维平面的散点集中，求出能够包含所有点的最小凸多边形（凸包）。
- **时间复杂度**：基准点寻找 $O(N)$，极角排序 $O(N \log N)$，单调栈扫描 $O(N)$。总复杂度为 $O(N \log N)$。
- **适用场景**：求最大围栏周长/面积、最远点对（旋转卡壳的前置步骤）、二维防线构建等外边界轮廓问题。
- **核心思想**：
  1. **找起点**：选出 y 坐标最小（相同则取 x 最小）的点 P_1 作为绝对在凸包上的基准点。
  2. **极角排序**：将其余点按相对 P_1 的极角大小（利用叉积判左转）逆时针排序。
  3. **单调栈维护**：按序遍历排序后的点，若新点与栈顶两个点构成的折线**没有向左转**（即叉积 ≤ 0），说明产生了凹陷，必须不断**弹出栈顶**，直到满足左转为止，最后将新点入栈。

File: 凸包.cpp

```
#include<bits/stdc++.h>
using namespace std;
#define int long long
#define endl '\n'

const int N = 1e5 + 10;

struct po {
    int x, y;
};

po p[N];
int st[N], top = 0; // st 栈存放凸包上的点在原数组中的下标

// 叉积: 向量 AB 叉乘 向量 AC
int cro(po a, po b, po c)
{
    return (b.x - a.x) * (c.y - a.y) - (c.x - a.x
    ) * (b.y - a.y);
}

// 距离平方: 用于极角相同时的排序
int dis(po a, po b)
{
    return (a.x - b.x) * (a.x - b.x) + (a.y - b.y
    ) * (a.y - b.y);
}

// 极角排序比较函数
bool cmp(po a, po b)
{
    int cc = cro(p[1], a, b);
    if(cc > 0) return true;  // a 在 b 的顺时针前方（相对 p[1] 的极角更小）
    if(cc == 0) return dis(p[1], a) < dis(p[1], b); // 共线时，距离近的排前面
    return false;
}

void graham(int n)
{
    top = 0;
    // 点数 <= 3 时，全部都在凸包上
    if(n <= 3)
    {
        for(int i = 1; i <= n; i++) st[++top] = i;
        return;
    }

    // 1. 找 y 最小（且 x 最小）的点作为基准点
    int mn = 1;
    for(int i = 2; i <= n; i++)
    {
        if(p[i].y < p[mn].y || (p[i].y == p[mn].y && p[i].x < p[mn].x)) mn = i;
    }
    swap(p[1], p[mn]); // 将基准点换到第一个位置

    // 2. 将剩余点按相对 p[1] 的极角进行逆时针排序
    sort(p + 2, p + n + 1, cmp);

    // 3. 栈初始化，先压入前两个点
```

```
    st[++top] = 1;
    st[++top] = 2;

    // 4. 扫描后续点，维护单调栈
    for(int i = 3; i <= n; i++)
    {
        // 核心：若栈顶两点与新点构成的路径向右转或共线（叉积 <= 0），则栈顶出局
        while(top >= 2 && cro(p[st[top - 1]], p[st[top]], p[i]) <= 0)
        {
            top--;
        }
        st[++top] = i;
    }
}

/* 避坑指南：
 * 1. 共线处理：如果题目要求凸包边上的点也全部保留，遇到 cro == 0 时的出栈判定条件和排序逻辑需要特判修改。
 * 2. 坐标极值：如果点的坐标绝对值 >= 1e9，叉积计算过程中会爆 long long，必须转 __int128。
 */
```

计算几何 - 圆的交点与切线全家桶 (无重载运算符纯净版)

解决什么实际问题 (实战映射)

在二维计算几何中，一旦引入了“圆”，往往意味着涉及范围覆盖、最短雷达探测、以及极角排序。这套板子专门对付以下经典恶心场景：

1. **雷达/基站覆盖判定 (Circle-Circle / Line-Circle)**：求两个圆形探测器重叠的精确坐标，或者一架飞机（沿直线飞行）在雷达覆盖区域内飞行的准确入点和出点。
2. **光源/视线切点遮挡 (Point-Circle Tangents)**：一个人站在点 P 看向一个圆柱体障碍物，求视线的边界边缘（即从点 P 出发到圆的两条切线及切点坐标）。这是计算几何中“阴影面积”或“视野锥”问题的绝对核心。
3. **皮带传动/外围凸包 (Common Tangents)**：多个圆组成的机械齿轮或滑轮组，求包裹它们的最短皮带长度（利用圆与圆的外公切线，不过实战中单点求切线更为高频和基础）。

核心结论与数学性质 (实战必背)

处理圆的问题，**绝对不要去解联立二次方程组！**算竞中的终极武器是**极坐标与解三角形（三角函数）**。

1. **直线与圆的交点**：
   - 利用向量投影！求出圆心到直线的投影点（垂足），然后计算圆心到直线的距离 d。
   - 若 $d > r$ 相离；$d = r$ 相切；$d < r$ 相交。利用勾股定理求出垂足到交点的偏移长度 $L = \sqrt{r^2 - d^2}$，在直线上沿方向向量平移即可。
2. **圆与圆的交点 (余弦定理)**：
   - 设两圆心距离为 d。在由两圆心和交点构成的三角形中，三边长分别为 r_1, r_2, d。
   - 利用**余弦定理** $\cos\alpha = \frac{r_1^2+d^2-r_2^2}{2 \cdot r_1 \cdot d}$，求出两圆心连线与圆心到交点连线之间的夹角 α。
   - 把圆心连线的极角 θ 分别加上和减去 α，即可得到两个交点的极角，利用极坐标转直角坐标搞定。
3. **点到圆的切线点**：
   - 设点 P 到圆心 O 的距离为 d。在切点构成的直角三角形中，斜边为 d，直角边为 r。
   - 圆心角 $\alpha = \arccos(\frac{r}{d})$。求出 $\overrightarrow{OP}$ 的极角 θ，切点极角即为 $\theta \pm \alpha$。

File: 圆.cpp

```
#include<bits/stdc++.h>
using namespace std;

#define int long long
#define endl '\n'

const double eps=1e-9;
const double PI=acos(-1.0);

int dcmp(double x)
{
    if(fabs(x)<eps)return 0;
    return x<0?-1:1;
}

struct Point
{
    double x,y;
    Point(double x=0,double y=0):x(x),y(y){}
};
typedef Point Vector;

struct Circle
{
    Point c;
    double r;
    Circle(){}
    Circle(Point c,double r):c(c),r(r){}
};

// --- 纯函数式向量运算 ---
Vector add(Vector A,Vector B) { return Vector(A.x+B.x,A.y+B.y); }
Vector sub(Vector A,Vector B) { return Vector(A.x-B.x,A.y-B.y); }
Vector mul(Vector A,double p) { return Vector(A.x*p,A.y*p); }

double dot(Vector A,Vector B) { return A.x*B.x+A.y*B.y; }
```

```
double cross(Vector A,Vector B) { return A.x*B.y-A.y*B.x; }

// 向量长度与极角
double length(Vector A) { return sqrt(dot(A,A)); }
double angle(Vector A) { return atan2(A.y,A.x); } // 返回极角 [-PI, PI]

// 极坐标转直角坐标 (已知起点O，极长r，极角a)
Point point_from_polar(Point O,double r,double a)
{
    return Point(O.x+r*cos(a),O.y+r*sin(a));
}

// ================= 1. 直线与圆的交点 =================
// 直线由点 A 和 B 确定，返回交点个数，交点存入 p1, p2
int line_circle_intersection(Point A,Point B, Circle C,Point &p1,Point &p2)
{
    Vector v=sub(B,A);
    Vector CA=sub(A,C.c);

    // a, b, c 对应二次方程系数，也可理解为投影计算
    double a=dot(v,v);
    double b=2*dot(v,CA);
    double c=dot(CA,CA)-C.r*C.r;
    double delta=b*b-4*a*c;

    if(dcmp(delta)<0)return 0; // 相离
    if(dcmp(delta)==0) // 相切
    {
        double t=-b/(2*a);
        p1=p2=add(A,mul(v,t));
        return 1;
    }
    // 相交
    double t1=(-b-sqrt(delta))/(2*a);
    double t2=(-b+sqrt(delta))/(2*a);
    p1=add(A,mul(v,t1));
    p2=add(A,mul(v,t2));
    return 2;
}

// ================= 2. 圆与圆的交点 =================
// 返回交点个数，交点存入 p1, p2
int circle_circle_intersection(Circle C1,Circle C2,Point &p1,Point &p2)
{
    double d=length(sub(C1.c,C2.c));
    if(dcmp(d)==0)
    {
        if(dcmp(C1.r-C2.r)==0)return -1; // 完全重合，无数个交点
        return 0; // 同心圆但不重合，无交点
    }

    if(dcmp(C1.r+C2.r-d)<0)return 0; // 外离
    if(dcmp(fabs(C1.r-C2.r)-d)>0)return 0; // 内含

    // 核心：利用余弦定理求圆心角
    double base_angle=angle(sub(C2.c,C1.c)); // 连心线的极角
    // a 为 C1.c 到交点的连线 与 连心线 之间的夹角
    double a=acos((C1.r*C1.r+d*d-C2.r*C2.r)/(2*C1.r*d));

    p1=point_from_polar(C1.c,C1.r,base_angle+a);
    p2=point_from_polar(C1.c,C1.r,base_angle-a);

    if(dcmp(C1.r+C2.r-d)==0||dcmp(fabs(C1.r-C2.r)-d)==0)return 1; // 恰好相切
    return 2; // 相交
}

// ================= 3. 点到圆的切点 =================
// 从点 P 向圆 C 作切线，返回切点个数，切点存入 p1, p2
int point_circle_tangents(Point P,Circle C,Point &p1,Point &p2)
{
    Vector u=sub(P,C.c);
    double d=length(u);

    if(dcmp(d-C.r)<0)return 0; // 点在圆内，无切线
    if(dcmp(d-C.r)==0) // 点在圆上，只有1个切点就是它自己
    {
        p1=p2=P;
        return 1;
    }

    // 核心：构建以 C.c 为顶点，C.c->P 为斜边的直角三角形
    double base_angle=angle(u);
    double a=acos(C.r/d); // 圆心角的偏移量

    p1=point_from_polar(C.c,C.r,base_angle+a);
    p2=point_from_polar(C.c,C.r,base_angle-a);
    return 2;
}

void solve()
{
    /* 实战调用示例
    Circle C1(Point(0,0), 5);
```

```
    Circle C2(Point(8,0), 5);
    Point p1, p2;

    int cnt = circle_circle_intersection(C1, C2, p1, p2);
    if(cnt == 2)
    {
        cout << fixed << setprecision(6) << "圆交点1: " << p1.x << " " << p1.y << endl;
        cout << fixed << setprecision(6) << "圆交点2: " << p2.x << " " << p2.y << endl;
    }

    Point P(10, 0);
    cnt = point_circle_tangents(P, C1, p1, p2);
    if(cnt == 2)
    {
        cout << "切点1: " << p1.x << " " << p1.y << endl;
        cout << "切点2: " << p2.x << " " << p2.y << endl;
    }
    */
}

signed main()
{
    cin.tie(0);
    ios::sync_with_stdio(0);
    int T=1;
    // cin>>T;
    while(T--) solve();
    return 0;
}
```

数据结构 - 扫描线求矩形面积并

算法说明

- **用途**：求 N 个相互重叠的矩形在二维平面上覆盖的总面积。
- **时间复杂度**：$O(N \log N)$，主要开销在坐标离散化和线段树的区间修改上。
- **核心思想**:。将每个矩形拆分为左右两条垂直线段（左边标记为 `1`，右边标记为 `-1`）。用一根垂直的扫描线从左向右扫，遇到线段就更新线段树。线段树维护的是**被截取的有效高度**，每次推进的面积 = 当前有效高度 × 扫描线向右推进的距离。
- **避坑指南**：
  1. 线段树维护的是**线段（区间）**而不是**点**。如果离散化后 `Y` 坐标的下标为 L 和 R，则在线段树中修改的区间是 $[L, R-1]$。
  2. 叶子节点 `l == r` 实际代表的是长度为 `Y[r+1] - Y[l]` 的这一小段线段。

File: 扫描线.cpp

```
#include<bits/stdc++.h>
using namespace std;

#define int long long
#define endl '\n'

const int MAXN=2e5+10; // 注意：N个矩形有2N条边，数组必须开两倍

// 扫描线结构体
struct Line
{
    int x,y1,y2,mar;
}line[MAXN];

int Y[MAXN];       // 离散化数组
int tr[MAXN<<2];  // 维护区间有效覆盖长度
int cnt[MAXN<<2]; // 维护区间被完全覆盖的次数（Lazy Tag的变体，不需要pushdown）

bool cmp(Line a,Line b)
{
    return a.x<b.x; // 按横坐标从左到右排序
}

// 向上更新区间有效长度
void pushup(int u,int l,int r)
{
    if(cnt[u])tr[u]=Y[r+1]-Y[l]; // 如果被完全覆盖，长度就是真实的物理距离
    else if(l==r)tr[u]=0;        // 叶子节点且未被完全覆盖，长度为0
    else tr[u]=tr[u<<1]+tr[u<<1|1]; // 由左右子节点拼凑
}

// 区间修改 (L, R 为目标离散化区间边界)
void modify(int u,int l,int r,int L,int R,int v)
{
    if(L<=l&&r<=R)
    {
        cnt[u]+=v;
        pushup(u,l,r); // 更新有效长度
        return ;
    }
    int mid=(l+r)>>1;
    if(L<=mid)modify(u<<1,l,mid,L,R,v);
    if(R>mid)modify(u<<1|1,mid+1,r,L,R,v);
    pushup(u,l,r);
}

void solve()
{
```

```
    int n;
    cin>>n;
    int tot=0;
    for(int i=1;i<=n;i++)
    {
        int x1,y1,x2,y2;
        cin>>x1>>y1>>x2>>y2;

        line[++tot]={x1,y1,y2,1};  // 左边入边，权值为 1
        Y[tot]=y1;
        line[++tot]={x2,y1,y2,-1}; // 右边出边，权值为 -1
        Y[tot]=y2;
    }

    // 离散化 Y 坐标
    sort(Y+1,Y+tot+1);
    int num=unique(Y+1,Y+tot+1)-(Y+1);

    // 按 X 坐标排序垂直扫描线
    sort(line+1,line+tot+1,cmp);

    int ans=0;
    for(int i=1;i<tot;i++) // 最后一条边不需要再计算面积，所以 i < tot
    {
        // 映射到离散化下标，注意右边界 R 是 lower_bound 找到的下标减 1
        int L=lower_bound(Y+1,Y+num+1,line[i].y1)-Y;
        int R=lower_bound(Y+1,Y+num+1,line[i].y2)-Y-1;

        modify(1,1,num-1,L,R,line[i].mar);

        // 累加面积：线段树根节点维护的总有效高度 * 扫描线推进的横向距离
        ans+=tr[1]*(line[i+1].x-line[i].x);
    }
    cout<<ans<<endl;
}
```

计算几何 - 线段与射线交点全家桶 (无重载运算符纯净版)

解决什么实际问题 (实战映射)

在二维计算几何（坐标系、物理模拟、碰撞检测、地图视线）中，当你需要解决以下场景时，直接掏出这套板子：

1. **地图视线与障碍物判定 (Raycasting / Sight)**：NPC 从点 A 向某个方向看去，求这条**射线**是否能刺穿某条**墙壁线段**，或者求它击中的第一个交点。
2. **光线反射与折射 (Laser / Light physics)**：模拟一束激光（射线）击中一面镜子（线段），需要精准求出**交点坐标**以进行下一次的反射向量计算。
3. **多边形相交与切割 (Polygon Clipping)**：用一条直线把一个多边形切成两半，或者判断两个复杂的几何图案是否发生交叉碰撞（求**线段交点**）。
4. **车辆/弹道轨迹预测 (Trajectory Intersect)**：两辆车或两颗子弹在二维平面上做直线运动，已知其起点和速度方向（射线），求它们未来是否会相撞以及相撞的具体坐标。

核心结论与数学性质 (实战必背)

处理线段/射线问题，纯粹用解析几何的 $y = kx + b$ 会面临斜率不存在（垂直线）的巨大灾难。算竞中必须统一使用**向量积（叉积 Cross Product）**。

1. **叉积的几何意义**：向量 $\vec{A} \times \vec{B} = x_1y_2 - x_2y_1$。
   - 若结果 > 0，说明 $\vec{B}$ 在 $\vec{A}$ 的**逆时针**方向；
   - 若结果 < 0，说明 $\vec{B}$ 在 $\vec{A}$ 的**顺时针**方向；
   - 若结果 $= 0$，说明 $\vec{A}$ 与 $\vec{B}$ **共线**。
2. **参数方程统一交点公式**：
   - 任何直线/射线/线段都可以表示为 $P(t) = S + t \cdot \vec{v}$。
   - **直线**：$t \in (-\infty, +\infty)$
   - **射线**：$t \in [0, +\infty)$
   - **线段**：$t \in [0, 1]$
   - 通过推导，两条线的参数 t_1, t_2 可以直接通过叉积比值求出。求出 t 后，只需判断 t 的范围即可一张板子通杀**直线、射线、线段**的所有交点组合。

File: 线段射线.cpp

```
#include<bits/stdc++.h>
using namespace std;

#define int long long
#define endl '\n'

const double eps=1e-9;

// 浮点数比较助手 (消除精度误差)
int dcmp(double x)
{
    if(fabs(x)<eps)return 0;
    return x<0?-1:1;
}

struct Point
{
    double x,y;
    Point(double x=0,double y=0):x(x),y(y){}
};
typedef Point Vector;

// --- 纯函数式向量运算 (拒接重载运算符) ---
Vector add(Vector A,Vector B) { return Vector(A.x+B.x,A.y+B.y); }
Vector sub(Vector A,Vector B) { return Vector(A.x-B.x,A.y-B.y); }
```

```
Vector mul(Vector A,double p) { return Vector(A.x*p,A.y*p); }

// 叉积 (Cross Product): A.x*B.y - A.y*B.x
double cross(Vector A,Vector B) { return A.x*B.y-A.y*B.x; }
// 点积 (Dot Product): A.x*B.x + A.y*B.y
double dot(Vector A,Vector B) { return A.x*B.x+A.y*B.y; }

// 判定点 P 是否在线段 AB 上 (包含端点)
bool on_segment(Point P,Point A,Point B)
{
    // 结论：P 在 AB 上 <=> 向量PA与PB共线(叉积为0) 且 P 在 AB 之间(点积<=0)
    Vector PA=sub(A,P);
    Vector PB=sub(B,P);
    return dcmp(cross(PA,PB))==0&&dcmp(dot(PA,PB))<=0;
}

// ================= 核心：万能交点函数全家桶 =================
// type1 和 type2 的含义：0 代表直线，1 代表射线，2 代表线段
// P1, P2 定义第一条线（若为射线，P1为起点，方向为 P1->P2）
// P3, P4 定义第二条线（若为射线，P3为起点，方向为 P3->P4）
// res 存储求出的交点坐标
// 返回值：-1 代表平行/共线（无交点或无数交点），1 代表存在唯一交点
int get_intersection(Point P1,Point P2,int type1,Point P3,Point P4,int type2,Point &res)
{
    Vector v1=sub(P2,P1); // 线1的方向向量
    Vector v2=sub(P4,P3); // 线2的方向向量

    double denom=cross(v1,v2);

    // 1. 分母为 0，说明两方向向量平行或共线
    if(dcmp(denom)==0)return -1;

    // 2. 利用叉积公式计算参数 t1 和 t2
    // 线1: P(t1) = P1 + t1 * v1
    // 线2: P(t2) = P3 + t2 * v2
    Vector P3_P1=sub(P3,P1);
    double t1=cross(P3_P1,v2)/denom;
    double t2=cross(P3_P1,v1)/denom;

    // 3. 根据类型校验 t1 的合法性
    if(type1==1&&dcmp(t1)<0)return -1; // 射线，参数 t1 必须 >= 0
    if(type1==2&&(dcmp(t1)<0||dcmp(t1-1)>0))
    return -1; // 线段，参数 t1 必须在 [0,1]

    // 4. 根据类型校验 t2 的合法性
    if(type2==1&&dcmp(t2)<0)return -1; // 射线，参数 t2 必须 >= 0
    if(type2==2&&(dcmp(t2)<0||dcmp(t2-1)>0))
    return -1; // 线段，参数 t2 必须在 [0,1]

    // 5. 校验通过，计算具体交点坐标并赋给 res
    res=add(P1,mul(v1,t1));
    return 1;
}

void solve()
{
    /* 实战调用示例
    Point A(0,0),B(2,2); // 线1
    Point C(0,2),D(2,0); // 线2
    Point p;

    // 示例 A: 求线段 AB 和线段 CD 的交点 (传参都为 2)
    if(get_intersection(A,B,2,C,D,2,p)==1)
        cout<<fixed<<setprecision(6)<<"线段交点: "<<p.x<<" "<<p.y<<endl;
    else
        cout<<"线段无交点"<<endl;

    // 示例 B: 求从 A 出发经过 B 的【射线】，与从 C 出发经过 D 的【射线】的交点 (传参都为 1)
    if(get_intersection(A,B,1,C,D,1,p)==1)
        cout<<"射线交点: "<<p.x<<" "<<p.y<<endl;

    // 示例 C: 求直线 AB 和线段 CD 的交点 (传参分别是 0 和 2)
    if(get_intersection(A,B,0,C,D,2,p)==1)
        cout<<"直线与线段交点: "<<p.x<<" "<<p.y<<endl;
    */
}

signed main()
{
    cin.tie(0);
    ios::sync_with_stdio(0);
    int T=1;
    // cin>>T;
    while(T--) solve();
    return 0;
}
```

计算几何 - 简单多边形面积 (Polygon Area)

算法说明

- **用途**：计算任意给定顶点顺序（顺时针或逆时针）的简单多边形面积。不仅限于凸多边形，凹多边形同样适用。

- **时间复杂度**：$O(N)$。
- **核心思想：三角剖分与有向面积**。固定多边形的起点 ar[0]，将其与后续所有相邻的边 ar[i]-ar[i+1] 连成三角形。利用叉积求出带有正负号的三角形面积并累加。正负号会自动抵消凹多边形外部的无效面积，最终取绝对值除以 2 即可。

File: 计算多边形面积.cpp

```cpp
#include<bits/stdc++.h>
using namespace std;
#define int long long
#define endl '\n'

struct po {
    int x, y;
};

// 叉积: 向量AB × 向量AC (>0逆时针, <0顺时针, =0共线)
int cro(po a, po b, po c)
{
    return (b.x - a.x) * (c.y - a.y) - (c.x - a.x) * (b.y - a.y);
}

// 求多边形面积 (传入按顺序排好顶点的 vector)
double pore(const vector<po>& ar)
{
    if(ar.size() < 3) return 0;

    int x = 0;
    for(int i = 1; i < ar.size() - 1; ++i)
    {
        // 以 ar[0] 为原点进行剖分，累加有向面积
        x += cro(ar[0], ar[i], ar[i+1]);
    }

    return abs(x) / 2.0;
}
```